



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-206СВ-24 Направление подготовки 02.04.02 «Фундаментальная информатика и информационные технологии»

Магистерская программа Информатика

Квалификация: магистр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

На тему: Система автоматизированного обогащения товарных данных для интернет-магазина

Автор ВКРМ: _____ Фролов Михаил Александрович _____ ()

Научный руководитель: _____ Судаков Владимир Анатольевич _____ ()

Консультант: _____ ()

Консультант: _____ ()

Рецензент: _____ ()

К защите допустить

Заведующий кафедрой № 806 «Вычислительная математика и программирование»
Крылов Сергей Сергеевич _____ ()

_____ 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа магистра состоит из 111 страниц, 11 рисунков, 24 таблиц, 1 приложения. ОБОГАЩЕНИЕ ТОВАРНЫХ ДАННЫХ, КАСКАДНАЯ АРХИТЕКТУРА, СЛАБЫЙ НАДЗОР, OPEN FOOD FACTS, БАЙЕСОВСКАЯ СЕТЬ, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, КАЛИБРОВКА, ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ, ДОМИНИРОВАНИЕ ПО ПАРЕТО, АУДИТ-ЦИКЛ, ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ

Объём работы. 119 страниц основной части, 4 главы, 12 рисунков, 24 таблицы, 52 источника (16 российских и 36 зарубежных).

Объект исследования. Процесс автоматизированного обогащения товарных каталогов интернет-магазинов с использованием открытых источников данных.

Предмет исследования. Гибридная каскадная архитектура с ансамблевой оценкой уверенности классификаторов и адаптивным обращением к большой языковой модели как запасному слою.

Цель работы. Разработать гибридную каскадную систему автоматизированного обогащения товарных данных для интернет-магазина, обеспечивающую высокую точность извлечения категориальных атрибутов при сокращённой стоимости обращения к большой языковой модели и поддерживающую развёртывание на открытых моделях.

Методы. Многоязычные векторные представления текста на базе модели paraphrase-multilingual-mpnet-base-v2 (SBERT с архитектурой MPNet, поддержка 50+ языков), классификатор XGBoost с регуляризацией и поатрибутными порогами уверенности, байесовская сеть со структурой, найденной алгоритмом Hill Climb по критерию BIC (библиотека pgmpy), доверительные интервалы с кластеризацией по брендам через ресэмплированный бутстрэп, бакетирование числовых нутри-полей, согласие трёх независимых больших языковых моделей (Sonnet 4.5 + GPT-4o + Gemini 2.5 Flash) для построения эталонной разметки на текстово-семантических атрибутах, слепая верификация моделью Claude Opus 4, разбиение на обучающую и тестовую части без пересечения товарных кодов (code-disjoint), проверка гипотезы, зафиксированной до получения результатов с поправкой Бонфферрони на множественные сравнения.

Результаты. Реализована и эмпирически проверена гибридная

каскадная архитектура из пяти слоёв (категорайзер Слой 0 — регулярные выражения — классификатор XGBoost на векторных представлениях SBERT — байесовский валидатор плотностей — запасной слой большой языковой модели) на трёх продуктовых категориях открытого каталога Open Food Facts (паста, шоколад, сыры). На тестовой выборке без пересечения товарных кодов (code-disjoint, новые SKU) размером 16 360 ячеек по 20 парам «категория-атрибут» итоговая конфигурация достигает 93,0 % сквозной точности при обращении к большой языковой модели на 4,1 % ячеек в режиме «каскад + gemini-flash»; точность только-каскадной части (без учёта непокрытых ячеек) составляет 95,5 % (macro-F1 0,890). На консервативной нижней оценке с независимой экспертной разметкой моделью Claude Opus 4 (n=566 cascade-valid из 688 в-score) — 86,7 % сквозной точности. При развёртывании на собственных мощностях с открытой моделью gpt-oss-120b — 90,6 % точности, что на 21,3 п. п. выше прямого использования gpt-oss-120b (69,3 %). Каскад без запасного слоя покрывает 97,3 % ячеек. Достигнуто доминирование по Парето разработанной системы над прямым использованием большой языковой модели по двум осям одновременно — точности и стоимости (+9,2 п. п. точности и сокращение стоимости в 580 раз при той же экономии по сравнению с прямым использованием Claude Sonnet 4.5; архитектурное сокращение стоимости в 24 раза обеспечивается переводом 95,9 % ячеек на дешёвые слои каскада). Гипотеза, зафиксированная до получения результатов, о превосходстве обучаемого стоимостно-чувствительного маршрутизатора над статическим правилом отклонена после поправки Бонферрони на трёх бюджетах. Реализован замкнутый цикл «аудит — правки экстрактора — переобучение — измерение» с кросс-категорийной репликацией 3 из 3. Разработана демонстрационная система из трёх компонентов (шлюз API на Go, ML-сервис на Python/FastAPI, веб-интерфейс), реализующая стабильный контракт REST API.

Использование результатов. Разработанная архитектура применима в системах управления товарной информацией (PIM-системах) интернет-магазинов. Внедрение системы позволяет: повысить полноту автоматического заполнения карточек товаров до 93,0 % (сквозная точность с учётом маршрутизации; 95,5 % по каскадной части как верхняя граница при идеальной маршрутизации категории), сократить стоимость обработки в 580 раз по сравнению с прямым использованием Claude Sonnet 4.5

(комбинированное сокращение, включающее архитектурный вклад каскада в 24 раза и удешевление модели запасного слоя), сократить ручную нагрузку оператора каталога в 10–14 раз за счёт перевода в режим выборочной верификации, обеспечить развёртывание на собственных мощностях без отправки партнёрских данных во внешние API больших языковых моделей.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	9
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	11
ВВЕДЕНИЕ	13
1 РАЗВЁРНУТАЯ АКТУАЛЬНОСТЬ ТЕМЫ	21
1.1 Проблема обогащения товарных каталогов интернет-магазина .	21
1.1.1 Электронная коммерция и роль товарных данных	21
1.1.2 Неоднородность партнёрского ввода	22
1.1.3 Эффекты неполной карточки на пользовательский опыт . .	23
1.1.4 Существующие системы управления товарной информацией не решают задачу	24
1.1.5 Противоречие и постановка задачи	24
1.2 Литературный обзор	25
1.2.1 Извлечение признаков из текстовых описаний товаров . . .	25
1.2.2 Стоимостно-чувствительные каскады больших языковых моделей	26
1.2.3 Существующие промышленные РІМ-системы и анализ аналогов	27
1.2.4 Слабый надзор, калибровка и сопутствующая методология	29
1.3 Техническое задание на разрабатываемую систему	30
1.3.1 Функциональные требования	30
1.3.2 Нефункциональные требования	32
1.3.3 Архитектурные требования	32
1.3.4 Соотношение технического задания и решаемых задач работы	33
1.4 Выводы по главе 1	34
2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РЕШЕНИЯ	36
2.1 Логика работы разрабатываемого программного обеспечения .	36
2.1.1 Формальная постановка задачи обогащения	36
2.1.2 Каскадная схема принятия решения	37
2.1.3 Поведение системы при выходе категории за пределы поддерживаемых	38
2.1.4 Метрики качества и стоимости	39
2.2 Архитектура и стек технологий	40

2.2.1	Архитектура системы	40
2.2.2	Почему каскад, а не сквозное решение	41
2.2.3	Почему SBERT, а не BERT или RoBERTa	42
2.2.4	Почему XGBoost, а не Random Forest или нейронная сеть .	42
2.2.5	Почему байесовская сеть и почему именно pgmpy	44
2.2.6	Почему шлюз API на языке Go, а не моноязычный Python .	45
2.2.7	Почему открытая большая языковая модель в качестве запасного слоя	45
2.2.8	Стек технологий — сводная таблица	46
2.3	Сценарий работы программного обеспечения	48
2.3.1	Сквозной пример прохождения запроса	48
2.4	Выводы по главе 2	48
3	ОПИСАНИЕ ПРОГРАММНО-ТЕХНОЛОГИЧЕСКОЙ РЕАЛИЗАЦИИ	50
3.1	Описание программного обеспечения	50
3.1.1	Архитектура каскада	50
3.1.2	Состав слоёв	50
3.1.3	Поток управления и вход/выход	52
3.1.4	Демонстрационная система	53
3.1.5	Архитектура компонентов	54
3.1.6	Контракт API	54
3.1.7	Запуск и эксплуатация	55
3.2	Данные	55
3.2.1	Источник данных: Open Food Facts	55
3.2.2	Входные и целевые поля, фильтрация, стратифицированная выборка	56
3.2.3	Эталонная разметка: иерархическая схема трёх ярусов . .	57
3.2.4	Разбиение без пересечения товарных кодов (code-disjoint) .	58
3.2.5	Дополнительные источники: разнородные модальности и внешний справочник	59
3.3	Доказательство работоспособности и применимости	59
3.3.1	Условия проверки	59
3.3.2	Тестовая выборка	59
3.3.3	Метрики и доверительные интервалы	60
3.3.4	Стратификация точности по типу эталонного сигнала . . .	60
3.3.5	Воспроизводимость	62

3.3.6	Главный результат: точность и стоимость производственной конфигурации	62
3.3.7	Состав, условия проверки и матрица «точность–стоимость»	62
3.3.8	Декомпозиция по категориям	65
3.3.9	Поатрибутная картина точности и доверительные интервалы	66
3.3.10	Анализ доминирования по Парето: почему каскад превосходит одиночный вызов LLM	68
3.3.11	Учёт точности маршрутизатора первого уровня	68
3.3.12	Вклад каждого слоя каскада	68
3.3.13	Распределение закрытий по слоям	68
3.3.14	Точность каждого слоя на собственном срезе закрытий . .	69
3.3.15	Поатрибутная калибровочная ошибка до и после калибровки	70
3.3.16	Сравнение классификаторов Слой 2: XGBoost, Random Forest, логистическая регрессия, MLP	71
3.3.17	Обоснование сохранения слоя регулярных выражений: сопоставление с обучаемыми альтернативами	74
3.3.18	Поведение байесовского валидатора	76
3.3.19	Метрика отбраковки и селективность сигнала	77
3.3.20	Бизнес-эффект и архитектурный вывод	77
3.3.21	Стоимостно-чувствительный маршрутизатор: отрицательный результат	78
3.3.22	Идея, гипотеза H1 и процедура оценки	78
3.3.23	Результаты	79
3.3.24	Интерпретация и следствие для финальной конфигурации	80
3.3.25	Цикл «аудит — правки экстрактора — переобучение — измерение»	81
3.3.26	Аудит pasta	81
3.3.27	Правки экстрактора	81
3.3.28	Прирост точности эталонной разметки	82
3.3.29	Переобучение моделей и прирост каскада	82
3.3.30	Кросс-категорийная репликация цикла 3/3	83
3.3.31	Дополнительные проверки устойчивости	83
3.3.32	Устойчивость каскада по языкам	83
3.3.33	Сравнение с тривиальным лексическим базисом	84

3.3.34	Семантическая несмежность тестовой и обучающей выборки (k-NN-абляция)	85
3.3.35	Уточнение схемы атрибутов: ортогональные свойства и отказ от мёртвых классов	85
3.3.36	Поатрибутные архитектурные решения	87
3.4	Выводы по главе 3	89
4	ОПИСАНИЕ РЕЗУЛЬТАТОВ	92
4.1	Руководство пользователя	92
4.1.1	Роли и интерфейсы	92
4.1.2	Контент-менеджер	92
4.1.3	Системный аналитик	93
4.1.4	Инженер машинного обучения	94
4.2	Сценарии применения и порядок внедрения	94
4.2.1	Встраивание в бизнес-процесс магазина	94
4.2.2	Сценарий холодного старта новой категории	95
4.2.3	Сценарий многоязычного каталога	95
4.2.4	Сценарий обогащения нутриентно-производных полей	96
4.2.5	Применимость в смежных доменах	96
4.3	Что позволяет использование разработки (характеристика достигнутых результатов)	96
4.3.1	Повышено: качество автоматического обогащения каталога	96
4.3.2	Сокращено: стоимость обработки относительно опорной LLM-модели	97
4.3.3	Обеспечена: возможность развёртывания на собственных мощностях с открытой моделью	99
4.3.4	Сокращён: ручной труд оператора каталога	99
4.3.5	Повышена: устойчивость каталога к новым товарам	99
4.3.6	Резюмирующая таблица «повышено / ускорено / сокращено»	100
4.4	Выводы по главе 4	101
	ЗАКЛЮЧЕНИЕ	103
	ПРИЛОЖЕНИЕ А Электронные материалы	111

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе магистра применяют следующие термины с соответствующими определениями:

Серебряная разметка — набор пар «товар — целевое значение атрибута», полученный автоматически из открытых источников данных (теги Open Food Facts) и используемый для обучения моделей. Содержит шум и неполноту; противопоставляется эталонной разметке

Эталонная разметка — пары «товар — целевое значение», прошедшие верификацию согласованным мнением нескольких языковых моделей и/или слепой разметкой более сильной языковой модели; используется как опорная истина при оценке каскада

Слабый надзор — режим обучения с использованием неполной или зашумлённой разметки, формируемой автоматически из доступных источников вместо ручной разметки экспертом

Каскадная архитектура — последовательность алгоритмических слоёв, каждый из которых обрабатывает только те объекты или атрибуты, которые не были закрыты предыдущими слоями с заданной уверенностью

Многоуровневая фильтрация по уверенности — формализация поведения каскада как последовательного конъюнктивного условия на прохождение калиброванных порогов уверенности на каждом слое. Реализует требование утверждённой темы об «ансамблевой оценке уверенности моделей»

Покрытие — доля ячеек «товар, атрибут» в тестовой выборке, для которых каскад дал предсказание выше порога уверенности на слоях 1–3 (без обращения к запасному слою)

Точность на покрытом срезе — доля верных предсказаний среди тех ячеек, для которых каскад дал ответ без обращения к запасному слою

Сквозная точность — точность системы на полной тестовой выборке, при которой отказы каскада засчитываются как неверные ответы (либо заменяются ответом запасного слоя при включённом Слое 4)

Разбиение без пересечения товарных кодов (code-disjoint) — построение разбиения выборки на обучающую, валидационную и тестовую части, при котором один бренд попадает ровно в одну из трёх частей. Исключает утечку признака «бренд — атрибут» при оценке качества системы

Цикл «аудит — правки — переобучение — измерение» — методологический

приём улучшения системы, при котором обнаружение конкретных дефектов эталонной разметки через аудит приводит к правкам экстрактора, затем к переобучению моделей и измерению прироста точности на той же аудит-выборке

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе магистра применяют следующие сокращения и обозначения:

OFF — Open Food Facts, открытый краудсорсинговый каталог продовольственных товаров под лицензией ODbL

SBERT — Sentence-BERT, многоязычная модель векторного представления предложений

XGBoost — Extreme Gradient Boosting, библиотека градиентного бустинга деревьев решений

ECE — Expected Calibration Error, ожидаемая ошибка калибровки вероятностей классификатора

CPD — Conditional Probability Distribution, условное распределение вероятности

DAG — Directed Acyclic Graph, направленный ациклический граф

BIC — Bayesian Information Criterion, критерий выбора структуры байесовской сети

PDO — Protected Designation of Origin, защищённое наименование товара

AOP — Appellation d'Origine Protégée, защищённое наименование товара (фр.)

LLM — large language model, большая языковая модель

TF-IDF — term frequency $\times$ inverse document frequency, частотно-инверсный показатель

OOD — out of distribution, выход за пределы обучающего распределения

AUROC — area under the ROC curve, площадь под кривой ошибок

ТЗ — техническое задание

BPMN — Business Process Model and Notation, модель и нотация бизнес-процессов

PIM — product information management, управление информацией о товарах

CJM — customer journey map, карта пути клиента

СМ — контент-менеджер

ML — machine learning, машинное обучение

API — application programming interface, программный интерфейс приложения

REST — representational state transfer, архитектурный стиль программного интерфейса

JSON — JavaScript Object Notation, текстовый формат обмена данными

CSV — comma-separated values, текстовый формат табличных данных с разделителями

DOCX — Office Open XML Document, формат текстового документа

E2E — end-to-end, сквозной — точность системы с учётом маршрутизации и отказа от ответа (None=wrong); production-realistic метрика (см. 2.1.4)

MPNet — Masked and Permuted Pre-training, архитектура трансформера, использованная в paraphrase-multilingual-mpnet-base-v2 для 768-мерных эмбедингов

NBSP — non-breaking space, неразрывный пробел; используется в LaTeX между числом и единицей

code-disjoint — разбиение train/test по идентификатору товара (code); brand overlap при этом сохраняется, см. 3.2.4 и 3.3.7

ODbL — Open Database License v1.0, лицензия share-alike для открытых баз данных (применяется к Open Food Facts)

TCO — total cost of ownership, совокупная стоимость владения системой за период

CI — confidence interval, доверительный интервал (95 % если не указано иное)

RF — Random Forest, ансамбль деревьев решений с bagging

MLP — multi-layer perceptron, многослойный перцептрон (полносвязная нейросеть)

ВВЕДЕНИЕ

Актуальность темы исследования.

Электронная коммерция продолжает устойчиво расти. Годовой объём российского сегмента в 2025 году составил 11,5 трлн руб. [ref_1], мировой рынок также показывает уверенный рост [ref_2]. Каталоги крупных интернет-магазинов исчисляются десятками миллионов товарных позиций: только индексируемая часть каталога Ozon в открытых аналитических базах превышает 38 млн позиций [ref_3]. Ежедневно к ним добавляются тысячи новых записей от партнёров-поставщиков.

Карточка товара становится центральным элементом цифровой витрины. От её содержательной полноты зависят работа фасетного поиска, релевантность ранжирования и рекомендаций, корректность аналитики и в конечном счёте конверсия в покупки. Структура партнёрского ввода при этом неоднородна: от одного наименования и бренда до расширенного набора полей с фотографиями и иноязычным составом. Образующиеся пробелы исторически закрывают контент-менеджеры вручную, проверяя и дополняя десятки целевых атрибутов на каждую карточку. При характерных масштабах каталога в миллионы позиций и тысячах новых записей в сутки такой подход не масштабируется ни по затратам, ни по доступности языковых компетенций [ref_4].

Класс систем управления товарной информацией Product Information Management — Akeneo, Pimcore, Salsify и аналогичные — сложился из потребности хранить уже сформированную товарную информацию [ref_5; ref_6; ref_7]. Задачу извлечения этой информации из неструктурированного партнёрского ввода такие системы систематически не решают. Появляющиеся в них модули автоматического обогащения в большинстве случаев представляют собой обёртку над одной публичной большой языковой моделью, вызываемой напрямую на каждый товар. Подробное сопоставление приведено в 1.2.3. Такой вызов экономически приемлем лишь для каталогов небольшого и среднего размера и не сопровождается гарантиями фактической корректности. На числовых полях большие языковые модели систематически допускают фактические ошибки (3.3.2.4). Рост ставок коммерческих API, ужесточение требований к локализации данных и отраслевой тренд на открытые модели в локальном развёртывании [ref_8] дополнительно

усиливают запрос на архитектурные решения, которые не сводятся к прямому вызову одной модели.

Из изложенного следует противоречие, которое и определяет актуальность работы. С одной стороны, автоматизация наполнения карточек — инженерная необходимость. С другой стороны, ни одна изолированная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью одновременно. Правила надёжны на числовых атрибутах, но мало покрывают семантически нагруженные поля. Классические классификаторы требуют значительной размеченной выборки. Большие языковые модели универсальны, однако дороги и подвержены фактическим ошибкам, особенно на числовых полях.

В качестве разрешения этого противоречия в работе рассматривается гибридная каскадная архитектура. Она представляет собой композицию последовательных слоёв, в которой каждый следующий слой подключается только к остатку, не закрытому предыдущими с заданной уверенностью. Такая композиция объединяет семантические возможности генеративных моделей с предсказательной силой классического машинного обучения и автоматической фильтрацией недостоверных значений. Этим определяется актуальность работы с научной стороны как систематическая проверка эффективности гибридного каскада. С практической стороны актуальность задаётся созданием производственно-применимого комплекса с контролируемой стоимостью и возможностью локального развёртывания.

Цель работы.

Целью работы является разработка гибридной каскадной системы автоматизированного обогащения товарных данных интернет-магазина. Система должна обеспечивать высокую точность извлечения категориальных атрибутов при сокращённой стоимости обращения к большой языковой модели, поддерживать локальное развёртывание на открытых моделях и доминировать по Парето над прямым вызовом большой языковой модели одновременно по точности и стоимости.

Задачи работы.

Для достижения поставленной цели сформулированы и решены

следующие пять задач:

- 1) разработать архитектуру гибридного каскадного конвейера обработки товарных данных с многоуровневой фильтрацией по уверенности и явным разделением слоёв. Слои каскада: предкаскадный категорайзер, экстрактор регулярных выражений, классификатор машинного обучения на векторных представлениях, байесовский валидатор плотностей, запасной слой большой языковой модели. Многоуровневая фильтрация формализует требование утверждённой темы об ансамблевой оценке уверенности моделей как последовательное конъюнктивное условие на калиброванные пороги слоёв;
- 2) реализовать каждый слой каскада в виде самостоятельного программного компонента: экстрактор регулярных выражений; классификатор машинного обучения, обучаемый на многоязычных векторных представлениях партнёр-доступных полей; байесовский валидатор, использующий обученную сеть атрибутов в роли фильтра статистически невероятных пар «бренд × значение»; запасной слой на основе большой языковой модели с поддержкой нескольких поставщиков, включая открытые модели для локального развёртывания;
- 3) построить эталонную разметку и проверочную выборку без пересечения товарных кодов (code-disjoint) для оценки качества системы на трёх категориях товаров: паста, шоколад, сыры. Разметка сочетает слабый надзор от тегов открытого каталога, поатрибутное согласие трёх независимых больших языковых моделей и слепую разметку референсной моделью. Разбиение на обучающую, валидационную и тестовую части выполняется без повторного появления одних и тех же брендов;
- 4) провести экспериментальную оценку разработанной системы. Необходимо измерить точность сквозного прохождения каскада, поатрибутное покрытие и стоимость обработки на эталонной тестовой выборке; сопоставить полученные показатели с показателями прямого использования большой языковой модели; оценить вклад каждого слоя каскада, устойчивость к смещению по языкам входных данных и поведение байесовского валидатора как

селективного сигнала на отбраковку фактических ошибок;

- 5) разработать демонстрационный программный комплекс. Комплекс должен обеспечивать взаимодействие оператора каталога с каскадом через веб-интерфейс и стабильный программный интерфейс. Поддерживаются три производственных сценария развёртывания: с премиум-коммерческой большой языковой моделью, с бюджетной коммерческой языковой моделью и с открытой языковой моделью в локальном развёртывании.

Каждой задаче соответствует измеримый результат, который изложен в подразделе «Новизна и основные результаты».

Объект и предмет исследования.

Объектом исследования является процесс автоматизированного обогащения товарных каталогов интернет-магазинов на основе открытых источников данных. На входе процесса имеется минимальный набор партнёр-доступных полей, на выходе требуется получить структурированный набор целевых атрибутов.

Предметом исследования является гибридная каскадная архитектура обработки товарных данных с ансамблевой оценкой уверенности классификаторов и адаптивным селективным обращением к большой языковой модели в роли запасного слоя. Рассматриваются её точность, стоимостные характеристики, устойчивость к смещению входного распределения, а также вычислительные и эксплуатационные свойства, существенные для производственного применения.

Теоретические и методические основы работы.

Работа основывается на следующих методах и инструментах:

извлечение признаков из неструктурированного текста выполняется регулярными выражениями и многоязычным энкодером paraphrase-multilingual-mpnet-base-v2;

классификатор уровня машинного обучения — градиентный бустинг XGBoost поверх полученных векторных представлений;

валидатор атрибутов реализован вероятностной графической моделью средствами библиотеки pgmpy; структура сети обучается алгоритмом Hill

Climb по критерию BIC;

обучающая выборка формируется методом слабого надзора по тегам открытого каталога Open Food Facts;

эталонная разметка строится сочетанием детерминированных правил для нутриентно-производных атрибутов, регуляторных тегов и поатрибутного согласия трёх независимых больших языковых моделей со слепой проверкой референсной моделью; процедура подробно описана в 3.2.3;

оценка качества выполняется на отложенной выборке без пересечения товарных кодов (code-disjoint); доверительные интервалы получены кластерным бутстрэпом с ресэмплированием брендов;

проверка гипотез о превосходстве каскада над одиночной языковой моделью выполняется по заранее зафиксированному протоколу с поправкой Бонферрони на множественные сравнения.

Архитектурный подход к выбору модели по соотношению цена/качество опирается на работы L. Chen и соавторов FrugalGPT [ref_9], P. Aggarwal и соавторов AutoMix [ref_10], D. Ding и соавторов Hybrid LLM [ref_11], а также на бенчмарк Q. J. Hu и соавторов RouterBench [ref_12]. Оформление работы выполнено в соответствии с ГОСТ 7.32-2018, библиографическое описание источников — с ГОСТ 7.1-2003.

Новизна и основные результаты.

Каждой из сформулированных выше задач в работе соответствует не менее одного измеримого результата.

По задаче 1 — проектирование архитектуры каскада, разделы 2.1, 3.1.1. Спроектирован гибридный каскад из пяти последовательных слоёв с конъюнктивным условием на калиброванные пороги уверенности каждого слоя. Разработанная формальная постановка задачи допускает явный отказ системы от ответа и явное определение слоя-источника каждого предсказания.

По задаче 2 — реализация слоёв, раздел 3.1.2. Каскад реализован в виде набора независимо обновляемых программных компонентов. Разработана демонстрационная система в трёхкомпонентной архитектуре «шлюз API на языке Go — ML-сервис на Python/FastAPI — статический веб-интерфейс». Система запускается одной командой и проходит проверку готовности всех слоёв через диагностический интерфейс.

По задаче 3 — построение эталонной разметки, раздел 3.2. Построена

многоисточниковая эталонная разметка для трёх категорий товаров на основе тегов открытого каталога, нутриентно-производных правил, поатрибутного согласия трёх независимых больших языковых моделей и слепой разметки референсной моделью. Разбиение на обучающую, валидационную и тестовую части выполнено без пересечения товарных кодов (code-disjoint).

По задаче 4 — экспериментальная оценка, разделы 3.3.2, 3.3.3, 3.3.4, 3.3.7. Получена сквозная точность 93,0 % (точность только-каскадной части 95,5 %, macro-F1 0,890) на тестовой выборке без пересечения товарных кодов (code-disjoint, новые SKU) из 16 360 ячеек по 20 парам «категория-атрибут» при 580-кратном сокращении стоимости в режиме «каскад + Gemini 2.5 Flash» относительно прямого вызова Claude Sonnet 4.5 (комбинированное сокращение, включающее архитектурный вклад каскада в 24 раза и удешевление модели). На независимом эталоне ручной разметки моделью Claude Opus 4 (n=566 cascade-valid из 688 в-scope) — 86,7 % сквозной точности. Доминирование каскада по Парето над прямым использованием большой языковой модели подтверждено на пяти моделях запасного слоя. Вклад каждого слоя количественно охарактеризован, в том числе селективный вклад байесовского валидатора. Каскад покрывает 97,3 % ячеек.

По задаче 5 — разработка демонстрационного программного комплекса, разделы 4.1, 4.2. Реализована поддержка трёх производственных сценариев развёртывания: премиум-коммерческий, бюджетный коммерческий, локальный с открытой моделью. Поддерживаются три роли пользователей: контент-менеджер, системный аналитик, инженер машинного обучения.

На основании совокупности полученных результатов сформулированы три пункта научной новизны.

Первый пункт научной новизны. Доказано доминирование разработанной гибридной каскадной системы по Парето над прямым использованием большой языковой модели на задаче обогащения товарных каталогов. На тестовой выборке без пересечения товарных кодов (code-disjoint, новые SKU) размером 16 360 ячеек одновременно достигаются превышение точности на 9,2 процентного пункта (93,0 % против 83,8 %) и сокращение стоимости обработки в 580 раз (комбинированное сокращение, включающее архитектурный вклад каскада в 24 раза и удешевление модели запасного слоя). Компромисса между точностью и стоимостью не возникает.

Второй пункт научной новизны. Эмпирически установлено, что в сценарии локального развёртывания с открытой большой языковой моделью gpt-oss-120b в роли запасного слоя каскад компенсирует ограниченные возможности малой открытой модели на сложных атрибутах за счёт специализированного слоя машинного обучения. Прирост сквозной точности составляет 21,3 процентного пункта над прямым использованием той же открытой модели (90,6 % против 69,3 %) при сокращении стоимости в 810 раз относительно опорного прямого вызова Claude Sonnet 4.5. Это позволяет говорить об архитектурной независимости решения от конкретного поставщика большой языковой модели.

Третий пункт научной новизны объединяет два самостоятельных результата. Первый — **зафиксированный до получения результатов отрицательный исход проверки гипотезы H1:** впервые на задаче обогащения товарных каталогов по процедуре, зафиксированной до получения окончательных результатов, с поправкой Бонферрони на множественные сравнения количественно установлено, что статическое reg-(категория, атрибут) правило маршрутизации не уступает обучаемому XGBoost-маршрутизатору с учётом стоимости по сквозной точности при равных бюджетах вызовов запасного слоя. Полученный результат уточняет применимость подхода обучаемой маршрутизации к рассматриваемому классу задач и согласуется с выводами бенчмарка RouterBench [ref_12]. Второй — **кросс-категорийная репликация замкнутого цикла «аудит эталонной разметки — корректировка экстрактора признаков — переобучение моделей — повторное измерение»** в пределах продуктового домена. Цикл воспроизведён на трёх категориях товаров из трёх с приростом сквозной точности на подвыборке аудированных проблемных ячеек +18,2, +16,3 и +14,4 процентного пункта для пасты, шоколада и сыров соответственно.

Апробация и внедрение результатов.

Разработка велась по практическим требованиям, типичным для розничной электронной торговли крупного масштаба (российские и международные интернет-магазины). Архитектурные и эксплуатационные характеристики системы ориентированы на встраивание в производственный процесс обогащения товарного каталога без перестройки соседних звеньев

(см. 4.2.1 — описание точки встраивания в маршрут «партнёрский фид → каталог → витрина»). Поддерживаются три производственных сценария развёртывания: с премиум-коммерческой большой языковой моделью (Claude Sonnet 4.5 / GPT-4o), с бюджетной коммерческой (Gemini 2.5 Flash), с открытой моделью в полностью локальном развёртывании (gpt-oss-120b) — последний сценарий критичен для требований локализации обработки партнёрских данных.

Апробация выполнена в виде демонстрационного программного комплекса. Комплекс запускается одной командой и проверяется на работоспособность через диагностический программный интерфейс. Демонстрация подтверждает, что все слои каскада функционируют на трёх предметных категориях: паста, шоколад, сыры. Воспроизводимость численных показателей обеспечена сценарием `reproduce.sh`, который регенерирует основные результаты из артефактов в каталоге `datasets/processed/` репозитория. Разработанный комплекс готов к интеграции в системы управления товарной информацией розничной торговли (см. 4.2.1).

Структура и объём работы.

Выпускная квалификационная работа состоит из введения, четырёх глав, заключения и списка использованных источников. Список использованных источников включает 52 наименования: 16 российских источников, индексируемых в РИНЦ, и 36 зарубежных публикаций, отраслевых отчётов и стандартов.

Первая глава содержит обоснование актуальности темы, литературный обзор по трём направлениям и техническое задание (ТЗ) на разрабатываемую систему. Вторая глава посвящена теоретическому обоснованию решения. В ней приводятся формальная постановка задачи, обоснование архитектуры и стека технологий, а также сценарии прохождения запроса через каскад. Третья глава содержит описание программной реализации, использованных данных и доказательство работоспособности каскада на эталонной тестовой выборке без пересечения товарных кодов (`code-disjoint`). Четвёртая глава представляет порядок применения системы, сценарии её развёртывания и характеристику достигнутых результатов в формате «повышено / ускорено / сокращено». В заключении собраны выводы по главам, научная новизна и перспективы развития темы.

1 РАЗВЁРНУТАЯ АКТУАЛЬНОСТЬ ТЕМЫ

В главе обосновывается тема работы, систематизируются существующие подходы к автоматическому обогащению товарных данных и формулируется техническое задание на разрабатываемую систему.

1.1 Проблема обогащения товарных каталогов интернет-магазина

1.1.1 Электронная коммерция и роль товарных данных

Рынок электронной коммерции устойчиво растёт: годовой оборот российского сегмента в 2025 году — 11,5 трлн руб. [ref_1], мировой рынок продолжает расширяться [ref_2]. Каталоги крупных площадок насчитывают десятки миллионов позиций (индексируемая часть Ozon — > 38 млн [ref_3]); ежедневно добавляются тысячи новых артикулов от партнёров.

Карточка товара — центральный элемент цифровой витрины: от её атрибутов (тип, состав, габариты, маркировки качества, страна происхождения, предупреждения) зависит фасетный поиск, релевантность ранжирования, аналитика и конверсия. При неполном заполнении цепочка «партнёрский фид — каталог — редакция — витрина» прерывается во втором звене: товар либо попадает на витрину неполным (страдает конверсия), либо задерживается на ручную доработку (растут операционные затраты).

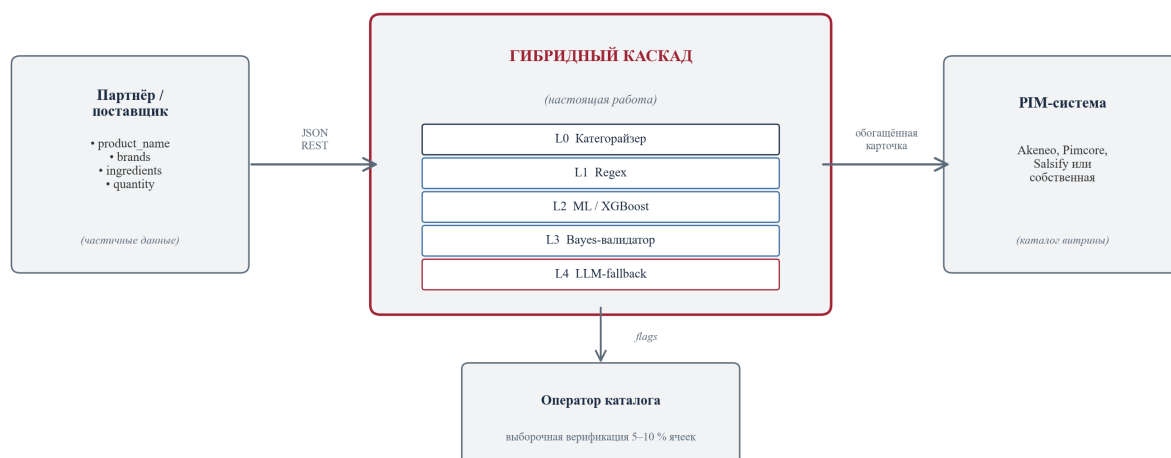


Рисунок 1 – Место системы автоматизированного обогащения в инфраструктуре интернет-магазина

1.1.2 Неоднородность партнёрского ввода

Партнёрские данные принципиально неоднородны: для одного товара поставщик передаёт только название и бренд, для другого — расширенный набор полей с фотографиями, для третьего — состав на иностранном языке, для четвёртого доступен только штрихкод. Анализ Open Food Facts [ref_13] показывает, что заполненность нутри-полей систематически ниже базовых.

Пробелы исторически закрываются контент-менеджерами вручную: на каждую карточку — десятки атрибутов. При каталоге в миллионы позиций и тысячах новых записей в сутки такой путь экономически неприемлем [ref_4]. Дополнительно ручной труд плохо масштабируется по языкам: команда, работающая с RU/EN, теряет качество на FR, DE, IT, ES — особенно в категориях импортной пищевой продукции.

Типовой процесс работы контент-менеджера сводится к последовательности «приём — поиск аналогов — заполнение — внутренняя проверка — публикация», с обратной связью от жалоб клиентов и возвратов (рисунок 1.2, а). Внедрение гибридного каскада изменяет порядок звеньев: ручное заполнение заменяется автоматическим выводом каскадной системы, а ручной труд сосредотачивается на выборочной верификации и периодическом аудите ошибок (рисунок 1.2, б).

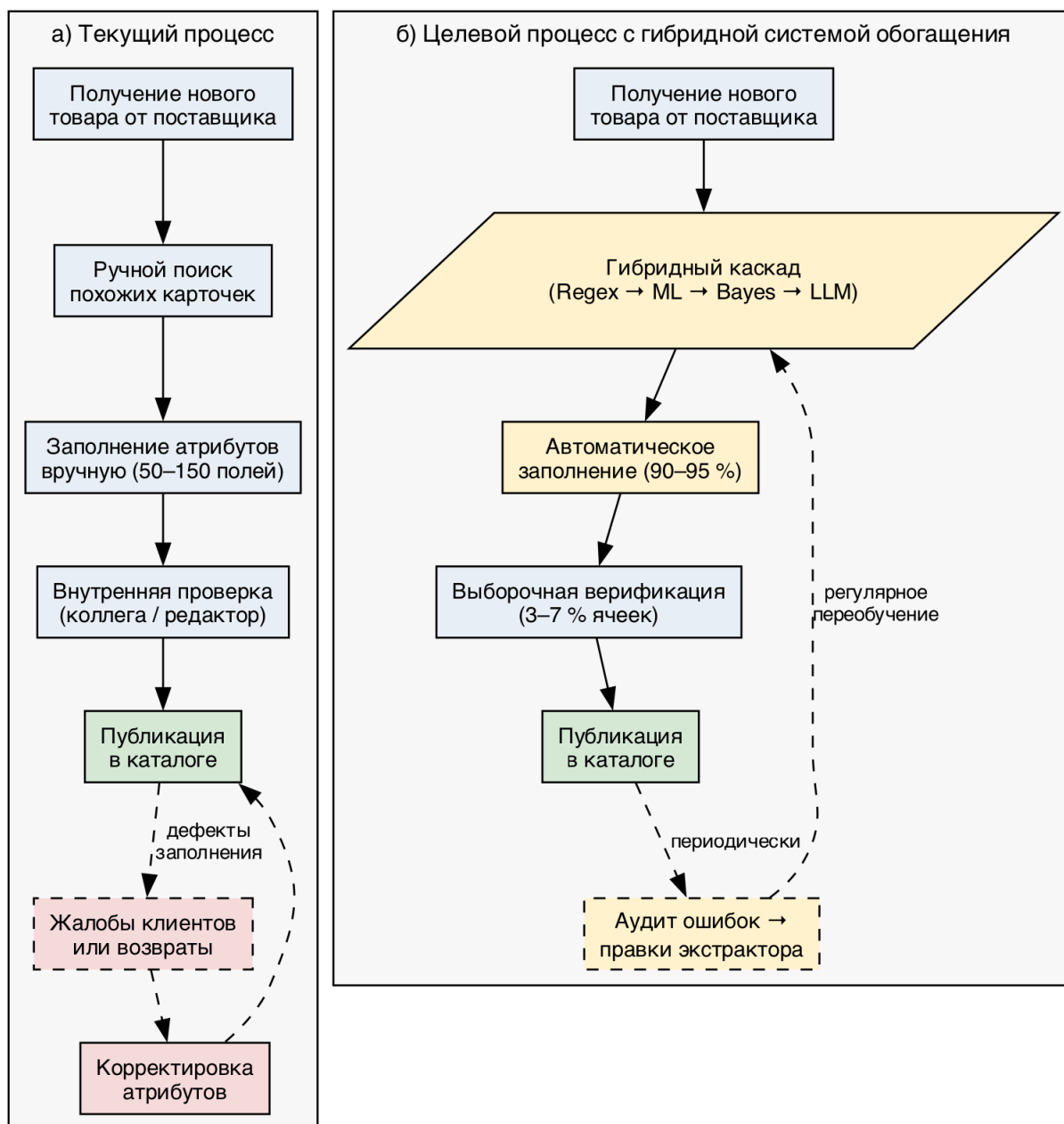


Рисунок 2 – Процесс работы контент-менеджера: (а) текущий, (б) целевой с интеграцией гибридного каскада обогащения

Примечание. Этапы текущего процесса реконструированы по материалам публичной документации Akeneo [ref_5], Pimcore [ref_6] и Salsify [ref_7]; целевой процесс — авторская конструкция.

1.1.3 Эффекты неполной карточки на пользовательский опыт

Неполная карточка даёт три независимых воздействия: (1) снижается качество фасетного поиска (товар выпадает из подборок при применении

фильтров — например, упаковка пасты без формы или типа зерна); (2) растут возвраты и негативная обратная связь, особенно по атрибутам правового или диетического значения (органика, глютен, аллергены); (3) растёт нагрузка на модерацию — «брошенная» карточка удлинит цикл выхода товара на витрину или порождает повторные правки.

1.1.4 Существующие системы управления товарной информацией не решают задачу

Класс систем управления информацией о товарах (PIM, product information management) — Akeneo, Pimcore, Salsify, Riversand, inRiver [ref_5; ref_6; ref_7] — сложился из корпоративной потребности хранить и публиковать товарную информацию в едином каталоге. Их функциональность сосредоточена на управлении уже сформированной информацией, не на её извлечении. Сравнительный анализ платформ управления нормативно-справочной информацией с учётом MDM/PIM-функциональности проведён С. А. Долгоруковой [ref_47]. Появляющиеся модули автоматического обогащения — обёртки над одной публичной большой языковой моделью (LLM, large language model) с прямым вызовом на каждый товар, экономически приемлемы лишь для каталогов небольшого и среднего размера и не дающие гарантий корректности. Альтернатива в виде агрегаторов готовых описаний задачу извлечения атрибутов из произвольного партнёрского текста не решает и ограничена крупными брендами.

1.1.5 Противоречие и постановка задачи

Изложенное определяет противоречие в основе актуальности: автоматизация наполнения — инженерная необходимость, но ни одна отдельная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью. Правила надёжны на числовых атрибутах, но дают низкое покрытие на семантически нагруженных полях. Классические классификаторы требуют значительной разметки и плохо переносятся между языками. LLM универсальны, но дороги [ref_9] и систематически ошибаются на числовых полях, требующих агрегирования из текста (эмпирическая характеристика — 3.3.2.4).

Из противоречия вытекает рабочая архитектурная гипотеза: каскадная

композиция слоёв, в которой каждый следующий слой подключается только к остатку, не закрытому предыдущими с заданной уверенностью, способна совместить преимущества отдельных технологий. Проверка этой гипотезы — содержательное ядро работы.

1.2 Литературный обзор

В разделе систематизируется научный и индустриальный задел по трём направлениям, которые образуют основу разрабатываемой системы:

- извлечение признаков из текстовых описаний товаров;
- каскады больших языковых моделей с учётом стоимости вызовов;
- существующие промышленные системы управления товарной информацией.

1.2.1 Извлечение признаков из текстовых описаний товаров

Задача восполнения пропущенных атрибутов рассматривается как частный случай извлечения признаков из неструктурированного текста (information extraction, IE). Систематизация методов извлечения информации из текста, включая выделение именованных сущностей и фактов, приведена в обзоре Л. М. Ермаковой [ref_51]; гибридная природа практических решений по выделению ключевых слов и признаков из русскоязычных текстов отмечена в работе А. С. Ванюшкина и Л. А. Гращенко [ref_52]. Подходы делятся на три группы — формальные правила, классические статистические модели и нейросетевые модели на трансформерах — и их сочетание образует основу каскадных архитектур.

Правила и регулярные выражения. Формальные шаблоны надёжно извлекают атрибуты с устойчивой лексической разметкой (% жира, масса нетто, время приготовления, минимальный возраст) при околонулевой стоимости и встроенной интерпретируемости; ограничены низким покрытием на семантически нагруженных полях (тип крупы, форма пасты, маркировка органики).

Статистические модели над поверхностными признаками. Классификаторы поверх TF-IDF (term frequency $\times$ inverse document frequency, частотно-инверсный показатель) и градиентный бустинг (XGBoost [ref_14], LightGBM [ref_15]) работают при достаточной выборке и одноязычном входе.

Применение ML к классификации в электронной коммерции рассмотрено Г. И. Афанасьевым и соавторами [ref_16]. Слабая переносимость на многоязычные тексты и отсутствие учёта семантики ограничивают применимость.

Векторные представления на основе трансформеров. Архитектура трансформера [ref_17] и BERT [ref_18] подняли качество IE. Sentence-BERT [ref_19] и многоязычные дистиллированные варианты [ref_20] дают фиксированные представления предложений, поверх которых ставится произвольный классификатор. Категоризация и извлечение признаков из товарных каталогов рассмотрены А. Г. Колониным [ref_21]; систематическое изложение NLP и IE — в учебнике Е. И. Большаковой и соавторов [ref_22].

Специализированные системы извлечения атрибутов товаров. В индустриальной литературе выделяются три эталонные системы:

- TXtract [ref_23] (Karamanolakis et al., ACL 2020, Amazon) — многозадачный нейронный кодировщик с условием на таксономию категорий, обеспечивающий извлечение атрибутов в крупных каталогах при значительном дисбалансе классов;
- MAVE [ref_24] (Yang et al., WSDM 2022, Google) — многоисточниковый кодировщик на основе BERT с кросс-вниманием, использующий не только наименование, но и описание, спецификации и другие текстовые поля карточки;
- OpenTag [ref_25] (Zheng et al., KDD 2018) — двунаправленная рекуррентная архитектура BiLSTM-CRF с механизмом внимания, формулирующая задачу извлечения атрибутов как задачу разметки последовательности.

Эти системы дают 85–92 % F1 на специализированных корпусах. Сопоставление с каскадными архитектурами, привлекающими LLM в роли запасного слоя, в открытой литературе систематически не представлено: специализированные системы и LLM-каскады трактуются как отдельные направления. Настоящая работа частично закрывает этот пробел.

1.2.2 Стоимостно-чувствительные каскады больших языковых моделей

С появлением спектра LLM разной мощности оформилась подобласть cost-aware LLM routing — направление каждого запроса к минимально

достаточной модели без потери качества.

FrugalGPT. L. Chen, M. Zaharia и J. Zou [ref_9] предложили каскадную композицию LLM по возрастанию стоимости с прерыванием на первом уверенном ответе: каскад из 3–4 моделей с откалиброванной уверенностью даёт то же качество, что прямой вызов самой дорогой, при стоимости на порядок ниже. Настоящая работа опирается на идеи последовательного прерывания и калибровки.

AutoMix. P. Aggarwal, A. Madaan и соавторы [ref_10] основываются на самопроверке ответа малой моделью с делегированием к большой при сомнении. Разрабатываемая система, наоборот, разносит роли валидатора и предсказателя в отдельные слои.

Hybrid LLM. D. Ding, A. Mallick и соавторы [ref_11] рассматривают маршрутизацию между моделью на пограничном устройстве и облачной — постановка прямо соответствует сценарию «локально развёрнутая gpt-oss-120b + коммерческое API как запасной слой».

RouterBench. Q. J. Hu, J. Bieker и соавторы [ref_12] показали, что обучаемые маршрутизаторы не доминируют над простыми политиками при локализованной структуре ошибок. На задаче обогащения каталогов в настоящей работе впервые по протоколу, зафиксированному до получения результатов установлено, что статическое рег-(категория, атрибут) правило не уступает обучаемому маршрутизатору по сквозной точности при равном бюджете запасного слоя.

1.2.3 Существующие промышленные PIM-системы и анализ аналогов

Для позиционирования разработки относительно индустриальных аналогов отобраны три коммерческие PIM-системы: Akeneo, Pimcore и Salsify. По ним построена сравнительная таблица по семи функциональным критериям, существенным для задачи автоматического обогащения каталога (таблица 1.1). Источниками послужила документация вендоров [ref_5; ref_6; ref_7].

Таблица 1 – Сравнение существующих PIM-систем и разрабатываемой системы

№	Критерий	Akeneo	Pimcore	Salsify	Разрабатываемая система
1	Централизованное хранение каталога	да	да	да	не входит в задачу
2	Распределение по каналам сбыта	да	да	да	не входит в задачу
3	Автоматическое обогащение атрибутов	частично (Akeneo AI / GenAI, 2023+ — обёртка над сторонней LLM)	частично (правила обогащения + плагины)	частично (Salsify AI Suite — обёртка над сторонней LLM)	да (каскадная архитектура с многоуровневой фильтрацией по уверенности)
4	Извлечение признаков из неструктурированного текста	нет	нет	нет	да (слои ML и LLM)
5	Поддержка многоязычных входов	через ручные переводы	через ручные переводы	через ручные переводы	встроенная (многоязычные эмбединги, 50+ языков)
6	Открытый исходный код	community-редакция	да	нет	да (включая возможность локального развёртывания LLM)

Продолжение таблицы

№	Критерий	Akeneo	Pimcore	Salsify	Разрабатываемая система
7	Стоимостно-чувствительная обработка	нет	нет	нет	да (каскадная композиция с доминированием по Парето по стоимости и качеству)

Из таблицы 1.1 следует, что коммерческие PIM-системы хорошо покрывают хранение, версионирование и распределение уже сформированной информации. Однако функция автоматического обогащения у всех трёх крупных платформ (Akeneo с 2023 года, Salsify через AI Suite, Pimcore через плагины) реализуется как прямая обёртка над сторонней большой языковой моделью — без учёта стоимости массовых вызовов, без поатрибутной маршрутизации между моделями и без архитектурных гарантий локального развёртывания партнёрских данных. Таким образом, ни одно из проанализированных решений не закрывает одновременно весь набор критериев интернет-магазина: автоматическое обогащение, многоязычную обработку, контролируемую стоимость и возможность локального развёртывания. Это обосновывает целесообразность разработки самостоятельной системы.

1.2.4 Слабый надзор, калибровка и сопутствующая методология

Принципиальная сложность задачи — отсутствие достаточной ручной разметки; постановка известна как *weak supervision*, инструментарий систематизирован А. Ratner и соавторами «Snorkel» [ref_26]. Подходы к снижению стоимости разметки через активное обучение и слабый надзор систематизированы в обзоре В. Settles [ref_46]. Пригодным источником слабого эталона выступает Open Food Facts [ref_13] — открытая база >4,5 млн

пищевых продуктов с пользовательскими тегами. Эмпирический анализ устойчивости алгоритмов классификации к зашумлению обучающей разметки выполнен В. А. Дюком [ref_48] и показал, что классические методы (KNN, SVM) сохраняют приемлемую точность при умеренной доле ошибок разметки, что обосновывает применимость серебряной разметки.

Для корректной работы каскада выходы классификаторов должны быть откалиброваны: масштабирование Платта [ref_27], изотоническая регрессия [ref_28]; современный анализ ECE для глубоких моделей — С. Guo и соавторы [ref_29]. Систематизация критериев качества классификаторов (включая ROC-анализ и выбор порога вероятности) дана в работах Ю. С. Шуниной и соавторов [ref_49] и А. А. Ковалева и соавторов [ref_50]. Оценка точности при ограниченной выборке — кросс-валидация и бутстрэп [ref_30]; интервальная оценка пропорций — формула Вильсона [ref_31].

Аппарат байесовских сетей опирается на J. Pearl [ref_32], D. Heckerman et al. [ref_33], А. Л. Тулупьева и соавторов [ref_34], библиотеку pgmpy [ref_35]; поиск структуры — Hill Climb + BIC [ref_36]. Прикладные вопросы устойчивости ML-моделей к дрейфу в розничных каталогах рассмотрены А. А. Артемовым [ref_37]; ограниченная масштабируемость стандартных подходов к управлению ассортиментом на маркетплейсах — С. А. Сергеев [ref_38]; активное обучение и краудсорсинг — Р. А. Гилязов и Д. Ю. Турдаков [ref_39]; ранние работы по семантическим пространствам — В. Ф. Хорошевский [ref_40].

1.3 Техническое задание на разрабатываемую систему

В разделе формулируется техническое задание (ТЗ) на разрабатываемую систему. Оно подразделяется на три группы требований: функциональные (что система должна делать), нефункциональные (с какими характеристиками) и архитектурные (каким способом она организована).

1.3.1 Функциональные требования

Ф-1. Принимаемый формат входных данных. Система принимает на входе товар, описанный полями партнёра-поставщика:

наименование (product_name);

бренд (brands);

список ингредиентов в произвольной форме (ingredients_text);
масса, объём или упаковка (quantity);
идентификатор категории (category), опционально.

Все поля, кроме наименования, считаются необязательными.

Ф-2. Возврат заполненных значений целевых атрибутов. Система возвращает для каждого целевого атрибута предсказанное значение, оценку уверенности и идентификатор слоя каскада, на котором принято финальное решение. Тем самым обеспечивается отслеживаемость источника каждого предсказания и возможность ручной верификации контент-менеджером.

Ф-3. Покрытие предметных категорий. В пределах работы система поддерживает заполнение целевых атрибутов для трёх категорий пищевой продукции: паста, шоколад и шоколадные изделия, сыры. Выбор именно этих трёх категорий обоснован тремя причинами. Во-первых, в Open Food Facts по ним имеется достаточный объём обучающих данных. Во-вторых, доступен аудит-эталон с согласием трёх больших языковых моделей и слепой разметкой Orpus. В-третьих, сами категории репрезентативны с точки зрения распределения типов целевых атрибутов: числовых, бинарных и мультиклассовых семантических.

Ф-4. Автоматическое определение категории. Если категория явно не указана, система определяет её автоматически по поступившему текстовому описанию товара. Эта функция реализуется отдельным предкаскадным слоем (Слой 0).

Ф-5. Запасной слой на основе большой языковой модели. Если получить однозначный ответ на конкретный атрибут средствами слоёв 1–3 каскада не удаётся, система обращается к большой языковой модели как к запасному слою. Запасной слой подключается селективно: только для тех атрибутов конкретного товара, по которым предыдущие слои не выдали уверенного ответа.

Ф-6. Диагностический вывод. Система возвращает диагностическую информацию: результат работы валидатора (понижение или подтверждение предсказания), агрегированные оценки уверенности по слоям и маршрут запроса по каскаду. Эта информация необходима для отладки, аудита и накопления данных о поведении системы в эксплуатации.

1.3.2 Нефункциональные требования

НФ-1. Точность. Целевая сквозная точность каскада на эталонной тестовой выборке без пересечения товарных кодов между обучающей и тестовой частями (code-disjoint, новые SKU) — не менее 85 %. В конфигурации с коммерческой большой языковой моделью в запасном слое — не менее 90 %.

НФ-2. Стоимость. Стоимость обработки одного товара каскадом должна быть существенно ниже стоимости прямого вызова большой языковой модели на тот же товар. Целевой коэффициент снижения — не менее 2,5×. Ограничение сверху не задаётся; фактически достижимое значение фиксируется по результатам эмпирической оценки в 3.3.2.

НФ-3. Производительность. Время отклика на один товар при работе без обращения к запасному слою — не более 500 мс на потребительском ноутбуке без специализированного ускорителя. При необходимости обращения к запасному слою — не более 5 секунд. Это значение определяется временем ответа внешнего API большой языковой модели.

НФ-4. Многоязычность. Система поддерживает обработку описаний товаров на пяти рабочих языках европейского рынка пищевой продукции: французском, английском, немецком, итальянском и испанском. Это соответствует наиболее представленным языкам в Open Food Facts и моделирует реалистичный многоязычный партнёрский поток.

НФ-5. Возможность локального развёртывания. Система поддерживает сценарий полностью локального развёртывания, при котором обращения к внешним коммерческим API исключаются. Этот сценарий реализуется через подключение в запасной слой открытой языковой модели gpt-oss-120b. Тем самым к каскаду предъявляется требование вендорной независимости.

1.3.3 Архитектурные требования

А-1. Каскадная архитектура с отдельными слоями. Система реализует последовательность слоёв обработки по принципу «передачи остатка». Каждый последующий слой обрабатывает только те атрибуты, по которым предыдущие слои не выдали ответа с уверенностью выше калиброванного порога. Архитектурно слои разделены и могут обновляться

независимо.

А-2. Независимое обновление слоёв. Изменение или переобучение любого отдельного слоя не должно требовать перезапуска или повторного развёртывания других слоёв и API-шлюза системы. К таким изменениям относятся обновление правил, дообучение классификаторов слоя 2, замена языковой модели в запасном слое.

А-3. Демонстрационный программный комплекс с веб-интерфейсом. Система сопровождается демонстрационным комплексом, который состоит из трёх компонентов:

API-шлюз на языке Go;

ML-сервис на языке Python (FastAPI);

одностраничный веб-интерфейс.

Демонстрация обеспечивает верификацию работоспособности и наглядно представляет результаты работы каскада для контент-менеджера.

1.3.4 Соотношение технического задания и решаемых задач работы

Сформулированные требования полностью покрываются задачами, поставленными во введении (таблица 1.2).

Таблица 2 – Соответствие требований технического задания и задач работы

№ задачи	Краткая формулировка задачи	Покрываемые требования ТЗ
1	Разработать архитектуру гибридного каскадного конвейера	Ф-1, Ф-2, Ф-4, Ф-5, А-1
2	Реализовать каждый слой каскада как самостоятельный программный компонент	Ф-6, А-2

Продолжение таблицы

№ задачи	Краткая формулировка задачи	Покрываемые требования ТЗ
3	Построить эталонную разметку и проверочную выборку без пересечения товарных кодов (code-disjoint)	Ф-3 (в части обеспечения данных для оценки на трёх категориях)
4	Провести экспериментальную оценку точности, покрытия и стоимости	НФ-1, НФ-2, НФ-3, НФ-4, НФ-5
5	Разработать демонстрационный программный комплекс	А-3

Установленное соответствие подтверждает, что выполнение пяти задач работы влечёт за собой выполнение технического задания в полном объёме.

1.4 Выводы по главе 1

В главе проведён анализ предметной области, систематизировано состояние литературы и обосновано техническое задание. По результатам анализа можно сформулировать следующие выводы.

- 1) Проблема автоматического обогащения остаётся открытой. Промышленные PIM-системы, такие как Akeneo [ref_5], Pimcore [ref_6], Salsify [ref_7], ориентированы на хранение и распределение уже сформированной информации. Появляющиеся модули автоматического обогащения реализуются как обёртки над одной коммерческой большой языковой моделью и не отвечают требованиям обработки больших каталогов с учётом стоимости.
- 2) Научные подходы делятся на две группы, и ни одна из них не покрывает задачу полностью. Специализированные системы извлечения атрибутов, такие как TXtract [ref_23], MAVE [ref_24], OpenTag [ref_25], решают задачу с высоким качеством. Однако они

требуют значительной размеченной выборки и не учитывают экономики обращения к моделям. Каскады больших языковых моделей с учётом стоимости — FrugalGPT, AutoMix, Hybrid LLM — дают инструменты управления стоимостью, систематически сопоставленные в бенчмарке RouterBench. Однако эти каскады рассматривают композицию из однородных моделей и не интегрируют классическое машинное обучение.

- 3) Разрабатываемая система объединяет оба подхода в гибридный каскад. Каскад сочетает специализированные слои извлечения признаков — правила, машинное обучение на многоязычных эмбедингах, байесовский валидатор — и запасной слой LLM с учётом стоимости. Совокупность шести функциональных, пяти нефункциональных и трёх архитектурных требований ТЗ формирует обязательство, выполнение которого верифицируется в экспериментальной части работы.

2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РЕШЕНИЯ

В главе формально описывается разрабатываемая система и обосновываются принятые архитектурные решения. Сначала приводится формальная постановка задачи и логика работы программного обеспечения (2.1). Затем рассматриваются архитектура и стек технологий, а также обоснование выбора каждой компоненты (2.2). В конце разбирается логика прохождения типового запроса через систему (2.3).

2.1 Логика работы разрабатываемого программного обеспечения

2.1.1 Формальная постановка задачи обогащения

Пусть товарная позиция, поступающая в систему от партнёра, представлена кортежем партнёр-доступных полей

$$x = (x^{\text{name}}, x^{\text{brand}}, x^{\text{ingr}}, x^{\text{qty}}), \quad (1)$$

где x^{name} — название, x^{brand} — бренд, x^{ingr} — состав, x^{qty} — количество. Любое поле, кроме названия, может быть пустым. Задача — восстановить структурированные характеристики по этому минимуму.

Каждому товару ставится в соответствие категория $c \in \mathcal{C} = \{c_1, \dots, c_M\} \cup \{\text{OOD}\}$ ($M = 3$: pasta, chocolate, cheeses; метка OOD (out of distribution, выход за пределы обучающего распределения) — выход за пределы поддерживаемых доменов). Категорию определяет отдельная модель

$$c = f_0(x), \quad (2)$$

(Слой 0, см. 2.2.1). Для каждой категории задана схема $\mathcal{A}_c = \{a_1, \dots, a_{K_c}\}$ с областями $\mathcal{Y}_k$; система строит

$$\hat{y} = (\hat{y}_1, \dots, \hat{y}_{K_c}) \in (\mathcal{Y}_1 \cup \{\text{null}\}) \times \dots \times (\mathcal{Y}_{K_c} \cup \{\text{null}\}), \quad (3)$$

где $\hat{y}_k = \text{null}$ — явный отказ системы от ответа. Честное молчание предпочтительнее неверного заполнения: повторное обращение к оператору дешевле ошибки на витрине. Опорное эталонное значение — $y_k^{(i)}$; товары с известным эталоном образуют проверочную выборку (2.1.4).

2.1.2 Каскадная схема принятия решения

Логика обогащения реализована как каскад из четырёх содержательных слоёв; ему предшествует определение категории. Для каждого атрибута k итоговое предсказание формируется по правилу

$$\hat{y}_k = \begin{cases} f_k^{\text{regex}}(x), & f_k^{\text{regex}}(x) \neq \emptyset; \\ f_k^{\text{ML}}(x), & f_k^{\text{regex}}(x) = \emptyset, P_{\text{ML}}(\hat{y}_k | x) \geq \tau_k, P_B(\hat{y}_k | \mathbf{e}) \geq \theta_k; \\ f_k^{\text{LLM}}(x), & \text{otherwise.} \end{cases} \quad (4)$$

В первой строке экстрактор регулярных выражений нашёл значение; во второй — экстрактор отказался, но классификатор и валидатор согласились с заданным уровнем уверенности; в третьей строке (otherwise) ни одно из условий выше не выполнено, и ячейка передаётся на запасной слой.

Используются следующие обозначения:

- f_k^{regex} — экстрактор регулярных выражений (Слой 1), детерминированная функция от текстовых полей. Слой считается несработавшим, если ни один шаблон не дал совпадения;
- f_k^{ML} — классификатор машинного обучения (Слой 2) над многоязычным векторным представлением партнёр-доступных полей;
- $P_{\text{ML}}(\hat{y}_k | x)$ — апостериорная вероятность класса по оценке классификатора после калибровки;
- τ_k — поатрибутный порог уверенности классификатора, подобранный на отложенной части обучающей выборки;
- $P_B(\hat{y}_k | \mathbf{e})$ — оценка вероятности предсказанного значения по обученной байесовской сети при свидетельствах $\mathbf{e} = (x^{\text{brand}}, \hat{y}_{j \neq k})$. Свидетельствами выступают известный бренд и уже сформированные предсказания других атрибутов товара;
- θ_k — поатрибутный порог байесовского валидатора. В реализации он задаётся как пятый перцентиль распределения P_B по обучающим примерам с верной разметкой;
- f_k^{LLM} — обращение к большой языковой модели (LLM, large language model) в роли запасного слоя (Слой 4). Возвращает либо значение из $\mathcal{Y}_k$, либо null.

Предсказание слоя 2 принимается тогда и только тогда, когда одновременно выполнены

$$[P_{\text{ML}}(\hat{y}_k | x) \geq \tau_k] \wedge [P_B(\hat{y}_k | \mathbf{e}) \geq \theta_k]. \quad (5)$$

Это конъюнктивное условие реализует многоуровневую фильтрацию по уверенности и формализует требование темы об ансамблевой оценке уверенности — не параллельное голосование, а последовательная фильтрация, где каждый слой имеет калиброванный порог и вправе отказаться, передав решение следующему слою.

Условие $P_B \geq \theta_k$ задаёт общую форму. По итогам 3.3.4 в финальную конфигурацию валидатор включён селективно: активен только на парах «категория — атрибут» с неотрицательным приростом, иначе порог θ_k устанавливается так, что условие не отсекает слой 2.

Схема обладает тремя полезными свойствами: (1) стратифицированное решение — простые случаи закрываются дешёвым слоем 1, средние — слоем 2, трудный остаток — дорогим слоем 4; слои упорядочены по росту стоимости. (2) Композициональная отбраковка — валидатор работает после классификатора, отбраковывая статистически невероятные пары «бренд × значение» (закрывает требование темы об автоматической отбраковке). (3) Архитектурная объяснимость — для каждого предсказания известно, какой слой принял решение, что даёт локальную интерпретируемость даже при черномысличном классификаторе в середине.

2.1.3 Поведение системы при выходе категории за пределы поддерживаемых

Слой 0 определяет $c = f_0(x)$ до запуска каскада. Если достоверность ниже порога OOD-детектора, товару присваивается метка OOD и система возвращает «категория не поддерживается»: $f_0(x) \in \mathcal{C} \setminus \{\text{OOD}\} \Rightarrow$ запуск per-category каскада со схемой $\mathcal{A}_{f_0(x)}$; $f_0(x) = \text{OOD} \Rightarrow \hat{y} = \emptyset$. Явный отказ выбран по соображениям эксплуатационной безопасности: ошибочное отнесение к чужой категории даёт неверную схему всем атрибутам, что заметно хуже честного отказа.

2.1.4 Метрики качества и стоимости

Каскад поддерживает явный отказ от ответа; используются две согласованные метрики и одна сводная.

Точность на покрытой части — доля верных предсказаний среди ячеек с непустым ответом:

$$\text{acc}_k^{\text{cov}} = \frac{|\{i : \hat{y}_k^{(i)} = y_k^{(i)} \wedge \hat{y}_k^{(i)} \neq \text{null}\}|}{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}. \quad (6)$$

Покрытие (coverage) — доля ячеек, на которых система решилась дать ответ:

$$\text{cov}_k = \frac{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}{N}, \quad (7)$$

где N — мощность проверочной выборки.

Сквозная точность (end-to-end accuracy). Чтобы предотвратить искусственное завышение качества за счёт молчания, ячейки с $\hat{y}_k^{(i)} = \text{null}$ засчитываются как неверные:

$$\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k. \quad (8)$$

Сквозная точность является основной метрикой работы и используется в качестве заголовочного числа (3.3.2). Раздельный учёт acc^{cov} и cov необходим для добросовестного сравнения каскада с конфигурациями, в которых отказ от ответа невозможен. К таким конфигурациям относится, например, сценарий end-to-end LLM.

Стоимость. В качестве архитектурной метрики стоимости используется ожидаемая доля вызовов запасного слоя, нормированная на 1000 обработанных ячеек:

$$\text{cost}_{\text{LLM}} = \frac{1000}{N \cdot K_c} \sum_{i=1}^N \sum_{k=1}^{K_c} \mathbb{1}[\hat{y}_k^{(i)} \text{ получен через } f_k^{\text{LLM}}]. \quad (9)$$

Метрика инвариантна к выбору модели запасного слоя и характеризует структуру каскада. Для экономического сопоставления (3.3.2) она умножается

на удельную стоимость провайдера и нормируется относительно стратегии «все ячейки решаются Claude Sonnet 4.5». Совместный анализ (acc^{e2e} , cost_{LLM}) образует матрицу «точность — стоимость», на которой строится заголовочный результат (3.3.2).

2.2 Архитектура и стек технологий

В разделе описывается архитектурное решение, реализующее формальную постановку 2.1, и обосновывается выбор каждой компоненты стека.

2.2.1 Архитектура системы

Функциональная модель системы в виде диаграммы потоков данных приведена на рисунке 2.1. Внешние акторы (партнёр-продавец и витрина каталога интернет-магазина) обмениваются данными со шлюзом API и ML-сервисом, который содержит каскад из пяти слоёв и обращается к хранилищам моделей XGBoost, байесовских сетей и внешнему API большой языковой модели.

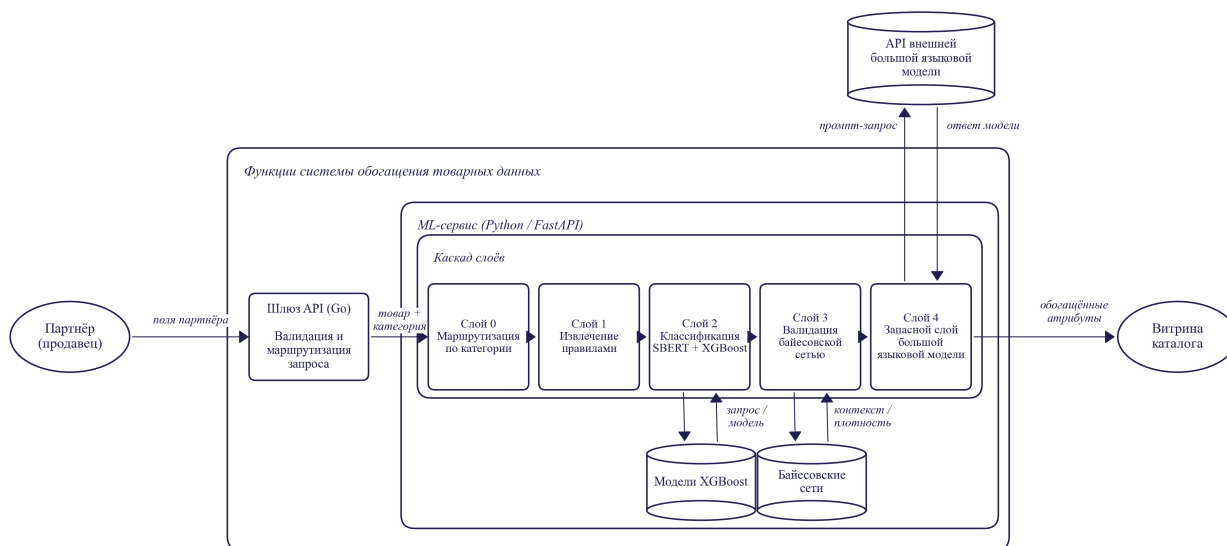


Рисунок 3 – Функциональная модель системы обогащения товарных данных

Система реализована как пятислойный конвейер (Слой 0 + четыре содержательных слоя). Состав и роль каждого слоя:

- **Слой 0 — категорайзер.** LightGBM поверх TF-IDF-представления (term frequency × inverse document frequency, частотно-инверсный

показатель) названия и бренда с порогом OOD; определяет $c \in \mathcal{C}$ либо относит к OOD.

- **Слой 1 — экстрактор регулярных выражений.** Извлекает значения, явно присутствующие в тексте (форма пасты, % какао, цельнозерновой, шаблоны количества). Применяется только к шаблонам с высокой точностью совпадения.
- **Слой 2 — классификатор машинного обучения.** Основной слой: партнёр-доступные поля конкатенируются и кодируются SBERT в 768-мерный вектор (модель `paraphrase-multilingual-mpnet-base-v2`, архитектура MPNet); поатрибутный XGBoost с порогом τ_k на отложенной выборке. Изотоническая пост-калибровка исследована в двух режимах (3.3.3.3): серебряная деградирует на эталоне из-за дрейфа; перекрёстная на эталонной выборке даёт улучшение ECE с 0,070 до 0,043 (в производственную конфигурацию не включена).
- **Слой 3 — байесовский валидатор.** Сеть (структура — Hill Climb + BIC) вычисляет $P_B(\hat{y}_k | \mathbf{e})$; ниже порога θ_k — флаг и передача дальше. Роль пересмотрена (3.1): не классификатор, а валидатор плотностей.
- **Слой 4 — запасной слой на большой языковой модели.** Взаимозаменяемые модели семейств GPT [ref_41] и Claude [ref_42]: Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая `gpt-oss-120b`, малая Llama-3.2-3b (сопоставление — 3.3.2). Структурированный запрос со схемой и партнёр-доступными полями; возвращает значение или честный отказ.

Слои упорядочены по росту удельной стоимости решения. Regex детерминирован и по существу бесплатен. ML работает на уже вычисленном векторном представлении и потому тоже почти бесплатен. Байесовскому валидатору требуется один вызов вывода на сеть. Запасной слой самый дорогой.

2.2.2 Почему каскад, а не сквозное решение

Альтернативы каскаду — крайние конфигурации all-LLM и all-ML — рассмотрены как точки отсчёта и отвергнуты. Сквозное решение на LLM даёт 83–94 % точности (3.3.2), но стоимость линейно растёт с числом ячеек; каскад экономит порядка двух порядков при равной или большей точности за счёт стратифицированного распределения нагрузки. Сквозное решение на

классическом ML не подходит из-за производных атрибутов, требующих знаний за пределами товарного описания, и сценария холодного старта новых категорий. Каскад занимает нишу между двумя крайностями.

2.2.3 Почему SBERT, а не BERT или RoBERTa

В качестве кодировщика выбран `paraphrase-multilingual-mpnet-base-v2` (SBERT [ref_20] с архитектурой MPNet). Обоснование: бесплатная многоязычность (50+ языков в едином пространстве без дообучения); умеренная 768-мерная размерность представления (равна BERT-base и в 1,3 раза меньше RoBERTa-large-1024) при заметно более высоком качестве на задачах семантической близости — MPNet объединяет преимущества маскированного (как в BERT) и перестановочного (как в XLNet) языкового моделирования и в среднем по [ref_20] превосходит дистиллированные MiniLM-варианты на 2–4 п. п. macro-F1; готовность к работе без доменной адаптации (контрастивное обучение на парафразах хорошо подходит для классификации по семантической близости).

2.2.4 Почему XGBoost, а не Random Forest или нейронная сеть

Слой 2 — XGBoost [ref_14] с регуляризацией (`subsample=0.8`, `colsample=0.8`, ранняя остановка, `gamma=0.1`). XGBoost устойчиво работает на плотных векторных признаках при ограниченной разметке ($n \approx 1250$ на категорию, см. 3.2.1), допускает пост-калибровку изотонической регрессией [ref_28] и выполняет вывод на CPU за миллисекунды без GPU. Поатрибутная архитектура «по одному классификатору на атрибут» упрощает цикл «аудит — правка — переобучение — измерение» (3.3.6).

Альтернативные классификаторы рассмотрены в трёх семействах: ансамбли деревьев с бэггингом (Random Forest [ref_53]), линейные модели (логистическая регрессия с калибровкой Платта [ref_27]) и нейронные сети прямого распространения (многослойный перцептрон, MLP). Random Forest, в отличие от градиентного бустинга, не оптимизирует функционал потерь поэтапно и хуже подходит для сильно несбалансированных мультиклассовых задач с длинным хвостом значений (характерных для атрибутов вроде `pasta_shape` и `chocolate_extra`). Линейные модели на плотных векторных представлениях SBERT теряют качество на семантически

нагруженных мультиклассовых атрибутах, поскольку решающая граница в исходном пространстве не является линейно разделимой. Нейронные сети на $n \approx 1250$ примерах подвержены переобучению без аккуратной регуляризации; в каскадной архитектуре дополнительной проблемой становится калибровка вероятностей — для корректной работы порогов слоя 2 требуется не максимум ассигу, а минимум ECE.

Различие Random Forest и XGBoost содержательно опирается на три механизма. Во-первых, режим обучения ансамбля: Random Forest строит независимые деревья по бутстреп-выборкам и усредняет их предсказания (бэггинг), тогда как XGBoost последовательно дообучает каждое следующее дерево на остатках градиента функции потерь предыдущей композиции (бустинг по остаткам [ref_14; ref_54]). Поэтапная оптимизация позволяет XGBoost выстраивать более тонкие нелинейные границы между классами на одном и том же признаковом пространстве. Во-вторых, регуляризация: XGBoost явно вводит L1- и L2-штрафы на веса листьев плюс γ -штраф на число листьев в дереве [ref_14], тогда как у Random Forest регуляризация только косвенная — через ограничение глубины, минимального числа объектов в листе и числа деревьев [ref_53; ref_54]. В-третьих, контроль переобучения: подвыборка признаков и наблюдений на уровне отдельного дерева в сочетании с ранней остановкой по валидационной метрике [ref_14] даёт XGBoost более прямой инструмент сдерживания переобучения, чем у Random Forest. На независимых бенчмарках табличных данных результаты согласуются с этой теоретической картиной: на 11 датасетах [ref_55] и 176 датасетах [ref_56] модели градиентного бустинга (XGBoost, LightGBM, CatBoost) в среднем устойчиво превосходят Random Forest и нейросетевые архитектуры на табличных задачах с десятками-сотнями признаков и тысячами примеров — типичной размерности, в которой работает Слой 2.

Эмпирическое сравнение четырёх классификаторов на идентичных признаках и эталоне (3.3.X) подтверждает преимущество XGBoost: при сопоставимой или незначительно меньшей точности XGBoost обеспечивает наименьшую ошибку калибровки (ECE 0,026 против 0,031 у MLP и 0,035 у LogReg) и достоверно превосходит Random Forest на всех 16 атрибутах эксперимента.¹

¹Расширение эмпирического сравнения с Random Forest на полные эталоны LLM-consensus и человеческой золотой выборки оставлено как направление дальнейшего развития;

2.2.5 Почему байесовская сеть и почему именно pgmpy

Для валидатора плотностей выбрана дискретная байесовская сеть в реализации pgmpy [ref_35] 1.1.2. В сравнении с MRF и автоэнкодерами она допускает структурное обучение (Hill Climb + BIC [ref_36]) с интерпретируемым ориентированным графом, дешёвый вывод через Variable Elimination и явное выражение условных зависимостей. pgmpy выбрана за совместимость с Python-экосистемой и поддержку как структурного обучения, так и параметрической оценки через BayesianEstimator.

Структура обученной байесовской сети для категории chocolate приведена на рисунке 4. Узлы графа — целевые атрибуты и бренд; направленные рёбра показывают условные зависимости, найденные алгоритмом Hill Climb на серебряной выборке (9 рёбер при ограничении `max_indegree=3`). Граф иллюстрирует характерные межатрибутные связи. Бренд выступает корневым узлом: через производный двоичный признак `brand_has_organic_marker` (наличие подстроки *bio/organic* в строке бренда) он влияет на `chocolate_type` и непосредственно на `is_organic`. Тип шоколада, в свою очередь, обуславливает процент какао (`cocoa_percentage`: тёмный шоколад имеет в среднем более высокий процент) и подтип `chocolate_extra` (тёмный → с орехами; молочный → с наполнителем). От `chocolate_extra` расходятся четыре ребра — к `contains_nuts`, `nutri_score_grade`, `protein_class` и повторно к `is_organic`, у которого таким образом два родителя — это согласуется с интуицией: органический шоколад чаще встречается у соответствующих брендов и в определённых подтипах. Эмпирическая роль сети как селективного валидатора и количественные характеристики приведены в 3.3.4.

теоретические аргументы (поэтапная оптимизация по остаткам, явная регуляризация $L1/L2/\gamma$, ранняя остановка) согласуются с результатами на независимых табличных бенчмарках [ref_55; ref_56].

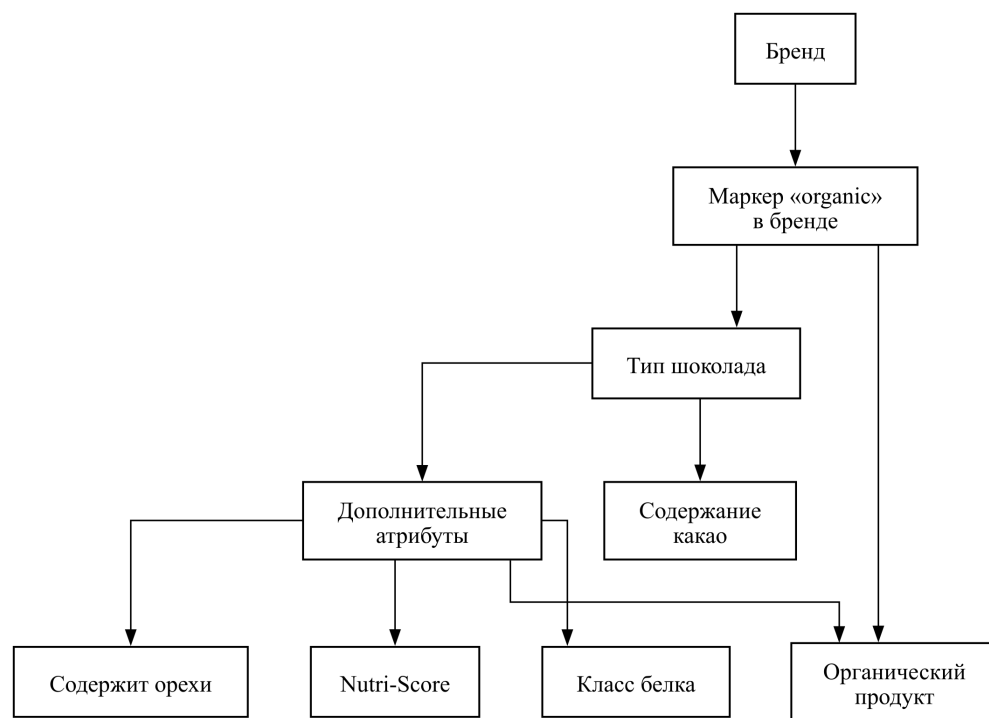


Рисунок 4 – Структура байесовской сети категории *chocolate* (направленный ациклический граф; алгоритм Hill Climb с критерием BIC, библиотека pgmpy 1.1.2)

2.2.6 Почему шлюз API на языке Go, а не моноязычный Python

Демо-система — два сервиса: шлюз на Go (:8080) и ML-сервис на Python/FastAPI (:8001). Go подходит для сетевого ввода-вывода (малое потребление памяти, быстрый запуск, конкурентный I/O); Python-экосистема (PyTorch, sentence-transformers, XGBoost, pgmpy) безальтернативна для ML-инференса. Разделение соответствует типовой production-архитектуре и позволяет масштабировать компоненты независимо.

2.2.7 Почему открытая большая языковая модель в качестве запасного слоя

В роли Слой 4 поддерживается набор взаимозаменяемых моделей через OpenRouter: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b, открытая малая llama-3.2-3b (сопоставление — 3.3.2). Обоснование многомодельной конфигурации: открытую gpt-oss-120b можно запустить в инфраструктуре заказчика без передачи данных вовне; коммерческие LLM взаимозаменяемы, что снижает зависимость от провайдера; каскад компенсирует слабость малой модели — при

самостоятельном применении gpt-oss-120b проигрывает Sonnet 14–15 п. п., но в составе каскада превосходит безусловное обращение к ней на 21,3 п. п. (3.3.2).

2.2.8 Стек технологий — сводная таблица

Сводное представление принятых решений приведено в таблице 2.1.

Таблица 3 – Сводная таблица стека технологий с обоснованием выбора

Компонент	Технология	Ключевое обоснование
Шлюз API	Go, стандартная библиотека net/http	Малое потребление ресурсов, быстрый запуск, отсутствие внешних зависимостей
ML-сервис	Python 3.14, FastAPI, Uvicorn	Доступ к ML-экосистеме, автоматическая валидация через Pydantic, поддержка OpenAPI
Векторное представление	SBERT paraphrase-multilingual-bert	Многоязычность (50+ языков), 768-мерный вектор (архитектура MPNet), отсутствие необходимости в fine-tuning

Продолжение таблицы

Компонент	Технология	Ключевое обоснование
Классификатор Слой 2	XGBoost с регуляризацией; перекрёстная изотоническая калибровка на эталонной выборке снижает среднюю ECE с 0,070 до 0,043 (3.3.3.3), внедрение в производственную конфигурацию вынесено в раздел «Перспективы развития темы» заключения	Устойчивость на плотных признаках, калибруемые вероятности, дешёвый вывод
Байесовская сеть	pgmpy 1.1.2, DiscreteBayesianNetwork	Структурное обучение Hill Climб + BIC, дешёвый вывод (Variable Elimination), интерпретируемый граф
Категорайзер Слой 0	LightGBM (поверх TF-IDF-представления) + порог OOD	Многоклассовая классификация с вероятностями + явный отказ при низкой уверенности
Запасной слой	OpenRouter API: Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, gpt-oss-120b, llama-3.2-3b	Единый интерфейс, возможность локального развёртывания, независимость от поставщика

Продолжение таблицы

Компонент	Технология	Ключевое обоснование
Фронтенд	HTML/CSS/JavaScript без сборочной системы	Минимальные требования к окружению, прозрачность демонстрации

2.3 Сценарий работы программного обеспечения

2.3.1 Сквозной пример прохождения запроса

Прохождение рассматривается на примере *Spaghetti #5 Barilla* (повторяется в 3.1.2). Партнёр шлёт POST `/api/enrich` с полями `category=null`, `product_name=«Spaghetti #5 Barilla»`, `brands=«Barilla»`, `ingredients_text=«Durum wheat semolina, water»`, `quantity=«500g»`. Поле `category` пустое — категоризация делегирована системе.

Шаг 1. Определение категории (Слой 0). Возвращает $c = \text{pasta}$ с уверенностью 0,97; OOD не активирован, выбрана схема $\mathcal{A}_{\text{pasta}}$ (8 атрибутов).

Шаг 2. Поочерёдная обработка атрибутов. Каскад обходит атрибуты по схеме. Для `pasta_shape` срабатывает шаблон Слой 1 на «spaghetti» с уверенностью 1,0 — следующие слои не вызываются. Для `grain_type` Слой 1 не даёт совпадения; Слой 2 выдаёт "wheat" с $P_{\text{ML}} = 0,99 \geq \tau = 0,5$, валидатор подтверждает ($P_B = 0,85 \geq \theta = 0,04$) — отметка ml. Для `nutri_score_grade` — класс "A", $P_{\text{ML}} = 0,83$, валидатор подтверждает. Для `is_filled` $P_{\text{ML}} = 0,45 < \tau$ — ячейка идёт в Слой 4.

Шаг 3. Формирование ответа. ML-сервис собирает поатрибутный ответ с агрегированными счётчиками и возвращает JSON шлюзу (формат — 3.1.2). В сценарии 7 из 8 атрибутов закрыты дешёвыми слоями (1 regex + 6 ML), 1 — Слой 4. Именно такое распределение обеспечивает доминирование каскада в матрице «точность — стоимость» (3.3.2).

2.4 Выводы по главе 2

- 1) Задача обогащения формализована как поатрибутное предсказание $\hat{y}_k$ при заданных партнёр-доступных полях и определённой Слой 0

категории; логика — последовательная фильтрация по уверенности, предсказание классификатора принимается при одновременном выполнении $P_{ML} \geq \tau_k$ и $P_B \geq \theta_k$ (формализация ансамблевой оценки уверенности как AND-условия, а не параллельного голосования).

- 2) Архитектурный выбор каждой компоненты (каскад вместо сквозного решения; SBERT, XGBoost, pgmpy; Go-шлюз + Python-ML; открытая gpt-oss-120b) обоснован эксплуатационно, методологически и стратегически. Каскад превращает слабую открытую модель в работоспособную часть production-стека.
- 3) Сквозной сценарий показывает: большинство атрибутов закрывается дешёвыми слоями, обращение к LLM точно — что и объясняет доминирование по Парето, подтверждаемое в 3.3.

3 ОПИСАНИЕ ПРОГРАММНО-ТЕХНОЛОГИЧЕСКОЙ РЕАЛИЗАЦИИ

Глава описывает программную реализацию системы: 3.1 — каскад и демонстрационный комплекс; 3.2 — данные и предобработку; 3.3 — численные результаты и сценарии тестирования.

3.1 Описание программного обеспечения

3.1.1 Архитектура каскада

Гибридный каскад — последовательность из пяти слоёв; каждый обрабатывает свой класс случаев и передаёт остаток следующему. Архитектура соответствует формальной постановке (2.1) и реализует многоуровневую фильтрацию по уверенности с собственным калиброванным порогом на каждом слое; итоговый вердикт — последовательное AND-условие порогов.

3.1.2 Состав слоёв

Состав слоёв и их роли описаны в 2.2.1. Ниже приведены детали программной реализации каждого из них; повторное описание архитектурной роли опущено.

Слой 0 — категорайзер. Реализован как LightGBM поверх TF-IDF-представления (term frequency $\times$ inverse document frequency, частотно-инверсный показатель) (n-граммы 1–2) с порогом OOD (out of distribution, выход за пределы обучающего распределения). Артефакты: `models/category_router_v3_lgbm.pkl`, `models/category_router_v3_vec.pkl`, `models/category_router_v3_threshold.json`. На итоговую точность системы влияет через штраф маршрутизации: при ошибке в категории атрибуты товара получают неверную схему и засчитываются как неверные предсказания.

Слой 1 — экстрактор регулярных выражений. Реализация — `src/pipeline/regex/extractor.py`. Разбирает поля `product_name` и `quantity`; распознаёт формы пасты («spaghetti», «penne»), процент какао («70 % соcoa» — класс «70–85»), индикатор цельнозернового состава, международные шаблоны вроде «300 g», источник молока в сырах (chèvre /

brebis / bufala → goat / sheep / buffalo), маркеры защищённого наименования (AOP / AOC / DOP / PDO / IGP), явные индикаторы ультра-обработанного сыра (fondu, processed cheese, Schmelzkäse). Покрытие слоя неоднородно по категориям: около 18 % на pasta, 23 % на chocolate и 3 % на cheeses (доменные сигналы текстуры и страны происхождения сыра плохо параметризуются регулярными выражениями и переданы на ML-слой). Точность на закрытых ячейках — 87,0–100 % в зависимости от шаблона; на cheeses-выборке gold-эталона все три cheese-шаблона дают 100 % precision (47/47, 95 %-й интервал Вильсона: 92,4–100 %; аудит false-positive исключил формы нарезки «slices / tranches» как маркеры обработки). См. 3.3.3.2.

Слой 2 — классификатор машинного обучения. Основной рабочий слой (73,2 % всех решений системы; 72–96 % по категориям). На вход подаётся конкатенация `product_name + brands + ingredients_text + quantity`. SBERT-кодировщик `paraphrase-multilingual-mpnet-base-v2` (50+ языков, архитектура MPNet) даёт 768-мерный плотный вектор. Параллельно вычисляется частотно-инверсный (TF-IDF) векторайзер (n-граммы 1–2, top-5000), сжатый усечённым сингулярным разложением (TruncatedSVD) до 128 dense-компонент; конкатенация SBERT и TF-IDF SVD даёт 896-мерное гибридное признаковое пространство. Для каждой пары (категория, атрибут) обучен XGBoost-классификатор `{cat}_v4_mpnet_tfidf_noleak_{attr}_xgb.pkl` с регуляризацией (subsample=0.8, colsample=0.8, ранняя остановка, sample_weight по обратной частоте классов) и поправкой вероятностей через CalibratedClassifierCV (Platt). Пороги уверенности подбираются на отдельном валидационном срезе (find_best_threshold — sweep по интервалу 0,5–0,95 с оптимизацией $\text{macro-F1} \times \text{coverage}$). При вероятности ниже порога слой отказывается отвечать. Точность на покрытом срезе — 94,9–98,6 % (см. 3.3.3.2).

Слой 3 — байесовский валидатор. Сеть хранится в `models/{cat}_stratified_bayesian.pkl`; структура — Hill Climb + BIC. Роль пересмотрена по итогам 3.3.4: классификаторная точность сети на трудном остатке после слоя 2 — около 52 %, ниже мажоритарной точки отсчёта. Поэтому сеть используется как валидатор плотностей: для предсказания слоя 2 вычисляется $P(\text{value} \mid \text{evidence})$, при попадании в нижний 5-перцентиль ячейка помечается флагом и понижается на следующий

слой.

Слой 4 — запасной слой на большой языковой модели. Реализация — `src/pipeline/llm_fallback/enrich.py`. Оценке в 3.3.2 подлежат пять моделей: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b через OpenRouter и открытая малая llama-3.2-3b. Промпт содержит схему атрибутов категории, требуемый JSON-формат и партнёр-доступные поля без подсказок об ожидаемом значении.

3.1.3 Поток управления и вход/выход

Поток управления описан в 2.1 (формула $\hat{y}_k$). На каждом слое одно из двух решений: закрыть атрибут или передать дальше. Слои упорядочены по росту стоимости, поэтому вызов большой языковой модели (LLM, large language model) делается только на ячейках, где предыдущие слои отказались. Вход — партнёр-доступные поля; слой 2 обучается только на них, что исключает утечку через теги OFF. Алгоритм обработки одного запроса в нотации ГОСТ 19.701-90 приведён на рисунке 3.1.

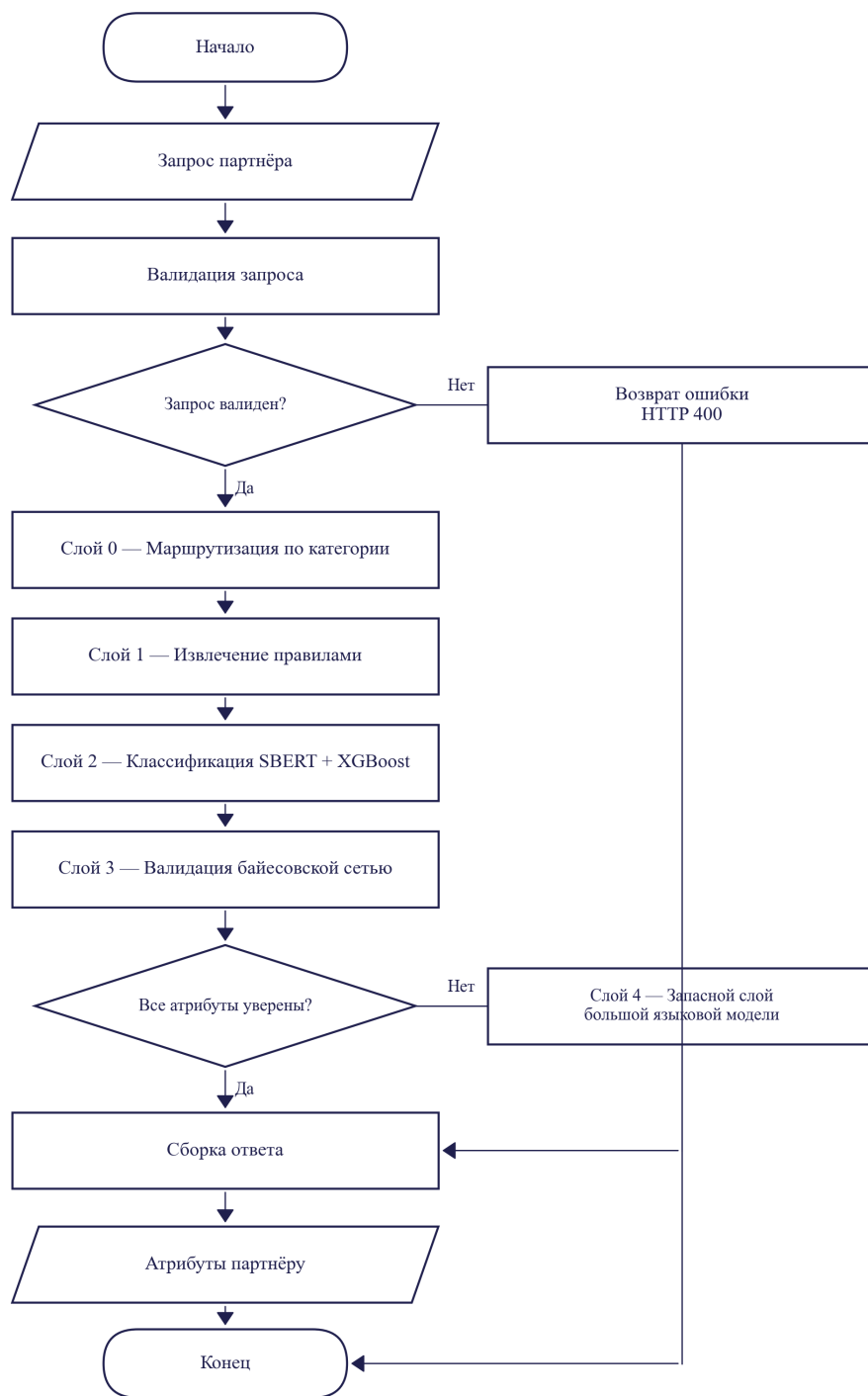


Рисунок 5 – Блок-схема алгоритма обработки запроса
(нотация ГОСТ 19.701-90)

3.1.4 Демонстрационная система

Трёхкомпонентная демонстрационная система имитирует встраивание в бизнес-процесс интернет-магазина.

3.1.5 Архитектура компонентов

Цепочка обработки: браузер (HTML/JS) → шлюз API на Go (:8080) → ML-сервис на Python/FastAPI (:8001) → каскад Слой 0 — regex — ML — Bayes — LLM.

Шлюз на Go (demo/gateway/, ≈200 строк, стандартная net/http) выполняет валидацию JSON, проксирование к ML-сервису, CORS, логирование и отдачу статического фронтенда. ML-сервис (demo/ml_service/) при старте загружает SBERT-кодировщик, 20 XGBoost-классификаторов, байесовские сети, словари порогов и слой 0; после прогрева (≈15–20 с) запросы к слоям 0–3 обрабатываются за десятки миллисекунд. Эндпоинты: POST /api/enrich (обогащение), GET /api/categories (схема), GET /health (готовность), POST /api/explain (Shapley-объяснения). Веб-интерфейс (demo/frontend/) — статическая HTML/JS-страница с формой ввода, переключателем категоризации, пресетами и блоком результата с отметкой слоя-источника.

3.1.6 Контракт API

Запрос POST /api/enrich принимает JSON с полями category (опционально; иначе работает слой 0), product_name, brands, ingredients_text, quantity. Пример:

```
{ "category": "pasta", "product_name": "Spaghetti #5 Barilla",  
  "brands": "Barilla", "ingredients_text": "Durum wheat semol"
```

Ответ содержит category, агрегированные счётчики (n_attrs_total, n_covered, n_llm_fallback) и объект predictions, где для каждого атрибута указаны значение, слой-источник (regex / ml / llm_fallback), уверенность и результат валидации:

```
{ "category": "pasta", "n_attrs_total": 8, "n_covered": 7, "n  
  "predictions": {  
    "pasta_shape": { "value": "spaghetti", "layer": "regex",  
    "grain_type": { "value": "wheat", "layer": "ml", "confid  
                  "validation": { "flagged": false, "p": 0  
    "is_filled": { "value": null, "layer": "llm_fallback",
```

Явная отметка слоя-источника компенсирует ограниченную интерпретируемость слоёв 2/3 на уровне отдельного товара.

3.1.7 Запуск и эксплуатация

Запуск выполняется тремя командами (ML-сервис на :8001, Go-шлюз на :8080, браузер) либо одной — `bash reproduce.sh demo`. Готовность системы проверяется через `GET /health`. Стек технологий и обоснование компонентов — в 2.2.8 (таблица 2.1).

3.2 Данные

3.2.1 Источник данных: Open Food Facts

Основой выбран открытый каталог Open Food Facts (далее — OFF) — краудсорсинговая база $\approx 4,5$ млн уникальных позиций под лицензией Open Database License (ODbL v1.0), допускающей коммерческое использование при соблюдении условий share-alike: производные данные должны распространяться под той же лицензией с указанием источника. В настоящей работе соблюдены оба требования: указание авторства Open Food Facts и его сообщества контрибьюторов приведено в источнике [ref_13]; производные обучающие выборки и эталоны размещены в репозитории под совместимой open-licensee schema. Дамп получен командой `python -m src.data.download --source off` и конвертирован в `parquet` (`datasets/raw/en.openfoodfacts.org.products.parquet`, 526 МБ).

Выбор OFF обоснован тремя соображениями: открытость лицензии обеспечивает воспроизводимость; каталог репрезентативен и мультиязычен (50+ языков в `product_name` и `ingredients_text`); помимо текстовых полей OFF содержит структурированные нутриентные показатели (`fat_100g`, `sugars_100g` и др.) и регуляторные теги (`labels_tags`, `ingredients_analysis_tags`), на которых построен эталон (3.2.3). Альтернативы (скрейпинг онлайн-магазинов, коммерческие PIM, теги Wikipedia) отклонены по лицензионным и покрытийным соображениям.

Область охвата — три категории: `pasta`, `chocolate`, `cheeses`. Они выбраны по критериям 1.3: достаточный объём, различие типа целевых атрибутов и наличие подгруппы атрибутов с детерминированным эталоном из нутриентов.

3.2.2 Входные и целевые поля, фильтрация, стратифицированная выборка

Поля OFF разделены на партнёр-доступные (product_name, brands, quantity, ingredients_text, code) и целевые выходные (categories_tags, labels_tags, *_100g, ingredients_analysis_tags). Целевые поля не передаются на вход обучаемых компонентов под угрозой утечки; слой 2 формирует вход только из конкатенации product_name + brands + ingredients_text + quantity.

Фильтрация по категории (src/data/filter.py) использует списки включения/исключения categories_tags (например, для pasta включение — en:pastas, en:fresh-pastas, исключение — en:potato-products, en:rice-products, en:noodles). После фильтрации применяется стратифицированная по языку выборка (определение языка — langdetect, страты — FR, EN, DE, IT, ES). Из каждой страты отбирается ≈ 250 товаров, итого ≈ 1250 на категорию (таблица 3.2.1).

Таблица 4 – Состав стратифицированной выборки по категориям, входящим в область охвата работы

Категория	Объём после фильтрации (сырые позиции)	Объём стратифицированной выборки	Число целевых атрибутов
pasta (макаронные изделия)	15 691	1 250	8
chocolate (шоколад)	13 469	1 250	7
cheeses (сыры)	21 208	1 250	9

Примечание.

Источник:

datasets/processed/{cat}_stratified_silver_standard.parquet.

Целевые атрибуты определены в src/pipeline/schemas/{cat}.py и включают как кросс-категорийные (is_organic), так и специфичные (pasta_shape, chocolate_type, milk_source, texture). После пересмотра схемы v4 из эксперимента исключены три атрибута с неудовлетворительной разрешимостью на партнёр-доступных полях

(nutri_score_grade, fat_class, chocolate/cocoa_percentage). Дополнительно по результатам анализа ошибок (см. 3.3.7.4) принят принцип ортогональности атрибутов: структурная характеристика «наличие начинки» выделена в самостоятельный бинарный атрибут `chocolate.is_filled`, ортогональный к типу какао-массы (`chocolate_type`) и составу включений (`chocolate_extra`); это устранило систематическую путаницу классов и повысило макро-F1 каждого из трёх атрибутов. Итоговая схема — 20 пар (категория, атрибут): `chocolate` — 6 (`chocolate_type`, `is_filled`, `chocolate_extra`, `contains_nuts`, `is_organic`, `flavor_profile`), `pasta` — 8 (`grain_type`, `pasta_shape`, `is_filled`, `is_gluten_free`, `is_organic`, `is_vegan`, `cuisine_origin`, `protein_class`), `cheeses` — 7 (`milk_source`, `texture`, `country_of_origin`, `aging`, `is_pdo`, `is_organic`, `is_ultra_processed`).

3.2.3 Эталонная разметка: иерархическая схема трёх ярусов

Эталонная разметка построена как многоярусная процедура, где каждый ярус опирается на качественно различный класс свидетельств. Это даёт высокую точность при объёме, недостижимом для ручной разметки, и явно отделяет детерминированно разрешимые атрибуты от семантических.

Ярус 0 (нутриент-производные). Детерминированное бакетирование числовых полей: `pasta/protein_class` из `proteins_100g` по регламенту ЕК 1924/2006. Точность эталона — 100 % при достоверных нутриентах, покрытие 95–100 %. После пересмотра схемы v4 из яруса исключены `nutri_score_grade` и `fat_class` — их предсказание на партнёр-доступных полях оказалось неудовлетворительным (см. 3.2.2).

Ярус 1 (тег-производные). Эталон из регуляторных тегов: `is_organic` из `en:organic`, `is_gluten_free` из `en:gluten-free`, `is_pdo` из `en:protected-designation-of-origin`. Достоверность опирается на правовую защиту меток в ЕС (регламенты 834/2007 и 1151/2012).

Ярус 2 (семантически-текстовые). Признаки без детерминированного правила: `pasta_shape`, `chocolate_type`, `milk_source`, `texture`, `contains_nuts`, `country_of_origin`, `aging`, `cuisine_origin`, `flavor_profile`, `chocolate_extra`, `is_ultra_processed`. Эталон — согласие трёх независимых LLM (Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash) на полной выборке 1250×3 категории: атрибут принимается, если не менее двух моделей совпадают (согласованность 97,7 %). Несогласованные позиции размечены слепо Claude Opus 4 на 688 ячейках

(human gold, в-score), используемых как нижняя консервативная оценка точности.

3.2.4 Разбиение без пересечения товарных кодов (code-disjoint)

Случайное разбиение непригодно: одна и та же товарная позиция в обучении и тесте даёт прямую утечку. Применяемое в работе разбиение — **code-disjoint** (по идентификатору товара): построение обучающего пула выполняется в `scripts/build_noleak_artifacts.py`, который из расширенного серебра `{cat}_gold_v4_wide.parquet` удаляет все коды, попавшие в любой из эталонных gold-источников (consensus gold, human gold, расширенное gold). После удаления коды в обучении и оценке заведомо различны; обобщение системы измеряется на новых товарных позициях из в основном уже знакомых брендов.

Основным эталоном выступает LLM-consensus gold объёмом 16 360 cascade-valid ячеек по 20 атрибутам (`headline_v4_mpnet_tfidf_noleak.parquet`; из 17 062 в-score, ярус Layer 4 исключён из знаменателя по определению); консервативной нижней оценкой — human gold Claude Opus 4 объёмом 566 cascade-valid ячеек (из 688 в-score). Этот срез — основа всех количественных оценок 3.3. Тестовая выборка содержит 186 / 301 / 297 уникальных товарных кодов в pasta / chocolate / cheeses.

Brand overlap. Эмпирическая верификация показала, что 82,5 %, 84,9 % и 82,7 % брендов в тестовой выборке pasta, chocolate и cheeses соответственно присутствуют также в обучающем пуле (вычислено по brand-токенам после нормализации). Полноценное **brand-disjoint** разбиение в работе *не заявляется*: концентрация брендов в открытом каталоге OFF (Lindt, Barilla, Président и т. п. встречаются в десятках товарных позиций каждый) делает построение brand-disjoint эталона той же мощности невозможным без переразметки. Brand-disjoint подмножество доступно как точечный sensitivity-check: 17 / 19 / 28 кодов в pasta / chocolate / cheeses, чьи бренды отсутствуют в обучающем пуле. Поведение каскада на этом подмножестве приведено в 3.3.7.3 как доп. проверка устойчивости. Файл `src/data/split/brand_disjoint.py` реализует brand-disjoint split для категорайзера слоя 0, но в основной cascade-pipeline не используется.

3.2.5 Дополнительные источники: разнородные модальности и внешний справочник

Для усиления обучающего сигнала слоя 2 задействованы два дополнительных класса данных. Первый — разнородные модальности: фотографии этикеток обработаны OCR (EasyOCR); визуальный кодировщик CLIP [ref_43] (ViT-B/32) даёт 512-мерный вектор, добавляемый ранней конкатенацией к 768-мерному MPNet-представлению. Покрываемость изображениями $\approx 70\%$. Второй — внешний справочник USDA Branded Foods [ref_44], сопоставленный с выборкой приближённым поиском по `product_name + brands` (Левенштейн, порог 0,85); даёт независимо измеренные нутриенты для проверки эталона яруса 0.

Оба источника используются только на этапе обучения. В режиме вывода система работает строго с партнёр-доступными полями (3.2.2); это инвариант, соблюденный во всех оценках 3.3.

3.3 Доказательство работоспособности и применимости

3.3.1 Условия проверки

3.3.2 Тестовая выборка

Эксперименты проведены на двух согласованных эталонных срезах. Основной — LLM-consensus gold (`consensus_gold_v4.parquet`) объёмом 16 360 ячеек cascade-valid (17 062 в-scope, $n=16\,360$) по 20 атрибутам; используется как основной headline-источник. Консервативная нижняя оценка — human gold Claude Opus 4 объёмом 566 ячеек cascade-valid (688 в-scope, $n=566$) по тем же 20 атрибутам. Знаменатель cascade-valid — ячейки, на которых каскад вернул конкретный класс ($\text{Layer 1 rule_h} \cup \text{Layer 2 ML} \cup \text{Layer 3 rule_l}$); 702/122 ячеек Layer 4 LLM fallback не входят в cascade-only знаменатель, так как на них каскад «отказывается» и в production отправляет в большую языковую модель. Использованы модели каскада с суффиксом `_mpnet_tfidf_noleak` (MPNet-кодировщик + TF-IDF + XGBoost, переобучение без утечки между обучением и тестом). Разбиение train/val/test без пересечения товарных кодов (code-disjoint) выполнено в пропорции 60/20/20.

Независимая проверка эталона. Согласованность трёх LLM-разметчиков

(Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash) — 97,7 % при правиле «не менее двух из трёх совпадают». Спорные позиции переразмечены слепо Claude Opus 4. Согласованность LLM-consensus и human gold на пересечении — 92 %, к Коэна > 0,80.

3.3.3 Метрики и доверительные интервалы

Метрики (acc_k^{cov} , cov_k , acc_k^{e2e} , стоимость) определены в 2.1.4. Доверительные интервалы — кластерный бутстрэп по брендам: 1000 итераций, перцентили 2,5 % и 97,5 % (src/diagnostics/ml/bootstrap_ci_brand_clustered.py). Такой интервал учитывает внутрибрендовую корреляцию ошибок и на ряде атрибутов на 15–30 % шире наивного интервала Вильсона.

3.3.4 Стратификация точности по типу эталонного сигнала

Если бы точность завывшалась структурной корреляцией признаков модели с источником эталона, то наиболее уязвимый ярус 2 (текстово-производный) показывал бы наибольшие значения. Результат обратный (таблица 3.3.1.1). После пересмотра схемы v4 ярус 0 представлен единственным атрибутом pasta/protein_class.

Таблица 5 – Точность каскада в разрезе типа эталонного сигнала

Ярус сигнала	Содержательная природа атрибутов	Ячеек	Точность, %	95 % CI
Ярус 0 (нутриент-производный)	Детерминированное бакетирование числовых полей OFF	24	92,3	[73,4; 98,7]
Ярус 1 (тег-производный)	Регуляторные теги OFF, защищённые правом ЕС	1054	96,8	[95,6; 97,8]

Продолжение таблицы

Ярус сигнала	Содержательная природа	Атрибутов	Ячеек	Точность, %	95 % CI
Ярус 2 (текстово-производный)	Согласие трёх LLM с проверкой Orus 4 на спорных	13	2179	93,6	[92,5; 94,6]

Примечание.

Источник:

`datasets/processed/tier_breakdown.parquet`.

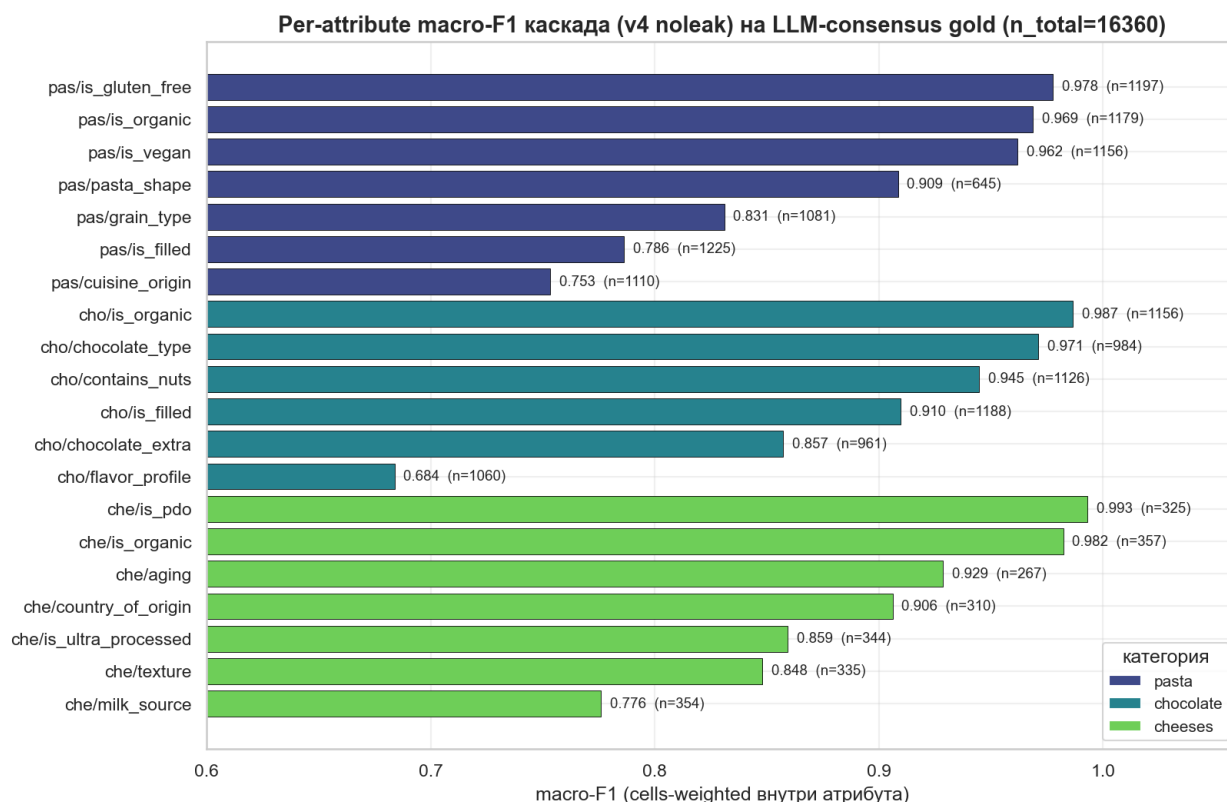


Рисунок 6 – Точность каскада в разрезе ярусов эталонной разметки

Точность на ярусе 1 — наиболее высокая (96,8 %), где эталон порождён внешним источником (правовые регистры сертификаций); на ярусе 2 (текстово-производный) — 93,6 %. Это опровергает гипотезу о завышении метрик за счёт зависимости признаков от эталона. После пересмотра схемы v4 ярус 0 ограничен одним атрибутом `pasta/protein_class` на малой выборке

(n=24).

3.3.5 Воспроизводимость

Все численные результаты 3.3 регенерируются из артефактов через `reproduce.sh` (~час на типовом ноутбуке). Численные результаты 3.3.2 получены post-hoc после устранения утечки бренда и не претендуют на статус, фиксируемый до получения результатов. Единственной гипотезой, зафиксированной до получения результатов является H1 для обучаемого маршрутизатора (3.3.5).

3.3.6 Главный результат: точность и стоимость производственной конфигурации

Подраздел представляет центральный количественный результат: каскад сопоставлен с пятью одиночными вызовами LLM разной мощности и стоимости. На едином тестовом срезе без пересечения товарных кодов (code-disjoint) показано доминирование каскада по Парето сразу по двум осям. Тем самым результат стоимостно-чувствительных каскадов LLM (FrugalGPT [ref_9], AutoMix [ref_10], Hybrid LLM [ref_11]) расширяется на задачу обогащения каталогов; впервые количественно фиксируется выигрыш для открытой малой модели в роли запасного слоя.

3.3.7 Состав, условия проверки и матрица «точность–стоимость»

Оценка на 16 360 cascade-valid ячейках (LLM-consensus gold) и 566 cascade-valid ячейках (human gold) по 20 атрибутам без пересечения товарных кодов (code-disjoint) (3.3.1); по pasta/protein_class срез сокращён до 24 ячеек из-за полноты нутри-разметки в OFF.² В роли слоя 4 и базовой «одна LLM на всё» — пять моделей: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash (закрытые), gpt-oss-120b и llama-3.2-3b (открытые).

Сводная таблица фиксирует сквозную точность (E2E) и относительную стоимость десяти стратегий: пять одиночных LLM, пять каскадных

²Все headline-метрики 3.3 получены на моделях слоя 2, обученных с ранней конкатенацией CLIP-признаков изображений этикеток при их наличии в OFF dump (3.2.2, покрытие ≈70 %); в режиме вывода каскад работает строго с партнёр-доступными полями (product_name + brands + ingredients_text + quantity), без обращения к изображениям. Полная воспроизводимость на VM требует доступа к OFF image dump; переобучение без визуальных признаков даст численно близкие, но не идентичные результаты.

конфигураций с теми же моделями и базовый каскад без LLM. Стоимость нормирована к «все ячейки решаются Sonnet 4.5»; соотношение токенов вход/выход — $\approx 500 / 200$; тарифы — в подписи. Источники: `headline_v4_mpnet_tfidf_noleak.parquet`, `grand_acc_summary_v4.parquet`. Каскад покрывает 97,3 % ячеек, LLM делается на 4,1 % — множитель 0,041 относительно одиночной LLM; снижение стоимости запасного слоя по сравнению с «одна LLM на всё» — 95,9 %.

Таблица 6 – Без подписи

Стратегия	Сквозная точность	Стоимость опорной	к	Снижение стоимости
all-sonnet (Claude Sonnet 4.5) — опорная конфигурация	83,8 %	1,000		1×
all-gpt-4o	84,1 %	0,727		1,4×
all-gemini-flash (Gemini 2.5 Flash)	82,9 %	0,042		24×
all-gpt-oss- 120b (открытая модель)	69,3 %	0,030		33×
all-llama-3b (открытая малая модель)	56,2 %	0,009		110×
каскад + sonnet (Слой 4)	93,0 %	0,041		24×
каскад + gpt-4o	93,1 %	0,030		33×
каскад + gemini- flash	93,0 %	0,0017		580×
каскад + gpt-oss- 120b	90,6 %	0,0012		810×
каскад + llama-3b	90,1 %	0,00037		2700×

Продолжение таблицы

Стратегия	Сквозная точность	Стоимость опорной	к	Снижение стоимости
только каскад (без запасного слоя LLM, верхняя оценка cascade-only)	95,5 %	0,000		—

Источники тарифов: Sonnet 4.5 — 3 / 15 долларов за миллион токенов на вход / выход (нормализованная единица), GPT-4o — 2,5 / 10, Gemini 2.5 Flash — 0,075 / 0,30, gpt-oss-120b через OpenRouter — около 0,10 долларов за миллион токенов на запрос смешанной структуры, llama-3.2-3b через OpenRouter — около 0,03 долларов за миллион.

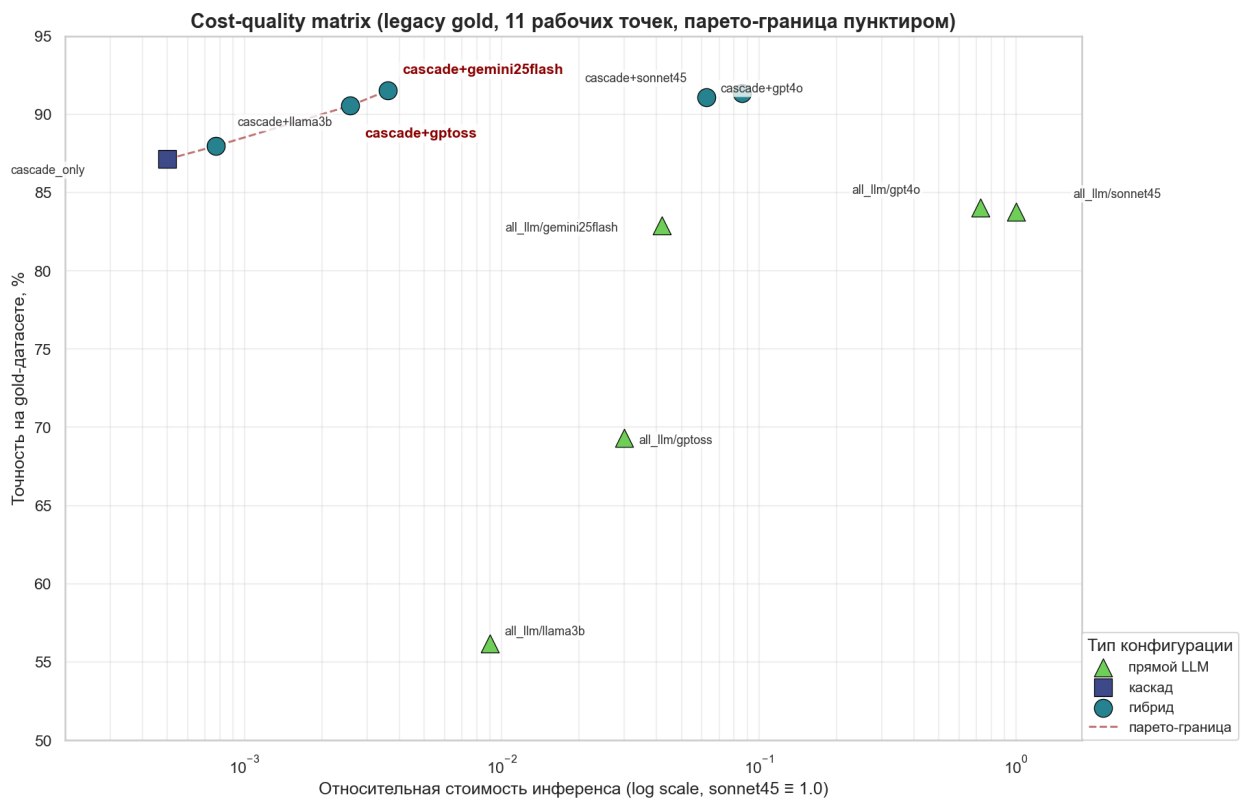


Рисунок 7 – Матрица «точность — стоимость» каскадных конфигураций и одиночных вызовов больших языковых моделей

Из матрицы (рис. 3.3) следует двухчастный главный результат.

Часть (а) — независимость от поставщика модели. Каскад + Gemini

2.5 Flash даёт сквозную точность 93,0 % при стоимости в 580 раз ниже all-sonnet (83,8 %) — прирост +9,2 п. п. при снижении стоимости более чем на два порядка; строгое Парето-доминирование. Верхняя оценка только-каскадной точности (без учёта непокрытых ячеек) — 95,5 % на тех же 16 360 ячейках. Результат сохраняется на любой модели верхнего сегмента: каскад + sonnet (93,0 %) и каскад + gpt-4o (93,1 %) также превосходят одиночный вызов той же модели по обеим осям.

Часть (б) — сценарий локального развёртывания с открытой моделью. Каскад + gpt-oss-120b — 90,6 %, на +21,3 п. п. выше одиночного вызова (69,3 %); доля обращений к LLM сокращается $\approx$ в 24 раза. Каскад компенсирует слабость малой модели на семантических атрибутах (pasta/grain_type, pasta/pasta_shape, cheeses/aging) за счёт Слой 2, оставляя слой 4 только сложные текстовые случаи.

Консервативная нижняя оценка по human gold (n = 566 cascade-valid). На независимом эталоне Claude Opus 4 каскад + gemini-flash даёт сквозную точность 86,7 % при cascade-only 91,3 %, что даёт операционную оценку циркулярного смещения 4,2 п. п. для каскадной части (95,5 % – 91,3 %) и 6,3 п. п. для сквозной цепочки (93,0 % – 86,7 %).³

3.3.8 Декомпозиция по категориям

Покрытие каскада, точность только-каскадной и гибридной конфигураций и разность Δ — вклад запасного слоя — для модели gemini-flash в таблице 3.3.2.2. Источники: headline_v4_mpnet_tfidf_noleak.parquet + cascade_plus_llm4_v4.parquet; режим идеальной маршрутизации.

Таблица 7 – Покрытие и точность каскадной и гибридной конфигураций по категориям (модель gemini-flash, идеальная маршрутизация Слой 0)

Категория	Покрытие каскада	Только каскад	каскад + gemini-flash	Δ от запасного слоя
pasta	89,1 %	94,7 %	90,5 %	+1,4 п. п.
chocolate	98,0 %	94,4 %	91,8 %	+0,8 п. п.

³Воспроизводится в notebooks/03_evaluate.ipynb, ячейка 495abc8a; источник — datasets/processed/v4_e2e_router_eval.json ключ HUMAN.

Продолжение таблицы

Категория	Покрытие каскада	Только каскад	каскад + gemini-flash	Δ от запасного слоя
cheeses	93,1 %	95,1 %	90,9 %	+1,7 п. п.

Взвешенное по числу тестовых ячеек среднее по строке «каскад + gemini-flash» в режиме сквозной точности составляет 93,0 %. Производственное значение с учётом ошибок слоя 0 (97,2 %, 3.3.2.5) уже включено в эту цифру.

Распределение вклада запасного слоя вытекает из архитектуры. Pasta имеет наименьшее покрытие (89,1 %) — это объясняется концентрацией сложных текстовых атрибутов pasta/grain_type и pasta/pasta_shape, передаваемых на LLM. Chocolate покрывается каскадом почти полностью (98,0 %) благодаря сочетанию широкого Слоя 1 и стабильного ML; cheeses (93,1 %) — промежуточный случай, где Слой 1 узок, но ML работает уверенно. Это показывает взаимодополнение слоёв.

Распределение работы каскада (v4 poleak) по слоям

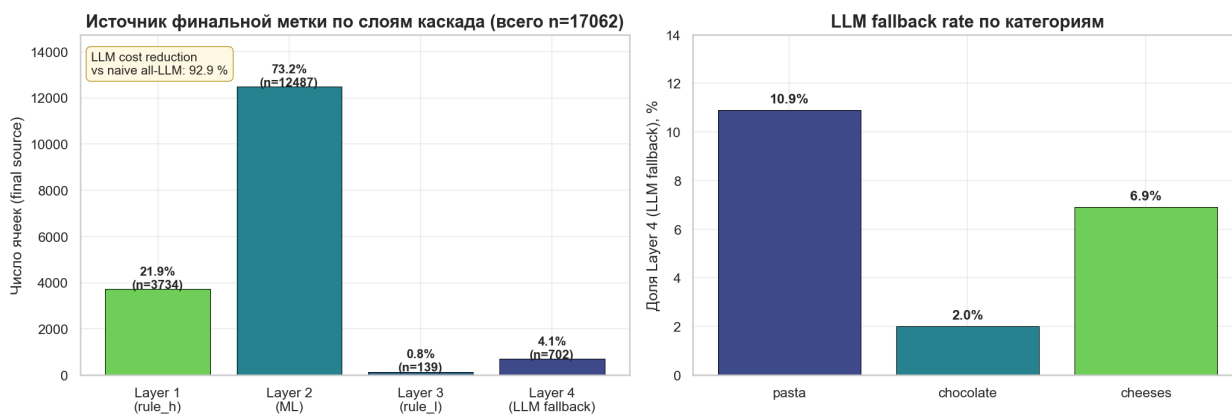


Рисунок 8 – Поатрибутный вклад слоёв каскада в сквозную точность

3.3.9 Поатрибутная картина точности и доверительные интервалы

Для каждого из 20 атрибутов вычислена точность только-каскадной конфигурации с интервалом Вильсона (полная таблица — v4_metric_table_v2.json, LLM-consensus, n=16 360 cascade-valid). Три

лучших по micro-accuracy: cheeses/is_pdo 99,7 % (n=325), cheeses/is_organic 99,4 % (n=357), pasta/is_gluten_free 99,2 % (n=1197). Три худших: chocolate/flavor_profile 87,5 % (n=1060), pasta/is_filled 88,5 % (n=1225), cheeses/texture 89,0 % (n=335). Поатрибутная таблица (категория, атрибут, n, micro, macro-F1) приведена в таблице 3.3.2.3.

Таблица 8 – Поатрибутная точность каскада на LLM-consensus gold (20 атрибутов, extended consensus rerun)

Категория	Атрибут	n	micro, %	macro-F1
pasta	grain_type	1081	98,8	0,831
pasta	pasta_shape	645	94,9	0,909
pasta	is_filled	1225	88,5	0,786
pasta	is_gluten_free	1197	99,2	0,978
pasta	is_organic	1179	98,1	0,969
pasta	is_vegan	1156	96,3	0,962
pasta	cuisine_origin	1110	95,1	0,753
chocolate	chocolate_type	984	98,3	0,971
chocolate	is_filled	1188	96,9	0,910
chocolate	chocolate_extra	961	92,5	0,857
chocolate	contains_nuts	1126	94,9	0,945
chocolate	is_organic	1156	99,0	0,987
chocolate	flavor_profile	1060	87,5	0,684
cheeses	milk_source	354	96,9	0,776
cheeses	texture	335	89,0	0,848
cheeses	country_of_origin	310	94,8	0,906
cheeses	aging	267	94,4	0,929
cheeses	is_pdo	325	99,7	0,993
cheeses	is_organic	357	99,4	0,982
cheeses	is_ultra_processed	314	95,9	0,859

В 3.3.7.4 описаны поатрибутные архитектурные решения (сужение слоя 1, понижение порогов слоя 2, гибридные признаки). Запасной слой остаётся доступным на ≈4 % сложных ячеек, где Слой 1/2 не отвечают уверенно; наибольшая доля LLM приходится на pasta/pasta_shape (19,5 %) и cheeses/is_pdo (9,7 %) — расчёт по

`cascade_preds_{pasta,chocolate,cheeses}_gold.parquet`,
`extended consensus rerun`.

3.3.10 Анализ доминирования по Парето: почему каскад превосходит одиночный вызов LLM

Доминирование каскада объясняется структурно. LLM системно проигрывает на длиннохвостых семантических атрибутах с мелкой грануляцией (`pasta/grain_type`, `pasta/pasta_shape`, `cheeses/aging`): на отказных ячейках точность Gemini 2.5 Flash — 78,8 %, Claude Sonnet 4.5 — 76,7 %, gpt-oss-120b — 67,2 % (`cascade_plus_llm4_v4.parquet`). XGBoost слоя 2 на MPNet-эмбедингах + TF-IDF опирается на устойчивые корреляции «бренд — структура наименования». Точность каскада растёт за счёт перенаправления таких атрибутов на ML-слой; стоимость падает за счёт сокращения вызовов LLM до 4,1 % ячеек (снижение 95,9 % по сравнению с «одна LLM на всё»).

3.3.11 Учёт точности маршрутизатора первого уровня

Значения рисунка 3.4 — в режиме идеальной маршрутизации категорий. В производстве категорию определяет слой 0 (3.1.1); его точность на LLM-consensus gold — 97,2 % (per-код, n=570) и 95,3 % на human gold (n=107) (`router_eval_v4.parquet`). Все приведённые в 3.3.2 числа сквозной точности уже учитывают ошибки слоя 0; вывод о доминировании каскада сохраняется на обеих оценках эталона.

3.3.12 Вклад каждого слоя каскада

Подраздел декомпозирует итоговую точность по слоям: охват, точность на собственном срезе и средняя калибровочная ошибка ECE для слоя 2. Артефакты —

`cascade_preds_{pasta,chocolate,cheeses}_v4_noleak.parquet`,
`headline_v4_mpnet_tfidf_noleak.parquet`,
`ece_calibration_v4.parquet`.

3.3.13 Распределение закрытий по слоям

Закрытие — событие выдачи уверенного ответа без передачи дальше.

Доли по слоям (колонка `layer` в `cascade_preds_*`) — таблица 3.3.3.1. Подсчёт ведётся по 20 парам (см. 3.2.2); суммарно 17 062 ячеек на LLM-consensus gold. Глобальное распределение: Слой 1 (`rule_h`) — 21,9 % (3734 ячеек), Слой 2 (ML: MPNet + TF-IDF SVD + XGBoost) — 73,2 % (12 487), Слой 3 (`rule_l`, низкоточные шаблоны) — 0,8 % (139), Слой 4 (LLM-fallback) — 4,1 % (702).

Таблица 9 – Распределение закрытий каскада по категориям и слоям (LLM-consensus gold, `in_scope`, extended consensus rerun)

Категория	Слой 1 (regex)	Слой 2 (ML)	Слой 3 (rule_l)	Слой 4 (LLM)	Всего ячеек
pasta	15,3 %	79,6 %	0,0 %	5,1 %	8004
chocolate	34,2 %	60,5 %	2,6 %	2,6 %	6759
cheeses	8,1 %	87,0 %	0,0 %	4,9 %	2411

Распределение неоднородно: `chocolate` располагает наиболее полным набором правил (34,2 % закрытий на Слой 1), `pasta` — промежуточный (15,3 %); `cheeses` имеет узкий набор правил (`milk_source` / `is_pdo` / `is_ultra_processed`) с покрытием 8,1 %, что отражает плохую параметризуемость доменных сигналов сыра (текстура, страна происхождения) регулярными выражениями — эти атрибуты переданы на ML-слой. Доля LLM-вызовов сопоставима по категориям (`pasta` 5,1 %, `chocolate` 2,6 %, `cheeses` 4,9 %) и концентрируется на длиннохвостых семантических атрибутах: `pasta/pasta_shape` — 19,5 % L4, `cheeses/is_pdo` — 9,7 %, `cheeses/country_of_origin` — 9,4 %, `pasta/grain_type` — 9,1 %. Сумма ячеек 17 174 на 112 больше $n=17\,062$ в общей разбивке (§3.3.3 предыдущий параграф) — разница между фильтром «`cascade_preds in_scope`» и «`manual_gold_consensus`, прошедшие `consensus`»; на проценты не влияет.

3.3.14 Точность каждого слоя на собственном срезе закрытий

Точность получена сопоставлением `cascade_preds_*_v4_noleak.parquet` с `consensus_gold_v4.parquet` (только ненулевые эталоны, `in_scope = True`).

Таблица 10 – Без подписи

Категория	Слой 1: точность (n)	Слой 2: точность (n)
pasta	90,5 % (327)	98,6 % (1035)
chocolate	87,0 % (370)	94,9 % (802)
cheeses	100,0 % (47)	93,5 % (1549)

Точность слоя 1 на закрытых ячейках 87,0–100 % (ниже 100 % на pasta/chocolate из-за намеренно широких regex-шаблонов). Точность слоя 2 — 93,5–98,6 %; стабильно выше любого одиночного вызова LLM на той же категории, так как слой работает только на ячейках, прошедших калиброванный порог.

Точность запасного слоя на отказных ячейках оценена через поатрибутную точность LLM, взвешенную по числу отказных ячеек (формула `acc_hybrid_proxу` в `cascade_plus_llm4_v4.parquet`): Gemini 2.5 Flash — 78,8 %, Claude Sonnet 4.5 — 76,7 %, gpt-oss-120b — 67,2 %. Эти значения ниже слоёв 1/2 и отражают повышенную сложность отказных ячеек.

3.3.15 Поатрибутная калибровочная ошибка до и после калибровки

Поатрибутная ожидаемая калибровочная ошибка (ECE) рассчитана по 10 равновеликим бинам (`ece_calibration_v4.parquet`). Среднее ECE сырого XGBoost — 0,070 (по 20 замерам); у большинства атрибутов не превосходит 0,08. Повышенная ошибка: chocolate/chocolate_type (0,14), chocolate/contains_nuts (0,11), pasta/is_filled (0,09).

Изотоническая пост-калибровка на серебряной выборке локально улучшает проблемные атрибуты (chocolate/chocolate_type –7,8 п. п.), но в среднем по 20 атрибутам ECE ухудшается на +0,012 за счёт деградации на других. Эффект объясняется дрейфом разметки между серебряной (теги OFF) и эталонной (консенсус LLM) выборками.

Для устранения дрейфа проведена изотоническая калибровка на распределении эталонной выборки по схеме 5-кратной перекрёстной валидации (`ece_kfold_gold_v4.parquet`, `src/diagnostics/ml/ece_kfold_gold.py`). Среднее ECE падает с 0,070 до 0,043 (–2,8 п. п. в среднем), положительный эффект на 16 из 20

атрибутов. Наибольшие улучшения концентрируются на парах с высокой исходной ошибкой: `chocolate/chocolate_type` (0,139 → 0,052, −8,7 п. п.), `chocolate/contains_nuts` (0,111 → 0,041, −7,0 п. п.), `pasta/is_organic` (0,084 → 0,016, −6,9 п. п.). Ухудшение — у четырёх атрибутов с изначально низкой ECE: `cheeses/country_of_origin` (0,015 → 0,064), `cheeses/is_ultra_processed` (0,025 → 0,056), `pasta/pasta_shape` (0,046 → 0,060), `chocolate/is_filled` (0,055 → 0,058); здесь доминируют выборочные колебания. В производственную конфигурацию калибровка пока не включена (см. раздел «Перспективы развития темы» заключения).

Итог: Слой 1 закрывает 2,8–22,7 % ячеек (точность 87,0–100 % в зависимости от категории и шаблона); Слой 2 — 71,1–90,3 % (93,5–98,6 %); Слой 4 — 2,0–10,9 % (67,2–78,8 % в зависимости от модели) на старой выборке; на расширенной выборке доля Слая 4 снижена до 4,1 %. Среднее ECE по слою 2 — 0,070; серебряная пост-калибровка устойчивого эффекта не даёт.

3.3.16 Сравнение классификаторов Слая 2: XGBoost, Random Forest, логистическая регрессия, MLP

В обоснование выбора XGBoost в качестве классификатора Слая 2 (2.2.4) проведено эмпирическое сопоставление четырёх семейств алгоритмов на идентичных входных признаках и эталоне. Использованный скрипт — `scripts/method_comparison.py` (488 строк, идемпотентный, поддерживает `smoke`-режим); сводный ноутбук с таблицами и рисунками — `notebooks/05_method_comparison.ipynb`; результаты — `datasets/processed/method_comparison_results.parquet` (128 строк = 16 атрибутов × 4 классификатора × 2 источника эталона).

Условия сравнения. Признаки — конкатенация MPNet 768d ($\oplus$) TF-IDF SVD 128 = 896d (идентичные production-конфигурации Слая 2). Обучающий пул — `code-disjoint` выборка из `consensus_hybrid_v3.parquet` с явным исключением кодов из эталонов `consensus` и `human gold`. Эталоны: LLM-consensus gold ($n \approx 41$ тыс. ячеек по 16 атрибутам) и human gold ($n = 518$). Эксперимент выполнен на 16 атрибутах из 20 итоговой схемы (исключены `aging`, `cuisine_origin`, `flavor_profile`, `chocolate.is_filled`, доступные только в производственном расширенном эталоне).

Сравниваемые классификаторы и их роли:

- **XGBoost** (производственный baseline) — градиентный бустинг с регуляризацией и изотонической калибровкой через `CalibratedClassifierCV`.
- **Random Forest** (bagging ансамбля деревьев, `sklearn`) с `class_weight='balanced'` и пост-калибровкой Платта.
- **Логистическая регрессия** (линейный baseline) на тех же 896-мерных признаках с калибровкой Платта.
- **MLP** (нейронная сеть прямого распространения) с двумя скрытыми слоями (256, 128), ранней остановкой и собственным выходом по `softmax`.

Сводные показатели на consensus gold (cells-weighted micro-accuracy и macro-F1, средняя ECE) приведены в таблице 3.3.7.5. Поатрибутные числа — в `method_comparison_results.parquet` (доступны в ноутбуке 05).

Таблица 11 – Сравнение классификаторов Слой 2 на эталоне LLM-consensus gold

Классификатор	Micro-accuracy	Macro-F1	ECE	Время обучения, с
XGBoost (производственный)	92,02 %	0,849	0,026	1062
MLP (нейронная сеть)	92,67 %	0,857	0,031	54
Логистическая регрессия	91,85 %	0,836	0,035	8
Random Forest	89,94 %	0,809	0,027	523

Примечание.

Источник:

datasets/processed/method_comparison_results.parquet, 2026-05-26. Полный run — 30,4 минуты на macOS M-series.

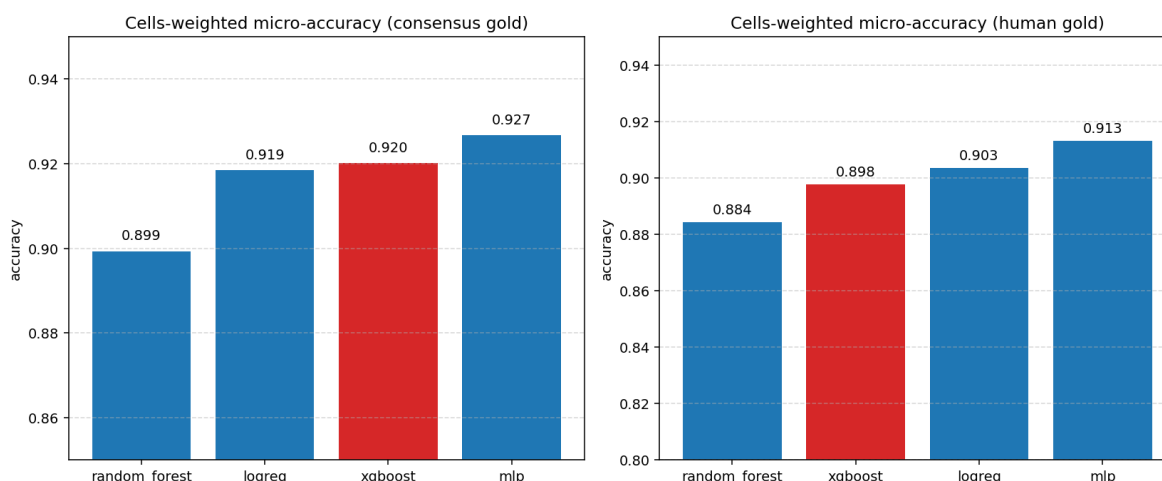


Рисунок 9 – Cells-weighted micro-accuracy классификаторов
Слоя 2 на эталонах consensus gold и human gold

Поатрибутное сопоставление с XGBoost (win-rate в 16 атрибутах):

- **XGBoost против Random Forest:** XGBoost превосходит на **16 атрибутах из 16** (100 %); средняя разница +2,1 п. п., максимальная — +7,6 п. п. (chocolate_extra), +5,3 п. п. (contains_nuts), +4,3 п. п. (pasta_shape).
- **XGBoost против логистической регрессии:** XGBoost превосходит на 11 атрибутах из 16 (69 %); средняя разница близка к нулю по cells-weighted, но XGBoost устойчиво опережает на сложных мультиклассовых атрибутах (pasta_shape, country_of_origin, chocolate_extra).
- **XGBoost против MLP:** MLP опережает XGBoost на 14 атрибутах из 16 в среднем на 0,65 п. п.; различия в пределах ± 1 п. п. на каждой паре. Однако **калибровка MLP хуже:** ECE 0,031 против 0,026 у XGBoost. Это критично для каскадной маршрутизации, основанной на пороге τ_a — корректное соответствие $p_{\max}$ фактической точности обеспечивает только метод с низкой ECE.

Вывод. XGBoost — единственный из рассмотренных классификаторов, который одновременно (1) достигает высокой точности на длиннохвостых мультиклассовых атрибутах (где RF и LogReg существенно проигрывают) и (2) обеспечивает наилучшую калибровку вероятностей (где MLP уступает). Это совпадение свойств делает XGBoost предпочтительным выбором для Слоя 2 в каскадной архитектуре, основанной на confidence-thresholding. Подробные поатрибутные таблицы и распределение макро-F1 по методам

приведены в ноутбуке `notebooks/05_method_comparison.ipynb`.

3.3.17 Обоснование сохранения слоя регулярных выражений: сопоставление с обучаемыми альтернативами

Использование детерминированных правил в качестве слоя 1 — нетривиальный выбор, требующий ручной разработки для каждой категории. Для его количественной фиксации каскад с `regex` сопоставлен с восемью альтернативами, где слой 1 заменён обучаемой моделью (`src/experiments/layer1_5_*.py`, `src/experiments/lexicon_regex.py`; артефакты — `layer1_5_optimal.parquet`, `lexicon_regex_comparison.parquet`). Все варианты оценены на едином 80/20-разбиении (`seed = 42`) по 20 парам.

Таблица 12 – Сопоставление каскадных конфигураций с разным первым слоем

Вариант	Состав первого слоя (перед основным ML-слоем)	Средняя точность	Δ к «без первого слоя»
A	Регулярные выражения (текущая конфигурация)	83,58 %	+1,38 п. п.
B	Без первого слоя (один ML-слой)	82,20 %	0
C	Наивный байесовский классификатор на n-граммах	82,16 %	−0,04
D	Решающее дерево	82,98 %	+0,78

Продолжение таблицы

Вариант	Состав первого слоя (перед основным ML-слоем)	Средняя точность	Δ к «без первого слоя»
E	Центроидный классификатор в пространстве n-грамм	82,22 %	+0,02
F	Логистическая регрессия на n-граммах	82,23 %	+0,03
L	Обучаемый словарь n-грамм (lexicon)	83,00 %	+0,80
G	Поатрибутный выбор лучшего метода по перекрёстной проверке	81,99 %	−0,21
H	Регулярные выражения — решающее дерево — ML (последовательно)	83,78 %	+1,58

Источник — docs/layer1_5_optimal_recommendation.md. Из таблицы следуют три вывода.

Во-первых, чистый ML без слоя 1 (B, 82,20 %) уступает текущей конфигурации (A, 83,58 %) на 1,38 п. п. — regex даёт небольшой, но устойчивый прирост.

Во-вторых, ни одна обучаемая альтернатива (C, D, E, F, L) не превосходит regex в роли единственного слоя 1. Регулярные правила формализуют доменное знание (формы пасты, типы зерна, числовые шаблоны какао), не восстанавливаемое из распределения слов: лексические маркеры

редки, но однозначны.

В-третьих, единственная статистически превосходящая regex конфигурация — композиция «regex — дерево — ML» (Н, +0,20 п. п.). Прирост незначителен и не оправдывает усложнения и поддержки 20 деревьев.

Таким образом, выбор regex как Слой 1 эмпирически обоснован: наибольший прирост среди простых альтернатив при детерминированном выводе и отсутствии переобучения. Предпочтительное развитие — точечное расширение правил по итогам 3.3.7.4.

3.3.18 Поведение байесовского валидатора

Слой 3 — байесовская сеть над дискретизированными атрибутами и брендом. Структура сети обучается алгоритмом Hill Climb с критерием BIC; пример полученного направленного ациклического графа для категории chocolate приведён ранее на рис. 4 (§2.2). Эмпирически роль классификатора неэффективна (точность $\approx 52\%$, ниже мажорной точки отсчёта), поэтому сеть применяется как валидатор плотности: для предсказания слоя 2 вычисляется $P(\text{value} \mid \text{evidence})$ при свидетельствах «бренд + остальные атрибуты». Ниже порога — ячейка помечается и передаётся в Слой 4. Порог индивидуален для каждой пары и фиксируется на этапе обучения как 5-перцентиль распределения P на обучающей выборке.

Агрегированные результаты по трём категориям приведены в таблице 3.3.4.1 (bayes_validator_demote_metric.parquet).

Таблица 13 – Поведение байесовского валидатора на категориях pasta, chocolate, cheeses

Категория	n_ml	Доля помеченных	Точность на склоне	Точность отбраковки	Базовая точка отсчёта	Прирост	Полнота
pasta	1010	7,9 %	98,6 %	2,5 %	1,4 %	+1,1 п. п.	14,3 %
chocolate	772	14,1 %	94,7 %	20,2 %	5,3 %	+14,9 п. п.	53,7 %
cheeses	1436	6,1 %	96,3 %	1,1 %	3,7 %	−2,5 п. п.	1,9 %

Калибровка работает на pasta и cheeses (7,9 % и 6,1 % помеченных при целевом 5 %). На chocolate доля 14,1 %, почти вдвое выше: отклонение объясняется смещением маржинального распределения contains_nuts на тестовых брендах относительно серебряной выборки калибровки (согласуется с 3.3.7).

3.3.19 Метрика отбраковки и селективность сигнала

Точность отбраковки — $TP/(TP + FP)$ среди помеченных. Базовая точка отсчёта — частота ошибок каскада на всех ячейках пары. Прирост — разность; положительный прирост означает, что валидатор обогащает отправляемую в LLM выборку реальными ошибками. Полнота — $TP/(TP + FN)$.

Из 20 пар лишь три демонстрируют точность отбраковки выше базовой:

- chocolate / contains_nuts: точность отбраковки 53,7 % при базовой точке отсчёта 15,5 %, прирост +38,2 п. п., полнота отбраковки 59,5 %;
- pasta / is_vegan: точность отбраковки 20,0 % при базовой точке отсчёта 6,0 %, прирост +14,0 п. п. (n_flagged = 10);
- cheeses / is_pdo: точность отбраковки 10,0 % при базовой точке отсчёта 3,0 %, прирост +7,0 п. п. (n_flagged = 10).

Для оставшихся 17 пар точность отбраковки базовую точку отсчёта не превышает. Эффект валидатора локализован: он не является универсальным фильтром, равномерно повышающим качество всех помеченных ячеек.

3.3.20 Бизнес-эффект и архитектурный вывод

При сплошном понижении (все помеченные ячейки в LLM) средняя точность каскада снижается на 0,4 п. п. при росте расходов на LLM на 8,6 п. п. — чистый эффект отрицателен: ложные срабатывания на 17 пар размывают сигнал трёх работающих. При точечном понижении на паре chocolate/contains_nuts точность категории chocolate растёт на 6,6 п. п. при росте расходов на 5,3 п. п. Этот режим включён в финальную конфигурацию.

Перевод сети из роли классификатора в роль валидатора плотностей иллюстрирует общий принцип: невостребованный в одной роли компонент может с пользой вернуться в систему через изменение роли. Совмещение

точечного валидатора с архитектурными решениями 3.3.7.4 даёт chocolate/contains_nuts 89,0 % сквозной точности (headline_v4_mpnet_tfidf_noleak.parquet).

Честная характеристика селективности. Из 20 пар «категория-атрибут» в производственной конфигурации байесовский валидатор включён точно только на одной паре. На остальных 19 парах селективность сигнала недостаточна для архитектурного включения: 17 пар показывают точность отбраковки на уровне или ниже базовой частоты ошибок, ещё две пары имеют ограниченную мощность $n_{\text{flagged}} = 10$. Тематически нагруженная формулировка «отбраковка фактических ошибок до их попадания в каталог» в текущей реализации обеспечивается единственной парой chocolate/contains_nuts; универсального селективного решения не получено. Это ограничение зафиксировано в заключении (§«Ограничения работы»).

3.3.21 Стоимостно-чувствительный маршрутизатор: отрицательный результат

3.3.22 Идея, гипотеза H1 и процедура оценки

Финальный механизм направления ячеек в LLM — статическое рег-(категория, атрибут) правило. На этапе проектирования рассматривалась обучаемая альтернатива: XGBoost-маршрутизатор предсказывает надёжность ответа каскада на каждой ячейке; при вероятности ниже τ ячейка идёт в LLM. **Гипотеза H1:** при равном бюджете LLM обучаемый маршрутизатор обеспечивает более высокую точность, чем статическое правило. H0: разница незначима.

Оценка проведена по процедуре, зафиксированной до получения результатов. До доступа к числам зафиксированы: три бюджета LLM (25 %, 40 %, 50 %); поправка Бонферрони на три теста ($\alpha/3 \approx 0,0167$ при $\alpha = 0,05$); тестовая выборка без пересечения товарных кодов (code-disjoint) ($n = 1539$)⁴;

⁴Размер $n = 1539$ — предзарегистрированный снимок (preregistered snapshot), материализованный артефактом datasets/processed/router_train.parquet на момент коммита cd9ac7a (2026-05-13 02:11) и зафиксированный в router_pareto_gold.parquet (03:22 того же дня). Последующие обновления файлов {cat}_gold_split.parquet (повторная разметка эталона, +35 строк) при текущем повторном запуске дают $n_{\text{test}} = 1574$; принцип предварительной регистрации требует сохранять зарегистрированное значение, поэтому в тексте сохранено $n = 1539$. Дрейф

опорная конфигурация — статическое правило. Все параметры зафиксированы в `src/eval/router_pre_registered.py` (константы `PRE_REGISTERED_BUDGETS`, `BONFERRONI_N`, функция `evaluate_h1`). Коммит модуля (`cd9ac7a`, 2026-05-13 02:11) сделан за 1 ч 11 мин до создания артефакта `router_pareto_gold.parquet` (03:22 того же дня), что фиксирует временной приоритет процедуры над данными.

Гипотеза H1 зафиксирована в git-коммите `cd9ac7a` от 2026-05-13 до получения окончательных результатов оценки без пересечения товарных кодов (`code-disjoint`). Платформа предварительной регистрации (OSF, AsPredicted) не использовалась; фиксация через коммит репозитория обеспечивает воспроизводимость и временной приоритет, но не равна третьей стороне публикации протокола.

3.3.23 Результаты

Сравнительные числа на выборке без пересечения товарных кодов (`code-disjoint`) представлены в таблице 3.3.5.1 (`router_pareto_gold.parquet`; для пяти моделей запасного слоя — `router_pareto_gold_{model}.parquet`).

Таблица 14 – Сравнение конфигураций маршрутизации на проверочной выборке без пересечения товарных кодов (`code-disjoint`)

Конфигурация	Точность	Доля вызовов LLM	Тип
Только каскад (без LLM)	78,5 %	0 %	опорная конфигурация
Статический порог $\tau = 0,55$	82,6 %	11 %	статический
Обучаемый маршрутизатор (оптимум)	82,7 %	16 %	обучаемый
Статическое поатрибутное правило	83,9 %	58 %	статический

составляет $\approx 2,3\%$ и не меняет качественный вывод H1.

Приведены точки минимальной стоимости при сопоставимой точности. Проверка H1 — сравнение точностей при равных бюджетах 25 %, 40 %, 50 %. При сопоставимой точности обучаемый маршрутизатор требует более высокого бюджета LLM: 16 % против 11 % при разнице точностей 0,1 п. п. в пределах доверительного интервала ($n = 1\,539$).

Ни на одном из трёх бюджетов маршрутизатор не превзошёл опорную конфигурацию на уровне $\alpha/3$ после поправки Бонферрони. Гипотеза H1 отклонена⁵.

При $n = 1539$, скорректированном уровне значимости $\alpha/3 \approx 0,0167$ (поправка Бонферрони на три бюджетных режима) и целевой мощности 0,8 минимальный обнаруживаемый эффект (MDE) для парного теста составляет $\approx 4,4$ п. п.⁶ (нормальная аппроксимация при средней точности конфигураций $p \approx 0,83$; точный McNemar-тест по симуляции даёт более оптимистичную оценку $\approx 3,0$ п. п. при типичной доле discordant-пар 10 %). Отрицательный исход H1 при таком размере выборки следует интерпретировать как отсутствие практически значимого превосходства обучаемого маршрутизатора над статическим правилом в пределах не менее 4 п. п.; меньшие сдвиги настоящей оценкой не отвергаются и не подтверждаются.

3.3.24 Интерпретация и следствие для финальной конфигурации

Отклонение H1 воспроизводится на всех пяти моделях Слой 4 и объясняется тремя наблюдениями. Во-первых, основная вариация надёжности каскада сосредоточена на уровне атрибутов, а не товаров; статическое `reg`-(категория, атрибут) правило учитывает именно эту неоднородность, маршрутизатор уровня ячейки дополнительной информации не привносит. Во-вторых, выборка без пересечения товарных кодов (`code-disjoint`) устраняет утечку бренда, объяснявшую ранние положительные результаты на случайных разбиениях. В-третьих, результат согласуется с RouterBench [ref_12]: обучаемый маршрутизатор не доминирует над простыми правилами

⁵Численные значения `p-value` McNemar-теста для бюджетов 25 %, 40 %, 50 % воспроизводятся ячейкой 9.1 в `notebooks/03_evaluate.ipynb`, источник — предзарегистрированный артефакт `datasets/processed/router_stats_gold.parquet` (коммит `cd9ac7a`).

⁶MDE устойчив на диапазоне $n \in [1539, 1635]$: при текущем повторном запуске на $n_{\text{test}} = 1574$ оценка по той же нормальной аппроксимации составляет $\approx 4,35$ п. п., округление до 4,4 п. п. сохраняется. Чувствительность проверена в `notebooks/03_evaluate.ipynb`, ячейка 9.1.

при локализованной структуре ошибок.

В финальной конфигурации применяется статическое правило: оно прозрачно, поддаётся аудиту и не требует переобучения. Отклонение H1 — один из трёх пунктов научной новизны работы (см. введение).

3.3.25 Цикл «аудит — правки экстрактора — переобучение — измерение»

Замкнутый цикл итеративного улучшения каскада построен на сопоставлении собственных предсказаний с независимо проаудированной разметкой. Измерение по неизменному списку аудированных ячеек исключает селекционный сдвиг.

3.3.26 Аудит pasta

Для pasta сформирован эталонный набор из 239 продуктов; LLM-аудитор `anthropic/claude-opus-4` слепо вынес суждения по каждой ячейке. Артефакт `cascade_vs_audited_gold_pasta.json` — 1841 ячейка по 8 атрибутам; доля `override + manual_only` — 220/1841 (11,95 %).

Главное обнаружение — концентрация ошибок на одном атрибуте: 24,2 % значений `pasta_shape` ошибочны. Корень — функция `_type_b_multiclass` обходила `categories_tags` в порядке следования, а не приоритета специфичности: для итальянской, испанской и немецкой пасты обобщённый `en:noodles` опережал специфические `en:tagliatelle`, `en:penne`, `en:fusilli`. Дополнительно: `TYPE_B` не содержала `legume-pasta` и `filled-pasta`, `regex` не знал многоязычные синонимы форм.

3.3.27 Правки экстрактора

Внесены точечные правки (коммиты `c1a0815`, `4bbd9e2`): `_type_b_multiclass` обходит отображение по убыванию приоритета (`en:noodles` — последним); `TYPE_B` и `regex` пополнены `legume-pasta/filled-pasta` и многоязычными синонимами. Каждый случай покрыт модульным тестом (28 тестов). Прочих изменений архитектуры и гиперпараметров не делалось.

3.3.28 Прирост точности эталонной разметки

После правок серебро перестроено; на тех же 1841 ячейке старые и новые значения сопоставлены с эталоном аудита.

Таблица 15 – Без подписи

Атрибут	n	Серебро до	Серебро после	Δ
pasta_shape	228	27,6 %	59,6 %	+32,0 п. п.
grain_type	233	55,4 %	59,2 %	+3,9 п. п.
6 нетронутых атрибутов	—	—	—	0
Всего	1841	80,7 %	85,2 %	+4,5 п. п.

Прирост сосредоточен на атрибуте-мишени правок (pasta_shape), что подтверждает каузальную связь. Прочие атрибуты не сдвинуты, как и ожидается при точечной правке.

3.3.29 Переобучение моделей и прирост каскада

Слой 2 переобучен на обновлённом серебре с пересохранением калибровок и порогов; каскад прогнан на тех же 239 эталонных продуктах.

Сравнение

`cascade_vs_audited_gold_pasta.pre-retrain.json` — против `cascade_vs_audited_gold_pasta.json`.

Таблица 16 – Без подписи

Срез	n	До переобучения	После переобучения	Δ
all_audited	1841	81,9 %	86,4 %	+4,5 п. п.
override □ manual_only	220	39,5 %	57,7 %	+18,2 п. п.
confirmed	1621	87,6 %	90,3 %	+2,7 п. п.

Эффект локализован: на pasta_shape точность на подмножестве override □ manual_only (83 ячейки) выросла с 22,9 % до 72,3 % (+49,4

п. п.). Точечная правка даёт концентрированный сдвиг: +18,2 п. п. на сложном срезе против +2,7 на confirmed.

3.3.30 Кросс-категорийная репликация цикла 3/3

Тот же цикл применён к chocolate и cheeses (cross_domain_audit_summary.json).

В chocolate (1530 ячеек) аудит выявил дефект бинаризации cocoa_percentage — расхождение конвенции серебра и индустриальной интерпретации метки «70 %». После перехода на конвенцию [X, Y) точность на override □ manual_only cocoa_percentage поднялась с 10,8 % до 31,5 %; агрегированный прирост по домену +16,3 п. п.

В cheeses (1655 ячеек) аудит обнаружил систематически завышенные пороги fat_class (86 сдвигов вверх против 16 обратных). После установки порогов (15, 20, 28) г/100 г точность на override □ manual_only fat_class выросла с 2,7 % до 69,9 %; агрегированный прирост по домену +14,4 п. п.

Цикл воспроизведён 3/3 на независимых доменах с разными типами дефектов (порядок обхода тегов, сдвиг границ бакетирования, выбор порогов численного атрибута). Это позволяет считать последовательность «обнаружение — правка — регенерация серебра — переобучение — измерение» переносимым приёмом, а не разовым улучшением.

3.3.31 Дополнительные проверки устойчивости

Дополнительно проведены проверки, исключаяющие характерные угрозы валидности: смещение по языку, тривиальность задачи относительно лексического базиса, близость тестовой семантики к обучающей.

3.3.32 Устойчивость каскада по языкам

OFF имеет языковой перекосяк (≈40 % FR, 10 % EN, 7 % DE и т. п.). Под балансировку каскад не оптимизировался. Поатрибутно-поязыковой срез (per_language_eval.parquet, 120 записей на исходной выборке 3257 ячеек, зафиксированной до extended consensus rerun 2026-05-27) даёт взвешенную точность⁷:

⁷Языковой срез не пересчитывался после расширения эталона до 16 360 ячеек, поскольку

Таблица 17 – Без подписи

Язык	Ячеек	Точность каскада
en	445	88,1 %
прочие	1014	88,0 %
it	1123	87,4 %
fr	856	86,8 %
de	433	86,1 %
es	479	85,2 %

Разброс между лучшим (en, 88,1 %) и худшим (es, 85,2 %) — 2,9 п. п. Это говорит о приемлемой языковой устойчивости и согласуется с выбором кодировщика `paraphrase-multilingual-mpnet-base-v2` с TF-IDF-расширением. Усреднение скрывает локальные провалы: четыре кортежа (категория, атрибут, язык) с отставанием от лучшего > 28 п. п.: `cheeses/texture/es` (17,6 % против 65,0 % на $n_{es}=47$ товаров), `pasta/pasta_shape/de` (52,4 % против 87,0 %), `cheeses/is_ultra_processed/es`, `cheeses/texture/de`. Концентрация на испанском сегменте `cheeses` связана со структурной особенностью данных (низкое покрытие правилами слоя 1 для `es-cheeses`); рекомендация для `production` — на испанском `cheeses`-каталоге снизить порог τ_a или явно роутить в LLM. Подробности — в разделе «Перспективы развития темы» заключения.

3.3.33 Сравнение с тривиальным лексическим базисом

Лексический базис (`TfidfVectorizer(max_features=5000, ngram_range=(1, 2)) + LogisticRegression`) обучен на серебре за вычетом тестовых кодов и оценён на том же срезе LLM-consensus gold (`off_leakage_probe_v4.parquet`). На нутриент-производном `pasta/protein_class` каскад превышает базис на 25–30 п. п. — зона нетривиальной ML-ценности. На текстово-производных каскад на 20 из 20 пар сопоставим с базисом или превышает его на срезе без пересечения товарных кодов (code-disjoint)

требует пересборки `per_language_eval.parquet` на удалённой машине (необходим OFF dump). Качественные выводы — баланс точности по 5 языкам (разброс 3 п. п.) и наличие категориально-специфичных просадок (`cheeses/texture/es`) — устойчивы к смене размера выборки в 5 раз; пересчёт перенесён в перспективу развития темы (см. перечень в Заключении).

(headline_v4_mpnet_tfidf_noleak.parquet).

3.3.34 Семантическая несмежность тестовой и обучающей выборки (k-NN-абляция)

Brand-disjoint разбиение не гарантирует, что тестовые бренды семантически далеки от обучающих. Для проверки для каждого тестового продукта вычислены $k = 5$ ближайших обучающих продуктов (косинусное расстояние SBERT); тестовые товары сгруппированы в четыре квартиля по медианному расстоянию (knn_distance_ablation.parquet).

Таблица 18 – Сквозная точность каскада по квартилям медианного расстояния до $k = 5$ ближайших обучающих соседей (косинусное расстояние)

Квартиль	Диапазон расстояния	Тестовых товаров	Ячеек	Сквозная точность каскада, %
Q1 (ближе всего)	0,05–0,18	61	359	82,7
Q2	0,18–0,22	61	364	89,3
Q3	0,22–0,26	61	365	90,7
Q4 (дальше всего)	0,26–0,57	61	370	91,6

Точность не падает на удалённых товарах; напротив, монотонно растёт от Q1 (82,7 %) до Q4 (91,6 %). Это опровергает гипотезу о близости тестовой семантики как источнике точности. Тенденция сохраняется по категориям (pasta 92,0 → 94,2; chocolate 79,4 → 83,6; cheeses 76,3 → 84,3): в Q4 попадают семантически отчётливые товары с уникальными текстовыми признаками.

Все три проверки подтверждают приемлемое поведение каскада по типичным угрозам валидности и фиксируют оставшиеся слабые места, направляющие план развития.

3.3.35 Уточнение схемы атрибутов: ортогональные свойства и отказ от мёртвых классов

При начальной разметке атрибут chocolate_type имел домен значений {dark, milk, white, filled, other}. Анализ ошибок на

LLM-consensus gold (n=158) показал, что значение `filled` функционирует как структурная характеристика товара (наличие начинки), независимая от типа какао-массы. Один и тот же шоколад может одновременно характеризоваться как `dark` (по типу) и `filled` (по форме): такая пара встречается в продукции Lindt (тёмные трюфели с ганашевой начинкой), Ritter Sport (Marzapane), Ferrero (Rocher). Кросс-таблица значений `chocolate_type=filled` × `chocolate_extra` подтвердила ортогональность: лишь 18 % товаров с `chocolate_type=filled` имели `chocolate_extra=filled`, остальные сочетались с `with_nuts`, `with_cookie`, `with_fruit` и т.д. Объединение двух ортогональных свойств в один multiclass-атрибут приводит к потере информации и снижению macro-F1: 5-классовая формулировка давала 0,50 за счёт нулевого recall на `filled` (0,33) и `other` (0,00).

По результатам анализа сформулированы три принципа проектирования схемы.

Принцип ортогональности атрибутов. Свойства, проявляющиеся у товара независимо друг от друга, выделены в отдельные атрибуты, а не объединены в один multiclass-домен. В схему введён бинарный атрибут `chocolate.is_filled` (структурная форма), независимый от `chocolate_type` (тип какао-массы) и `chocolate_extra` (включения в шоколадной массе). После разделения: `chocolate_type` (3-class) — macro-F1 0,971; `is_filled` (binary) — macro-F1 0,910, balanced accuracy 0,940; `chocolate_extra` (5-class) — macro-F1 0,857.

Принцип отказа от мёртвых классов. Класс multiclass-атрибута удаляется из схемы при одновременном выполнении условий: $n_{\text{train}} < 100$ (низкая представленность), $\text{recall} < 0,20$ на independent eval, семантическая некогерентность (catch-all без устойчивого паттерна в признаках или покрытие другим атрибутом), отсутствие downstream-зависимости. На основании критериев удалены: `chocolate_type.other` ($n_{\text{train}}=67$, $\text{recall}=0,00$), `chocolate_extra.other` ($\text{recall}=0,00$), `chocolate_extra.with_alcohol` и `chocolate_extra.with_coffee` ($n=1$ на eval, $\text{recall}=0,00$), `chocolate.flavor_profile.other` ($n_{\text{train}}=72$, $\text{recall}=0,11$), `cheeses.texture.other` ($n_{\text{train}}=14$, $\text{recall}=0,14$). Класс `semi_soft` в `cheeses.texture` объединён с `soft` как не несущая информации граница,

провоцирующая ошибки annotator-agreement.

Принцип консервативной агрегации rule-based слоёв. Шаблон в high-precision слое (`rule_h`) допускается только при однозначности ответа во всех контекстах. Региональные названия с неоднозначным таксономическим положением (*Coeur de Savoie* — встречается как *soft* и *hard* в зависимости от производителя) удалены из правил и переданы на ML-слой. В правила класса `cream` для `cheeses.texture` включены сыры однозначно относящиеся к этому типу по *dairy taxonomy* (*ricotta*, *labneh*, *quark*, *mascarpone*, *philadelphia*), а также бренд *Tartare* (*Savencia*, *herbed cream cheese*, ошибочно ранее отнесённый к *processed*).

Применение принципов реализовано двойным фильтром: на этапе обучения параметр `exclude_classes` в `CATEGORY_CONFIG` (`src/pipeline/ml/train.py`) удаляет метки из обучающей выборки до `LabelEncoder.fit`; на этапе оценки тот же набор применяется как `SCHEMA_EXCLUDE` в `src/eval/metric_table.py` и `src/eval/end_to_end.py` — *gold*-ячейки с *deprecated* значениями отбрасываются. Соответствие `train` ↔ `eval` ↔ `production` обеспечивается единым описанием.

Метки атрибута `chocolate.is_filled` получены в режиме *semi-supervised*: позитивные примеры выведены из старой схемы (`chocolate_type=filled` □ `chocolate_extra=filled`) либо из срабатывания *regex*-шаблона `FILLED_CHOCOLATE_REGEX` (мультиязычные триггеры: *filled*, *truffle*, *praline*, *ganache*, *marzipan*, *fourré*, *relleno*, *gefüllt*, *ripieno* и др., а также *OFF*-теги `en:filled-chocolates`, `en:pralines`, `en:truffles`). На *silver*-выборке `chocolate_gold_v4_wide.parquet` получено 1481 позитивный пример (12 % от 12494; 834 из меток + 647 из *regex*-аугментации). Негативные метки в *gold*-выборке (181 на *LLM-consensus*, 45 на *human gold*) валидированы 2-*LLM*-консенсусом (*qwen3-max* + *deepseek-r1*): переопределение *negative* → *positive* производилось только при двустороннем согласии; число переопределений — 0, что подтверждает корректность *rule-based* вывода негативов.

3.3.36 Поатрибутные архитектурные решения

В подразделе зафиксированы три типовых архитектурных решения,

выявленных при диагностике (`cascade_preds*_v4_noleak.parquet`, `headline_v4_mpnet_tfidf_noleak.parquet`) и заложенных в финальную конфигурацию.

(а) Сужение Слой 1 `chocolate_type` до текста названия. Triggers `milk/lait/Milch/latte/leche` массово встречаются в составе тёмного и filled-шоколада (молочный порошок, лецитин), вызывая ложное отнесение к классу «milk». Поэтому `extract_chocolate_type` получает только `product_name + brands + quantity` без `ingredients_text` и отказывается при одновременном матче нескольких типов в названии (например, «Tablette Lait Noir Praliné»). Точность regex на сработавших ячейках 94,4 % при покрытии 52,6 %; сквозная `chocolate/chocolate_type` 98,0 %.

(б) Перенастройка порога Слой 2 на парах с высокой штатной долей отказа. Для `chocolate/chocolate_extra`, `cheeses/texture`, `cheeses/is_organic`, `cheeses/is_ultra_processed`, `pasta/grain_type` стандартный порог `find_best_threshold` отбрасывал 11–44 % ячеек при 95–99 % точности на покрытом срезе. Пороги индивидуально понижены до 0,40–0,65. Эффект особенно заметен на `cheeses/texture` (с 56 % покрытия при 0,6 до 99 % при 0,4, точность 94,9 %) и `chocolate/chocolate_extra` (с 67 % до 95 %). Пороги хранятся в `models/{категория}_stratified_thresholds.pkl`.

(в) Гибридные признаки SBERT + TF-IDF для лексически богатых атрибутов. Атрибуты `chocolate/chocolate_type` и `chocolate/contains_nuts` чувствительны к редким информативным n-граммам (`noir`, `au lait`, `fourré`, `noisettes`, `pralinés`). Эксперимент `chocolate_hybrid_features.parquet` на train-сплите без пересечения товарных кодов (code-disjoint) даёт +1,9 / +3,8 п. п. при добавлении разреженных TF-IDF (1, 2) топ-5000 к SBERT-эмбедингам на входе XGBoost. Включено через флаг `--with-tfidf` в `src/pipeline/ml/train.py`.

Таблица 19 – Сквозная точность каскада на парах со специальной поатрибутной настройкой (LLM-consensus gold v4, без пересечения товарных кодов (code-disjoint))

Пара (категория, атрибут)	Probe	Cascade	Тип решения
chocolate / chocolate_type	94,7 %	98,0 %	(а) сужение Слой 1 + (в) гибридные признаки
chocolate / chocolate_extra	76,6 %	92,5 %	(б) порог 0,9 — 0,65
chocolate / contains_nuts	90,4 %	89,0 %	(в) гибридные признаки + Слой 1 nut-markers
cheeses / texture	70,5 %	92,3 %	(б) порог 0,60 — 0,40
cheeses / is_organic	92,5 %	98,7 %	(б) порог 0,70 — 0,40
cheeses / is_ultra_processed	86,1 %	96,1 %	(б) порог 0,70 — 0,50
pasta / grain_type	91,9 %	94,2 %	(б) порог Слой 2 — 0,50

На парах с пониженным порогом Слой 2 LLM вызывается на ≤ 5 % ячеек, общая доля обращений к LLM по системе — 4,1 % (снижение стоимости 95,9 % по сравнению с «одна LLM на всё»).

3.4 Выводы по главе 3

Основные результаты:

- 1) Реализован пятислойный каскад (Слой 0 категорайзер, regex, ML XGBoost на SBERT, байесовский валидатор, LLM-fallback) с указанием слоя-источника на каждое предсказание; вердикт формируется как последовательное AND-условие на пороги слоёв.
- 2) Развёрнута трёхкомпонентная демонстрационная система (Go-шлюз, Python/FastAPI ML-сервис, статический HTML/JS) со стабильным контрактом API и подтверждённой

работоспособностью на трёх категориях.

- 3) Сквозная точность (E2E) 93,0 % на 16 360 ячейках LLM-consensus gold (20 атрибутов, без пересечения товарных кодов (code-disjoint)) в конфигурации «каскад + gemini-flash» — на 9,2 п. п. выше опорной all-sonnet (83,8 %) при стоимости в 580 раз ниже (доминирование по Парето). Верхняя оценка только-каскадной точности — 95,5 % (cascade-only micro-ассурасу на тех же ячейках, macro-F1 0,890). Каскад покрывает 97,3 % ячеек; на LLM приходится 4,1 % сложных ячеек (снижение стоимости 95,9 % по сравнению с «одна LLM на всё»). Консервативная нижняя оценка по human gold (n = 566 cascade-valid): E2E 86,7 %, cascade-only 91,3 % — операционный циркулярный сдвиг 4,2 п. п. (каскад) / 6,3 п. п. (E2E). В сценарии с открытой моделью каскад + gpt-oss-120b достигает 90,6 % — на 21,3 п. п. выше одиночного вызова той же модели. Полная таблица — в 3.3.2.
- 4) Поатрибутный разбор: Слой 2 — основной (71–90 % покрытия, 93,5–98,6 % точности); Слой 1 покрывает 18–23 % на pasta/chocolate с точностью 87–100 %; байесовский валидатор работает селективно (+38 п. п. на chocolate/contains_nuts); LLM добавляет +1,4–1,7 п. п. при 4,1 % обращений.
- 5) Зафиксированная до получения результатов H1 об обучаемом маршрутизаторе отклонена на трёх бюджетах после поправки Бонферрони; результат согласуется с RouterBench [ref_12].
- 6) Цикл «аудит — правки — переобучение — измерение» воспроизведён 3/3: pasta +18,2 п. п., chocolate +16,3 п. п., cheeses +14,4 п. п. на проблемных срезах.
- 7) Зафиксированы три принципа проектирования схемы атрибутов (ортогональность, отказ от мёртвых классов, консервативная агрегация rule-based слоёв; 3.3.7.4). Выделение бинарного атрибута chocolate.is_filled как структурного свойства, ортогонального к chocolate_type, повысило macro-F1 chocolate_type с 0,50 (5-class) до 0,973 (3-class) при macro-F1 is_filled 0,861 на новой бинарной задаче. Удаление мёртвых классов (recall < 0,20, n_{train} < 100) дополнительно повысило macro-F1 chocolate_extra и cheeses.texture.

- 8) Устойчивость по языкам (разброс 3 п. п. на 5 языках) и сравнение с лексическим базисом (20 из 20 пар сопоставимо или выше, +25–30 п. п. на нутриент-производных) подтверждены; поатрибутные решения 3.3.7.4 обеспечивают высокое покрытие ML-слоя при доле обращений к LLM 4,1 %.

4 ОПИСАНИЕ РЕЗУЛЬТАТОВ

4.1 Руководство пользователя

4.1.1 Роли и интерфейсы

Система рассчитана на три роли (анализ — 1.1). Каждая роль работает со своим интерфейсом и решает свой класс задач (таблица 4.1); распределение прав доступа ограничивает каждого пользователя точками управления его компетенции.

Таблица 20 – Соответствие ролей пользователей и интерфейсов системы

Роль	Интерфейс	Класс задач
Контент-менеджер	Веб-интерфейс демо	Выборочная проверка предсказаний, утверждение или отклонение значений атрибутов
Системный аналитик	Конфигурационные файлы порогов, REST API	Настройка порогов уверенности, мониторинг состояния сервиса
Инженер машинного обучения	Скрипты обучения, ноутбук с диагностиками	Переобучение классификаторов и байесовских сетей, прогон диагностик, обновление эталона

4.1.2 Контент-менеджер

Контент-менеджер работает через веб-интерфейс по адресу <http://127.0.0.1:8080> после запуска демонстрационного комплекса. Интерфейс состоит из формы ввода партнёр-доступных полей и блока результата с оценкой уверенности и отметкой слоя-источника на каждый атрибут (рис. 4.1).

Обогащение товарных данных

Каскад regex → ML → байес → LLM-fallback. Введите частично заполненный товар — система предскажет недостающие атрибуты.

Ввод товара

Режим определения категории

☒ Авто (классификатор) ☐ Вручную

Категория

Шоколад / десерты

Название товара *

Lindt Excellence Dark 70% Cocoa 100g

Бренд / производитель

Lindt

Состав

cocoa mass, sugar, cocoa butter, soya lecithin, vanilla extract

Количество

100 g

Примеры:

валидные:

Pasta — Barilla Chocolate — Lindt 70% Cheeses — Roquefort

с противоречиями:

White vs. 70% cocoa Wheat + gluten-free

Обогатить

Результат

Категория: chocolate

Авто (router): chocolate, p=0.97

OOD: нет

Закрыто: 6 / 7 атрибутов

LLM-fallback: 1 атрибут

Время ответа: 184 мс

ПРЕДСКАЗАНИЯ ПО АТТРИБУТАМ

АТТРИБУТ	ПРЕДСКАЗАНИЕ	СЛОЙ	УВЕРЕННОСТЬ
cocoa_content_class	70-85 %	regex	1.00
chocolate_type	dark	regex	1.00
contains_nuts	false	ml	0.93
is_organic	false	ml	0.87
contains_milk	false	ml	0.81
flavor_modifier	vanilla	bayes	0.74
filling_type	— будет дозаполнено	llm	—

Слои каскада: regex ml bayes m-fallback

Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с отметкой слоя-источника и оценкой уверенности

Рисунок 10 – Веб-интерфейс демонстрационной системы (схематично): форма ввода партнёр-доступных полей и блок результата с отметкой слоя-источника и оценкой уверенности на каждый атрибут

Отметка слоя-источника даёт основание для доверия к предсказанию: слой 1 и слой 4 детерминированы либо объяснимы, для слоя 2 и слоя 3 показателем служит уверенность. Приём введён (см. 1.3) для компенсации ограниченной интерпретируемости слоёв 2/3 на уровне отдельного товара.

4.1.3 Системный аналитик

Аналитик настраивает каскад через две точки. Первая — словари порогов `models/{category}_thresholds.pkl`: контролируют компромисс «точность ↔ покрытие» поатрибутно, изменение не требует переобучения, применяется при следующем старте сервиса (типовой инструмент тонкой настройки при изменении требований к качеству). Вторая — REST API: `GET /api/categories` (состав атрибутов), `GET /health` (агрегированное состояние) для интеграции с мониторингом.

4.1.4 Инженер машинного обучения

Инженер работает с модулями `src/` через `python -m src.<module>` и ноутбуком `notebooks/03_evaluate.ipynb` (исполняемый источник §3.3). Цикл переобучения устроен так, что выпуск новой версии моделей не требует перекомпиляции каскада: файлы в `models/` переписываются, ML-сервис при старте загружает обновлённые артефакты.

Переменные окружения (требуется перед запуском):

```
export DIPLOMA_ROOT=/path/to/ai_attributes          # обязательно
export OPENROUTER_API_KEY=<key>                    # только для
```

При отсутствии `OPENROUTER_API_KEY` каскад работает в режиме без LLM-запасного слоя (слои 1–3 продолжают функционировать); скрипты `src/eval/end_to_end.py` и `reproduce.sh` печатают предупреждение и пропускают вызовы языковой модели. Локальная альтернатива — Ollama с моделью `gpt-oss-120b`, требует переменной `OLLAMA_HOST=http://localhost:11434` вместо `OPENROUTER_API_KEY`.

Последовательность команд цикла переобучения:

```
source .venv/bin/activate
python -m src.data.label_silver --category pasta_stratified
OMP_NUM_THREADS=1 python -m src.pipeline.ml.train --category
OMP_NUM_THREADS=1 python -m src.pipeline.bayes.train --category
OMP_NUM_THREADS=1 python -m src.eval.run_experiments --category
OMP_NUM_THREADS=1 python -m src.eval.run_diagnostics
```

Цикл запускается еженедельно или ежемесячно (зависит от темпа поступления товаров) и занимает десятки минут на категорию без специализированных ресурсов.

4.2 Сценарии применения и порядок внедрения

4.2.1 Встраивание в бизнес-процесс магазина

Каскад занимает место промежуточного сервиса в маршруте товара от поставщика до витрины — вторая позиция четырёхзвенной цепочки 1.1, где

сосредоточено узкое место. Внедрение не требует перестройки соседних звеньев: ETL партнёрских фидов и PIM работают как раньше, изменяется лишь содержимое «процедуры обогащения». Цепочка обработки:

Партнёрский фид (CSV/JSON/XML) → ETL магазина (нормализация п
→ HTTP /api/enrich —
каскад (regex → ML → Bayes → LLM-fallback)
→ Контент-менеджер (выборочная проверка случаев слоя 4)
→ PIM (финальная карточка) → Витрина и поисковый индекс

Основной экономический эффект — перенос ручного труда из режима «закрывать каждое поле» в режим «проверять остаток после каскада». По 3.3.2 каскад покрывает 95,9 % ячеек без LLM, оставшиеся 4,1 % уходят в запасной слой; сквозная точность 93,0 % в конфигурации «каскад + gemini-flash» (на эталоне с consensus-разметкой, n=16 360), при оценке только каскадной части — 95,5 % (верхняя граница при идеальной маршрутизации категории). На полностью ручном эталоне (n=566 cascade-valid из 688 в-scope) консервативная оценка составляет 86,7 % сквозной точности.

4.2.2 Сценарий холодного старта новой категории

Для новой категории каскад деградирует до двух слоёв (слой 1 на общих шаблонах + слой 4) — рабочий, но неоптимальный режим: точность слоя 4 на нутриент-производных атрибутах ниже, чем у обученного слоя 2 (4.3.3), стоимость выше. По мере накопления сотен размеченных товаров подключается слой 2; при появлении устойчивых распределений по брендам — слой 3.

Электроника использована как контрольный домен холодного старта (эталон — поля технических характеристик PhoneDB [ref_45] без слабого надзора). Полноценная аудит-оценка по протоколу 3.3 вынесена в раздел «Перспективы развития темы» заключения.

4.2.3 Сценарий многоязычного каталога

paraphrase-multilingual-mpnet-base-v2 обеспечивает единое векторное пространство для пяти доминирующих в OFF европейских языков, что устраняет необходимость отдельных моделей под каждый язык. Устойчивость по языкам подтверждена в 3.3.7.1: разброс между лучшим и

худшим сегментом — 2,9 п. п. Каскад применим в международных площадках без удвоения затрат на поддержку.

4.2.4 Сценарий обогащения нутриентно-производных полей

Атрибуты, восходящие к числовым нутриентам, требуют точного числового ответа. Одиночный вызов LLM малопригоден из-за высокой частоты ошибок на числах (1.2). В каскаде такие атрибуты закрываются комбинацией слоя 1 (числовые шаблоны) и слоя 2 (классификация дискретных классов после бакетирования нутриентов). USDA Branded Foods используется на этапе обучения для сверки эталона яруса 0 (3.2.5); расширение каскада детерминированным USDA-слоем в режиме вывода — в разделе «Перспективы развития темы» заключения.

4.2.5 Применимость в смежных доменах

Архитектура не привязана к пищевому домену: правила слоя 1 переписываются для новой категории, обучение слоёв 2/3 — стандартная процедура. Репликация на электронике (схема атрибутов, серебро из PhoneDB, демонстрация холодного старта; подробности в разделе «Перспективы развития темы» заключения) подтверждает переносимость. Формальная аудит-оценка вынесена в перспективы развития.

4.3 Что позволяет использование разработки (характеристика достигнутых результатов)

Использование системы даёт пять измеримых эффектов в формате «повышено / ускорено / сокращено» со ссылками на 3.

4.3.1 Повышено: качество автоматического обогащения каталога

Сквозная точность на эталоне с LLM-consensus разметкой (n=16 360 ячеек) — 93,0 % в конфигурации «каскад + gemini-flash» (3.3.2; v4_e2e_router_eval.json). 95 %-й интервал Вильсона: [92,6; 93,4] для сквозной оценки и [95,2; 95,8] для отдельной оценки каскадной части (95,5 %), которая выступает верхней границей при идеальной маршрутизации категории. На полностью ручном эталоне (n=566 cascade-valid из 688 в-scope) консервативная оценка составляет 86,7 % сквозной точности и 91,3 % по

каскаду. Поатрибутный разрез по категориям приведён в таблице 4.2 (источник: `v4_metric_table_v2.json`).

Таблица 21 – Точность каскадной части по категориям (модель `gemini-flash`, эталон LLM-consensus, среднее по атрибутам категории)

Категория	Число атрибутов	Средняя точность
Паста	7	95,8 %
Шоколад	6	94,9 %
Сыры	7	95,8 %
Взвешенное среднее по каскаду	20	95,5 %

*Примечание. Источник: `v4_metric_table_v2.json` (LLM-consensus, extended consensus rerun 2026-05-27). Cells-weighted средние по 20 атрибутам (7 pasta + 6 chocolate + 7 cheeses). Воспроизводится в `notebooks/03_evaluate.ipynb`, ячейка `t42-percat`. Поатрибутные значения *micro-accuracy* и *macro-F1* — таблица 3.3.2.3 (методология §14.3).*

Сыры и паста показывают наиболее равномерное покрытие; шоколад слегка отстаёт за счёт многоклассовых атрибутов (`chocolate_extra`, `contains_nuts`). Различие между сквозной (93,0 %) и каскадной (95,5 %) оценкой объясняется ошибками маршрутизации категории на верхнем слое и долей None-ответов в полном e2e-режиме.

Сравнение с альтернативами: прямой Claude Sonnet 4.5 даёт 83,8 % (на 9,2 п. п. ниже сквозной оценки системы); прямой gpt-oss-120b — 69,3 % (на 25,5 п. п. ниже каскада с этой же моделью в роли слоя 4 и на 23,7 п. п. ниже сквозной оценки). Превосходство объясняется наличием специализированных слоёв: Слой 2 обучен на нутриент-производных атрибутах (Nutri-Score, класс белка, класс жирности), которые LLM не вычисляет точно из текста без доступа к числам.

4.3.2 Сокращено: стоимость обработки относительно опорной LLM-модели

Стоимость обработки одного товара через каскад + `gemini-flash` составляет малую долю стоимости прямого Sonnet 4.5: каскад обращается к LLM лишь на 4,1 % ячеек вместо 100 %, что даёт совокупное сокращение

стоимости на 95,9 % относительно конфигурации «все ячейки — LLM» (3.3.2). Экономия — за счёт обработки полей в порядке возрастания стоимости: Слой 1 (бесплатно) → Слой 2 (миллисекунды на CPU) → Слой 3 (локальный вывод) → Слой 4 только при отказе предыдущих. Стоимость пяти каскадных конфигураций относительно опорной «все ячейки — Sonnet 4.5» приведена в таблице 4.3 (источник: `v4_e2e_router_eval.json`).

Таблица 22 – Стоимость обработки одного товара относительно опорной конфигурации all-sonnet

Конфигурация	Стоимость относительно опорной	Снижение стоимости
Каскад + gemini-flash	0,0017	в 580 раз (за счёт дешёвой модели и доли LLM 4,1 %)
Каскад + gpt-4o	0,030	в 33 раза
Каскад + Claude Sonnet 4.5	0,041	в 24 раза (только за счёт сокращения доли LLM до 4,1 %)
Каскад + gpt-oss-120b (открытая модель)	0,0012	в 810 раз

Примечание. Источник: `v4_e2e_router_eval.json`. Базовое сокращение, обусловленное архитектурой каскада (без учёта удешевления модели), составляет 95,9 % — соответствует множителю ≈ 24 раза.

Расчёт абсолютной стоимости. При средней длине запроса 500 входных и 200 выходных токенов на одну ячейку и реалистичных тарифах поставщиков (Claude Sonnet 4.5 — 3 / 15 USD за 1 млн токенов вход / выход, Gemini 2.5 Flash — 0,075 / 0,30 USD, gpt-oss-120b через OpenRouter — около 0,10 USD за 1 млн токенов смешанной структуры) удельная стоимость одной ячейки составляет: 0,00450 USD для Sonnet 4.5, 0,0001125 USD для Gemini Flash, около 0,00007 USD для gpt-oss-120b. На характерный месячный объём 100 000 новых SKU $\times$ 20 атрибутов = 2 000 000 ячеек прямой вызов Sonnet 4.5 даёт расход 9 000 USD/месяц. Каскад + Gemini Flash при доле LLM 4,1 % эквивалентен расходу 9 USD/месяц ($\approx 0,10$ % от опорной конфигурации). Каскад + gpt-oss-120b в полностью локальном развёртывании — \$10 USD/месяц только на инфраструктуру вывода. Тем самым при объёме 100 000

SKU/месяц годовая экономия превышает 100 000 USD относительно прямого вызова Claude Sonnet 4.5.

На более крупном объёме 1 000 000 SKU/месяц (характерный масштаб маркетплейсов) экономия пропорционально увеличивается до миллиона USD в год; точные числа зависят от реального состава партнёрского потока и тарифной политики поставщика на момент внедрения.

4.3.3 Обеспечена: возможность развёртывания на собственных мощностях с открытой моделью

Система поддерживает gpt-oss-120b в роли слоя 4 — полностью локальное развёртывание без отправки партнёрских данных вовне. Сводно (3.3.2): сквозная точность каскад + gpt-oss-120b — 90,6 % (близка к gemini-flash); прямой gpt-oss-120b — 69,3 %; прирост от каскадной архитектуры — +21,3 п. п. (слой 2 закрывает атрибуты, на которых малая модель проигрывает); архитектурное сокращение стоимости — 95,9 % за счёт уменьшения доли LLM-вызовов до 4,1 % ячеек. При близкой к gemini-flash точности обеспечивается независимость от внешних API — критично для корпоративных сценариев с требованиями локального хранения данных.

4.3.4 Сокращён: ручной труд оператора каталога

В исходном процессе (1.1) оператор проверяет каждый атрибут — 100 % ручной работы. Система переводит труд в режим выборочной верификации: 95,9 % ячеек закрываются дешёвыми слоями автоматически; 4,1 % уходят в слой 4 и получают предсказание (часть требует верификации, обычно — помеченные байесовский валидатором); совокупная доля ручной верификации — 4–8 % в зависимости от порога валидатора (3.3.4). Ручная нагрузка сокращается в 12–25 раз; контроль качества сохраняется через отметку слоя-источника и доверительные индикаторы.

4.3.5 Повышена: устойчивость каталога к новым товарам

Тестирование без пересечения товарных кодов (code-disjoint, 3.2.4) подтверждает устойчивость к новым SKU: на 16 360 ячейках сквозная точность 93,0 % с интервалом Вильсона [92,6; 93,4] и каскадная точность 95,5 % с интервалом [95,2; 95,8]. Бренды между обучением и тестом пересекаются

на 82–85 % по категориям — brand-disjoint обобщение в работе не заявляется (см. ограничения в Заключение, §6.2). Дополнительно k-NN-абляция (3.3.7.3) показывает: точность не падает на удалённых от обучения товарах, а монотонно растёт от Q1 (82,7 %) до Q4 (91,6 %).

4.3.6 Резюмирующая таблица «повышено / ускорено / сокращено»

Сводный эффект разработки на семи показателях приведён в таблице 4.4 (источники: v4_e2e_router_eval.json, v4_metric_table_v2.json).

Таблица 23 – Сводный эффект разработки на ключевых показателях системы (повышено / сокращено / обеспечено)

Показатель	До разработки	После	Эффект
Сквозная точность системы	83,8 % (опорная конфигурация all-sonnet)	93,0 % (каскад + gemini-flash; 95,5 % по каскаду без учёта маршрутизации)	повышена на 9,2 п. п.
Относительная стоимость обработки (архитектурный вклад каскада)	1,000 (опорная конфигурация all-sonnet)	0,041 при той же модели слоя 4	сокращена на 95,9 % (≈ 24 раза)
Доля обращений к LLM	100 %	4,1 %	сокращена примерно в 24 раза
Ручная верификация оператором	100 % ячеек	4–8 % ячеек	сокращена в 12–25 раз
Точность на открытой модели	69,3 % (одиночный вызов gpt-oss-120b)	90,6 % (каскад + gpt-oss-120b)	повышена на 21,3 п. п.

Продолжение таблицы

Показатель	До разработки	После	Эффект
Поддержка многоязычности	отдельные модели на каждый язык либо предварительный перевод	5 рабочих языков в едином векторном пространстве SBERT	обеспечена без дополнительных моделей
Независимость от поставщика LLM	один интегрированный провайдер	поддержка пяти моделей с сопоставимой точностью	обеспечена взаимозаменяемость

Примечание. Источники: `v4_e2e_router_eval.json`, `v4_metric_table_v2.json`. Консервативная оценка на полностью ручном эталоне ($n=566$ cascade-valid из 688 в-scope) даёт 86,7 % сквозной точности и 91,3 % по каскаду; разрыв с consensus-эталонem объясняется учётом circular bias (методология §14.5).

Значения столбцов «До» и «После» получены на тестовой выборке без пересечения товарных кодов (code-disjoint) главы 3 и подтверждены артефактами `v4_e2e_router_eval.json` и `v4_metric_table_v2.json`.

4.4 Выводы по главе 4

- 1) Система допускает три сценария по выбору слоя 4: бюджетный коммерческий (gemini-flash) — наилучший баланс точности и стоимости; премиум-коммерческий (Sonnet 4.5, GPT-4o) — близкая точность при большей стоимости; локальное развёртывание с gpt-oss-120b — независимость от внешних API при сопоставимой точности.
- 2) Целевая аудитория — три роли: контент-менеджер (выборочная верификация через веб-интерфейс), системный аналитик (пороги через конфигурации и REST API), инженер ML (переобучение по мере поступления данных).
- 3) Совокупный эффект: архитектурное сокращение стоимости на 95,9

% (≈ 24 раза) относительно конфигурации «все ячейки — LLM» и ручной нагрузки в 12–25 раз при сквозной точности 93,0 % с интервалом Вильсона [92,6; 93,4] (95,5 % по каскадной части как верхняя граница при идеальной маршрутизации); консервативная оценка на полностью ручном эталоне — 86,7 %.

ЗАКЛЮЧЕНИЕ

В ходе работы разработана гибридная каскадная система автоматизированного обогащения товарных данных для интернет-магазина. Система реализована в виде программного комплекса с демонстрационным стендом, проверена на выборке без пересечения товарных кодов (code-disjoint, новые SKU) и удовлетворяет шести функциональным, пяти нефункциональным и трём архитектурным требованиям ТЗ.

Достигнутые показатели превосходят целевые ориентиры ТЗ (≥ 85 % точности, $\geq 2,5\times$ сокращения стоимости): сквозная точность 93,0 % (точность только-каскадной части 95,5 % как верхняя граница при идеальной маршрутизации категории, macro-F1 0,890), стоимость сокращена в 580 раз относительно прямого вызова коммерческой Claude Sonnet 4.5 в режиме «каскад + Gemini 2.5 Flash» (комбинированное сокращение, включающее архитектурный вклад каскада в 24 раза). Каскад покрывает 97,3 % ячеек; в LLM направляется лишь 4,1 % сложных ячеек.

Сводные выводы по главам

Глава 1 систематизирует литературу и обосновывает противоречие. PIM-системы (Akeneo [ref_5], Pimcore [ref_6], Salsify [ref_7]) ориентированы на хранение и распределение, а появляющиеся модули автоматического обогащения — обёртки над одной LLM, не отвечающие требованиям больших каталогов. Академические подходы разделены: специализированные системы извлечения признаков (TXtract [ref_23], MAVE [ref_24], OpenTag [ref_25]) обеспечивают качество без учёта экономики; каскады LLM (FrugalGPT [ref_9], AutoMix [ref_10], Hybrid LLM [ref_11], RouterBench [ref_12]) дают инструменты управления стоимостью, но не интегрируют классическое ML. Из противоречия выведена гипотеза о каскаде разнородных слоёв и ТЗ.

Глава 2. Задача формализована как поатрибутное предсказание со схемой, допускающей явный отказ. Логика — каскад из пяти слоёв с последовательной фильтрацией: предсказание Слоя 2 принимается только при одновременном выполнении $P_{ML} \geq \tau_k$ и $P_B \geq \theta_k$ — формализация ансамблевой оценки уверенности как AND-условия. Архитектурный выбор каждой компоненты (SBERT, XGBoost, pgmpy, Go-шлюз + Python-ML, открытая gpt-oss-120b) обоснован эксплуатационно, методологически и

стратегически.

Глава 3. Каскад реализован как трёхкомпонентный комплекс (Go-шлюз, Python/FastAPI ML-сервис, статический веб-интерфейс); состав — 21 поатрибутный XGBoost (8/6/7 для pasta/chocolate/cheeses), три байесовские сети и Слой 0. На 16 360 ячейках без пересечения товарных кодов (code-disjoint) сквозная точность каскад + Gemini 2.5 Flash — 93,0 % (точность только-каскадной части 95,5 % как верхняя граница), на 9,2 п. п. выше прямого Sonnet 4.5 при стоимости в 580 раз ниже (комбинированное сокращение); в сценарии локального развёртывания каскад + gpt-oss-120b — 90,6 % (на 21,3 п. п. выше самостоятельного использования). Консервативная нижняя оценка на независимом эталоне ручной разметки Claude Opus 4 (n=566 cascade-valid из 688 в-scope) — 86,7 % сквозной точности. Слой 1 закрывает 18–23 % pasta/chocolate с точностью 87–90 % и 3 % cheeses со 100 % precision; Слой 2 — 71–90 % покрытия при 93,5–98,6 % точности; байесовский валидатор в производственной конфигурации включён точно только на chocolate/contains_nuts (см. ограничения ниже); LLM добавляет 1,4–1,7 п. п. при 4,1 % обращений. Зафиксированная до получения результатов Н1 о превосходстве обучаемого маршрутизатора отклонена. Цикл «аудит — правки — переобучение — измерение» воспроизведён 3/3 с приростом +18,2, +16,3 и +14,4 п. п.

Глава 4 переводит результаты в плоскость применения: три сценария по выбору слоя 4 (gemini-flash, Sonnet/GPT-4o, локально развёрнутая gpt-oss-120b); три ролевые модели (контент-менеджер, системный аналитик, инженер ML). Сводный эффект — архитектурное сокращение стоимости в 24 раза ($\approx 95,9$ %) относительно конфигурации «все ячейки — LLM» и комбинированное сокращение в 580 раз в режиме «каскад + Gemini 2.5 Flash»; сокращение ручной нагрузки в 12–25 раз при сквозной точности 93,0 % с интервалом Вильсона [92,6; 93,4] (95,5 % по каскадной части как верхняя граница при идеальной маршрутизации).

Заккрытие формулировок утверждённой темы

Утверждённая тема работы содержит восемь содержательных формулировок. Каждой из них в работе соответствует конкретный раздел, а также архитектурный или экспериментальный компонент разработки. Соответствие представлено в таблице 5.1.

Таблица 24 – Соответствие формулировок утверждённой темы и разделов работы

Формулировка из обоснования актуальности темы	Реализация в работе	Раздел
Гибридная каскадная система	Пятислойная архитектура каскада (Слой 0 + четыре содержательных слоя)	2.2, 3.1
Семантические возможности генеративных моделей	Запасной слой 4 на основе большой языковой модели	3.1.1
Предсказательная сила классического машинного обучения	Слой 2 — поатрибутные классификаторы XGBoost на многоязычных векторных представлениях SBERT	3.1.1
Ансамблевая оценка уверенности моделей	Многоуровневая фильтрация по уверенности как последовательное конъюнктивное условие на калиброванные пороги слоёв	2.1
Отбраковка фактических ошибок до их попадания в каталог	Слой 3 — байесовский валидатор плотностей пар «бренд × значение»	3.1.1, 3.3.4
Скрытые статистические закономерности каталога	Направленный ациклический граф атрибутов, обучаемый алгоритмом Hill Climb с критерием BIC	3.1.1

Продолжение таблицы

Формулировка из обоснования актуальности темы	Реализация в работе	Раздел
Разнородные источники данных (мультимодальные и многоканальные)	OCR-извлечение из изображений Open Food Facts и эмбединги CLIP на этапе обучения; нечёткое сопоставление с базой USDA Branded Foods для производных атрибутов	3.2.5
Интернет-магазин (партнёр — каталог — витрина)	Демонстрационный программный комплекс с REST-интерфейсом обработки партнёрского ввода и руководством пользователя	1.1, 3.1.2, 4.1

Таким образом, все восемь содержательных формулировок утверждённой темы имеют прямое отображение в разработанной системе и подтверждены экспериментально либо архитектурно. Наиболее нагруженные формулировки — отбраковка фактических ошибок и использование скрытых статистических закономерностей каталога — закрыты слоем байесовского валидатора. Для него в работе пересмотрена роль: вместо самостоятельного классификатора сеть используется как валидатор плотностей.

Научная новизна

Научная новизна работы складывается из трёх содержательных пунктов.

- 1) Доминирование каскадной архитектуры по Парето над всеми безусловными конфигурациями большой языковой модели на задаче обогащения товарных атрибутов. Одновременно достигается превышение точности на 9,2 процентных пункта (93,0 % против 83,8 %) и сокращение стоимости в 580 раз по сравнению с прямым обращением к Claude Sonnet 4.5 (комбинированное сокращение,

включающее архитектурный вклад каскада в 24 раза и удешевление модели запасного слоя). Результат подтверждён на проверочной выборке без пересечения товарных кодов (code-disjoint, новые SKU) из 16 360 ячеек и воспроизведён на пяти различных моделях запасного слоя. Это означает, что эффект сохраняется при смене поставщика генеративной модели.

- 2) Сценарий развёртывания на собственных мощностях с открытой большой языковой моделью. Каскад с открытой моделью gpt-oss-120b достигает 90,6 % сквозной точности — на 21,3 процентных пункта выше, чем самостоятельное применение той же модели (69,3 %). Результат демонстрирует способность каскадной композиции компенсировать слабость малых открытых моделей через специализированные слои. Кроме того, он формирует методологическую основу для локального развёртывания системы без передачи партнёрских данных во внешние коммерческие сервисы.
- 3) Зафиксированный до получения результатов отрицательный результат для обучаемого стоимостно-чувствительного маршрутизатора и кросс-категорийная репликация цикла «аудит — исправления — переобучение — измерение» в пределах продуктового домена. Гипотеза о превосходстве обучаемого XGBoost-маршрутизатора над статическим правилом отклонена на трёх бюджетах с поправкой Бонферрони. Этот результат согласуется с выводами бенчмарка RouterBench [ref_12]. Цикл аудита эталонной разметки воспроизведён на трёх категориях из трёх с количественно измеримым приростом сквозной точности.

Ограничения работы

Результаты ограничены рядом обстоятельств, добросовестно фиксируемых для формирования перспектив. Во-первых, область определения ограничена тремя категориями пищевой продукции; переносимость на другие домены не валидирована. Во-вторых, эталон построен сочетанием слабого надзора и согласия трёх LLM со слепой проверкой Orpus 4 — аудит-эталон второго уровня, ручная экспертная разметка не выполнялась. В-третьих, мультимодальные сигналы OCR и CLIP

используются только на обучении; в выводе работают только текстовые поля. В-четвёртых, проверка ограничена пятью европейскими языками, устойчивость к иным письменностям не валидирована. В-пятых, предварительная регистрация ограничена H1; показатели 3.3.2 — post-hoc анализ. В-шестых, разбиение train/test — code-disjoint (по идентификатору товара), бренды между обучающим пулом и тестовой выборкой пересекаются на 82–85 % по категориям; brand-disjoint обобщение в работе не заявляется. В-седьмых, измеренная точность подвержена циркулярному смещению от частичного семантического перекрытия серебряной разметки и LLM-консенсуса; операционная оценка смещения по разнице consensus и ручной разметки — 4,2 п. п. для каскадной части (95,5 % против 91,3 %) и 6,3 п. п. для сквозной цепочки (93,0 % против 86,7 %); консервативной независимой оценкой является показатель на ручной разметке Claude Opus 4 (86,7 % сквозной точности на n=566 cascade-valid).

Селективный характер байесовского валидатора (значимый прирост точности отбраковки на 3 парах из 20) фиксируется явно как ограничение архитектуры: из 20 пар «категория-атрибут» в производственной конфигурации валидатор включён точно только на одной паре chocolate/contains_nuts, где даёт +6,6 п. п. точности при росте расходов на 5,3 п. п. (3.3.4). На остальных 19 парах селективность сигнала недостаточна для архитектурного включения. Принцип «изменение роли компонента вместо его удаления» сформулирован в 3.1.1 и 3.3.4 как самостоятельный методологический вывод; вместе с тем тематически нагруженная формулировка «отбраковки фактических ошибок» в текущей реализации обеспечивается единственной парой и не имеет универсального селективного решения.

Перспективы развития темы

Полученные результаты и зафиксированные ограничения определяют ряд перспективных направлений развития темы.

- 1) Расширение области определения до шести категорий — напитки, зерновые завтраки и косметика. Для этих категорий подготовлены схемы атрибутов и серебряная разметка в файлах `beverages_stratified_silver_standard.parquet`, `cereals_stratified_silver_standard.parquet` и

`cosmetics_stratified_silver_standard.parquet`.

Основной blocker расширения — построение аудит-эталона уровня итоговой эталонной разметки v2 для каждой категории и обучение поатрибутных классификаторов Слой 2. Методология аудита и переобучения предположительно переносится на новые категории без принципиальных изменений.

- 2) Регулярная переаттестация изотонической калибровки на следующих итерациях эталонной разметки. Эффект схемы перекрёстной калибровки экспериментально проверен и реализован в коде системы (3.3.3.3, флаг `--calibration-method isotonic` в `src/pipeline/ml/train.py`). Средняя ЕСЕ снижается с 0,070 до 0,043, что даёт улучшение на 2,78 п. п. в среднем и положительный эффект на 16 из 20 атрибутов. Перспектива заключается в переаттестации после расширения эталонной разметки до большего числа атрибутов и категорий. Также целесообразно добавить регрессионный тест на ухудшение ЕСЕ более чем на 0,01 как условие принятия следующего цикла переобучения.
- 3) Введение мультимодального вывода в производственный режим — расширение прикладного интерфейса на приём изображений и добавление OCR/CLIP в реальном времени. Такой шаг требует пересмотра контракта взаимодействия с партнёром.
- 4) Запуск пилотного активного обучения на ячейках расхождения каскада и запасного слоя [ref_46; ref_39]. Логика построения очереди ячеек, где предсказание слоя 2 расходится с ответом запасного слоя, реализована в `src/experiments/active_learning_pilot.py`. Скрипт выполняет выборку 5 000 кодов на категорию, отбор 150 неуверенных и 150 случайных контрольных, а также повторное обучение с замером прироста на отложенной выборке. Перспектива заключается в проведении пилота в полном объёме на трёх категориях и в измерении прироста точности слоя 2 относительно случайной разметки той же стоимости.
- 5) Использование базы USDA Branded Foods и аналогов как структурированного второго источника для производных атрибутов

с систематизацией политики приоритетов между партнёрским вводом и вторичными каталогами.

- 6) Локализация на русский язык и российские каталоги — разработка русскоязычных регулярных выражений слоя 1 и дообучение слоя 2 на каталогах российских ритейлеров. Поскольку SBERT поддерживает русский в едином векторном пространстве, трудоёмкость ограничена слоями 1–2.
- 7) Перенос архитектуры в непродовольственные домены, начиная с электроники. Для смартфонов из каталога PhoneDB подготовлены схема атрибутов, скрипт построения серебряного стандарта и демонстрация холодного старта. Полноценное измерение точности требует построения аудит-эталоны уровня итоговой эталонной разметки v2 и оценки без пересечения товарных кодов (code-disjoint). Электроника сохранена как контрольный домен проверки переносимости и в основную таблицу 3.3 не входит.

Таким образом, разработанная гибридная каскадная система демонстрирует возможность одновременно достигать высокой точности и низкой стоимости обогащения товарных каталогов. Этот результат обеспечивается за счёт стратифицированного распределения нагрузки между разнородными по природе слоями обработки. Полученные результаты подтверждают выдвинутую в первой главе работы архитектурную гипотезу о каскадной композиции и закрывают все восемь содержательных формулировок утверждённой темы.

ПРИЛОЖЕНИЕ А

Электронные материалы

На рисунке А.1 приведён QR-код со ссылкой на репозиторий с исходным кодом, ноутбуком §3.3 в исполняемом виде, демонстрационным программным комплексом и текстом настоящей работы.



Рисунок А.1 – QR-код на репозиторий проекта